Full outline — Library Control Center

Full outline — Library Control Center

1. Page identity

Nav label

Library Control

Page title

Library Control Center

Subtitle

Central authority for governing, inspecting, maintaining, and directing the Axion library system.

Core role of the page

This page should act as the **top-level governing surface** for all libraries.
Its job is to show:

- what libraries exist
- which libraries are authoritative
- which libraries are weak, stale, or drifting
- what dependencies are blocked
- what maintenance action should run next
- what the operator should fix first

---

2. Page mission

This page should answer six control questions:

1. **What is the current state of the library estate?**
2. **Which libraries are safe to trust right now?**
3. **Which libraries are degraded, stale, or incomplete?**
4. **What is blocking system quality or downstream work?**
5. **What action should be taken next?**
6. **Which maintenance modes apply right now?**

That is a much better fit than the current phase-upgrade framing. The current implementation is organized around phase counts, average score, and phase-specific progress, which makes it a transformation tracker rather than a governing console.

knowledge-dashboard

---

3. Top-level page structure

Recommended top-to-bottom layout:

1. **Hero / Command Header**
2. **Estate Status Cards**
3. **Control Actions Strip**
4. **Library Health Overview**
5. **Library Estate Table**
6. **Library Detail Panels**
7. **Highest Leverage Fixes**
8. **Blockers and Risks**
9. **Library Governance Rules**
10. **Maintenance Modes**
11. **Dependency and Authority Map**
12. **Recent Library Activity**

---

4. Section-by-section outline

A. Hero / Command Header

Purpose

Immediately establish that this is the **control authority** over the library system.

Content

Left side:

- page icon
- **Library Control Center**
- subtitle

- one-line mission statement

Right side:

- estate control score
- authoritative library count
- degraded library count
- active blockers

Replace from current page

Replace the current:

- “Library Upgrade Matrix”
- artifact completion percent
- upgrade completion language
  knowledge-dashboard

New meaning

This header should communicate:

- not “how far through upgrades are we”
- but “what is the state of control over the library estate”

B. Estate Status Cards

Purpose

Provide a fast operational read of the system.

Recommended cards

- **Total Libraries**
- **Authoritative**
- **Needs Review**
- **Stale**
- **At Risk**
- **Blocked**
- **Missing Authority**
- **Drift Alerts**

Why this replaces the current cards

The current cards are:

- Libraries
- Avg Score
- Critical
- Phase 1
- Phase 2
- Phase 3
  knowledge-dashboard

Those are useful for an upgrade campaign, but not for runtime control.

C. Control Actions Strip

Purpose

Make the page actionable immediately.

Section title

Control Actions

Action buttons

- Run Health Check
- Run Coverage Audit
- Run Drift Detection
- Rebuild Registries
- Review Authority Gaps
- Resolve Blockers
- Refresh Dependencies
- Open Critical Libraries

Behavior

Each action should feel like a control-plane move, not just a report link.

Why this matters

The current page mostly tracks and displays; this strip should shift it into a governing surface.

D. Library Health Overview

Purpose

Provide a grouped system-health view of the full estate.

Section title

Library Health Overview

Layout

Use grouped bands or stacked cards:

- **Authoritative**
- **Stable**
- **Incomplete**
- **Stale**
- **Fragmented**
- **Untrusted**

Each band should show

- count of libraries
- names of affected libraries
- severity trend
- next control action

Replace from current page

This should replace the current **PhaseTimeline** and phase-centric mental model. The current page uses explicit phase progression, which is tied to phased upgrade work.

E. Library Estate Table

Purpose

This becomes the main operating grid for the page.

Section title

Library Estate

Columns

- Library
- Authority Status
- Coverage
- Freshness
- Integrity
- Dependency Pressure
- Last Audit
- Current Risk
- Recommended Action
- Control State

Recommended control states

- Governed
- Review Required
- Degraded
- Blocked
- Unsafe
- Recovery

Row actions

- Open Library
- Inspect Gaps
- Review Drift
- Audit Now
- Repair Registry Links
- Assign Maintenance Mode

Replace from current page

The current grid is based on:

- score
- phase
- priority
- tier
- done

- status

That should be replaced with estate-control fields.

F. Library Detail Panels

Purpose

Provide expandable command-level detail for each library.

Section title

Library Detail Panels

Each library panel should include

1. Identity

- library name
- purpose
- owner / governing authority
- estate role

2. Governance state

- authority source present or missing
- trusted / degraded / stale / unsafe
- policy coverage state

3. Health state

- completeness
- freshness
- integrity
- dependency fitness

4. Gap summary

- missing artifacts
- stale artifacts
- unresolved conflicts
- broken references
- unsupported templates or standards

5. Dependency surface

- upstream libraries it depends on
- downstream systems depending on it
- blockage impact

6. Recommended action

- best next step
- urgency
- suggested maintenance mode

7. Control actions

- audit
- drift check
- registry repair
- mark under review
- escalate
- open authority source

Replace from current page

The current expandable panel is **PhaseDetailPanel**, centered on phase title, target outcomes, expected results, and artifact checklists.

knowledge-dashboard

That should become **library-specific** instead of **phase-specific**.

G. Highest Leverage Fixes

Purpose

Show the operator the best next moves by impact.

Section title

Highest Leverage Fixes

Ranking logic

Prioritize fixes that:

- unblock many dependencies
- restore authoritative control
- reduce system-wide drift
- complete a critical artifact family
- remove unsafe governance conditions

Each item should show

- affected library
- problem summary
- impact radius
- effort estimate
- recommended action

Keep from current page

This is the best repurpose of the current “fastest lift” idea. The current file imports `fastestLiftOrder`, which suggests the page already has a ranked-value section worth preserving in spirit.

knowledge-dashboard

H. Blockers and Risks

Purpose

Surface what is actively preventing control quality.

Section title

Blockers and Risks

Split into two zones

Active Blockers

- missing canonical authority
- broken registry references
- incomplete artifact chain
- conflicting standards

- blocked dependency path

System Risks

- stale governing source
- partial library coverage
- drift between libraries
- no audit freshness
- unsupported downstream reliance

Keep from current page

The current page already has `hardBlockers`, and that concept should remain because it fits a control-center model well.

knowledge-dashboard

I. Library Governance Rules

Purpose

Define the permanent control rules for the estate.

Section title

Library Governance Rules

Example rules

- every governed subject must have one authoritative source
- no stale source may silently govern runtime decisions
- no template may govern without source backing
- registry integrity must pass before trust status is restored
- dependency-critical libraries require recent audit coverage
- unresolved conflicts force review state
- unsafe libraries cannot be marked governed

Repurpose from current page

This replaces the current **cross-phase rules** with evergreen estate-control rules. The current file imports `crossPhaseRules`, which shows there is already a rules section to convert.

J. Maintenance Modes

Purpose

Connect the control page to the actual maintenance system.

Section title

Maintenance Modes

Use the existing locked maintenance registry

Relevant modes include:

- **MM-01 — Health Check**
- **MM-03 — Coverage Audit**
- **MM-04 — Drift Detection**
- **MM-12 — Rebuild Indexes/Registries**
- **MM-13 — Registry Integrity Cleanup**
- **MM-17 — Dependency Refresh**
- **MM-18 — Workspace Compatibility Check**
- **MM-20 — Failure Triage Mode**
- **MM-21 — Emergency Recovery Mode**
  Response: Axion Updates Planning

Each mode card should show

- mode ID
- name
- purpose
- trigger condition
- affected libraries
- expected outputs
- launch action

Why this section matters

This is what upgrades the page from a passive monitor into a real command surface.

K. Dependency and Authority Map

Purpose

Show control relationships across the estate.

Section title

Dependency and Authority Map

Show

- which libraries govern which areas
- dependency chains between libraries
- authority bottlenecks
- single points of failure
- where stale or weak libraries create downstream risk

Visual options

- simple graph
- layered lane diagram
- grouped dependency cards

Value

This makes the page truly system-level rather than just a list of libraries.

L. Recent Library Activity

Purpose

Show recent meaningful control events.

Section title

Recent Library Activity

Show

- audits run

- drift detected
- authority restored
- maintenance launched
- blockers cleared
- libraries downgraded or restored
- registry repairs completed

Why it belongs

A control center should show recent control actions, not just current state.

---

5. Recommended interaction model

Primary controls

The page should support:

- filter by health state
- filter by authority state
- filter by risk
- filter by dependency pressure
- sort by urgency
- sort by impact
- expand library details
- launch maintenance modes
- open governing sources

Suggested quick filters

- Critical Only
- Stale Only
- Blocked Only
- Ungoverned
- Drifted
- Needs Audit
- Recovery State

---

6. What should be removed from the current version

These should no longer be the main structure:

- upgrade percent as primary headline metric
- phase counts
- phase timeline
- phase detail panels
- phase-oriented target outcomes
- average upgrade score as the main estate signal

These belong in a separate page such as:

- **Library Upgrade Planner**
- **Library Upgrade Matrix**
- **Library Transformation Plan**

That page can stay for planning, but not as the main control page.

---

7. Best final page identity

Nav

Library Control

Page title

Library Control Center

Role

The central governing surface for the full Axion library estate.

Design tone

This page should feel like:

- command authority
- oversight
- trust evaluation
- maintenance direction
- dependency governance

Not:

- spreadsheet tracker
- upgrade checklist
- planning-only dashboard

---

8. Clean final outline

If you want the shortest official version to build from, use this:

1. Hero / Command Header
2. Estate Status Cards
3. Control Actions
4. Library Health Overview
5. Library Estate Table
6. Library Detail Panels
7. Highest Leverage Fixes
8. Blockers and Risks
9. Library Governance Rules
10. Maintenance Modes
11. Dependency and Authority Map
12. Recent Library Activity

Library Control Center — Build Spec

Library Control Center — Build Spec

1. Page identity

Route

- keep current route temporarily if needed
- target route: `/library-control`

Component name

- `LibraryControlCenterPage`

Nav label

- `Library Control`

Page title

- `Library Control Center`

Subtitle

- `Central authority for governing, inspecting, maintaining, and directing the Axion library system.`

---

2. Page objective

This page is the **control surface for the full library estate**.

It must let the operator:

- see which libraries are authoritative
- see which libraries are degraded, stale, blocked, or unsafe
- identify dependency pressure
- identify authority gaps
- launch the right maintenance action
- inspect individual library governance state

---

3. Replace/remove from current page

The following current structures should be removed as primary framing because they are tied to the upgrade-program model:

- `Library Upgrade Matrix` hero/title
  knowledge-dashboard
- upgrade percent / artifact-completion hero block
  knowledge-dashboard
- phase count metric cards (`Phase 1`, `Phase 2`, `Phase 3`)
  knowledge-dashboard
- `PhaseTimeline` component
  knowledge-dashboard
- phase-based progress state like `openPhases` / `togglePhase`
- phase-driven detail panels
- `Cross-Phase Rules` label and framing
  knowledge-dashboard

These can be moved to a separate planning page later.

---

4. Preserve/reuse from current page

These current ideas are worth reusing, but under the new control model:

- `GlassPanel`
- `MetricCard`
- `StatusChip`
- row/grid/table layout patterns
- `hardBlockers` concept → becomes Blockers and Risks
  knowledge-dashboard
- `crossPhaseRules` concept → becomes Library Governance Rules
  knowledge-dashboard
- `fastestLiftOrder` concept → becomes Highest Leverage Fixes
  knowledge-dashboard

---

5. Final page layout

Section order

1. Hero Header
2. Estate Status Cards
3. Control Actions Strip
4. Library Health Overview
5. Library Estate Table
6. Selected Library Detail Panel
7. Highest Leverage Fixes
8. Blockers and Risks
9. Library Governance Rules
10. Maintenance Modes
11. Dependency and Authority Map
12. Recent Library Activity

---

6. Section specs

A. Hero Header

Purpose

Establish the page as the governing surface.

UI

Single large `GlassPanel` across the top.

Left side content

- icon: `Library`
- H1: `Library Control Center`
- subtitle text
- optional badge: `CONTROL AUTHORITY`

Right side content

Four compact summary values:

- Control Score

- Governed Libraries
- Active Risks
- Blocked Dependencies

Data fields

```
type EstateHeaderStats = {
 controlScore: number;          // 0-100
 governedLibraries: number;
 activeRisks: number;
 blockedDependencies: number;
}
```

Notes

This directly replaces the current upgrade-completion hero with control-state summary. The current hero is percent/artifact oriented, which is the wrong signal for the new page.

knowledge-dashboard

B. Estate Status Cards

Purpose

Fast operational snapshot.

UI

Grid of 6–8 `MetricCard`s.

Cards

- Total Libraries
- Governed
- Review Required
- Stale
- Unsafe
- Blocked
- Missing Authority
- Drift Alerts

Data fields

```
type EstateStatusMetrics = {
 totalLibraries: number;
 governed: number;
 reviewRequired: number;
 stale: number;
 unsafe: number;
 blocked: number;
 missingAuthority: number;
 driftAlerts: number;
}
```

Card behavior

Each card should support click-to-filter the main table.

C. Control Actions Strip

Purpose

Immediate control-plane actions.

UI

Horizontal action bar in a `GlassPanel`, using button groups.

Actions

- Run Health Check
- Run Coverage Audit
- Run Drift Detection
- Rebuild Registries
- Review Authority Gaps
- Refresh Dependencies
- Open Unsafe Libraries
- Resolve Blockers

Data fields

```
type ControlAction = {
 id: string;
 label: string;
 icon: string;
```

```
 actionType:
   | "health-check"
   | "coverage-audit"
   | "drift-detection"
   | "rebuild-registries"
   | "review-authority-gaps"
   | "dependency-refresh"
   | "open-unsafe"
   | "resolve-blockers";
 enabled: boolean;
}
```

Behavior

These can initially be stubbed buttons with handlers. Later they can connect to maintenance-mode launch flows.

D. Library Health Overview

Purpose

Show grouped health bands across the estate.

UI

Six band cards or columns.

Bands

- Authoritative
- Stable
- Incomplete
- Stale
- Fragmented
- Untrusted

Band card content

- count
- short description
- top affected libraries
- recommended action

Data fields

```
type LibraryHealthBand = {
 id: "authoritative" | "stable" | "incomplete" | "stale" | "fragmented" | "untrusted";
 label: string;
 count: number;
 libraryIds: string[];
 recommendedAction: string;
}
```

Behavior

Clicking a band filters the Library Estate table.

Notes

This is the replacement for `PhaseTimeline`, which is currently phase-oriented and no longer matches the page job.

knowledge-dashboard

E. Library Estate Table

Purpose

Primary operating surface for all libraries.

UI

Large table/grid in a `GlassPanel`.

Columns

- Library
- Authority Status
- Coverage
- Freshness
- Integrity
- Dependency Pressure
- Last Audit
- Risk
- Control State

- Recommended Action

Data model

```
type AuthorityStatus = "authoritative" | "partial" | "missing" | "disputed";
type ControlState = "governed" | "review-required" | "degraded" | "blocked" | "unsafe" |
"recovery";
type RiskLevel = "low" | "moderate" | "high" | "critical";

type LibraryControlRecord = {
 id: string;
 name: string;
 route?: string;
 authorityStatus: AuthorityStatus;
 coveragePercent: number;        // 0-100
 freshnessPercent: number;       // 0-100
 integrityPercent: number;       // 0-100
 dependencyPressure: number;     // 0-100
 lastAuditAt?: string;           // ISO date
 risk: RiskLevel;
 controlState: ControlState;
 recommendedAction: string;
 dependencyBlockers: string[];
 owner?: string;
}
```

Row actions

- Open
- Inspect
- Audit
- Drift Check
- Repair
- Assign Mode

Interaction

Clicking a row selects that library and populates the detail panel below or to the side.

Notes

The current row model is still built around upgrade concepts like score, phase, priority, tier, done, and status. That should be replaced with control-state fields.

F. Selected Library Detail Panel

Purpose

Deep inspection of one library at a time.

UI

Expandable side panel or full-width detail panel under table.

Sections inside panel

1. Identity

- library name
- purpose
- owner
- route
- estate role

2. Governance State

- authority source
- authority status
- trust decision
- governing artifacts count

3. Health State

- coverage
- freshness
- integrity
- audit freshness
- dependency fitness

4. Gap Summary

- missing artifacts
- stale artifacts
- broken refs
- unresolved conflicts
- unsupported downstream use

5. Dependency Surface

- upstream dependencies
- downstream dependents
- blocker impact

6. Recommended Action

- next best action
- urgency
- maintenance mode recommendation

7. Control Actions

- Audit Now
- Run Drift Detection
- Rebuild Registry Links
- Mark Under Review
- Escalate
- Open Source

Data model

```
type LibraryDetail = {
 id: string;
 name: string;
 purpose: string;
 owner?: string;
 estateRole?: string;
 route?: string;

 authoritySource?: string;
 authorityStatus: AuthorityStatus;
 trustDecision: "trusted" | "conditional" | "untrusted";

 coveragePercent: number;
 freshnessPercent: number;
 integrityPercent: number;
 dependencyFitnessPercent: number;

 missingArtifacts: string[];
 staleArtifacts: string[];
 brokenReferences: string[];
 unresolvedConflicts: string[];
 unsupportedDependents: string[];

 upstreamDependencies: string[];
```

```
 downstreamDependents: string[];
 blockerImpact: string[];

 recommendedAction: string;
 urgency: RiskLevel;
 suggestedMaintenanceMode?: string;
}
```

G. Highest Leverage Fixes

Purpose

Rank small-to-medium fixes with outsized estate impact.

UI

Ranked list cards.

Each item shows

- library
- issue
- impact radius
- effort
- payoff
- action button

Data model

```
type LeverageFix = {
 id: string;
 libraryId: string;
 title: string;
 summary: string;
 impactRadius: "local" | "cross-library" | "estate-wide";
 effort: "small" | "medium" | "large";
 payoffScore: number;            // rank weight
 recommendedAction: string;
}
```

Notes

This is the best reuse path for the current `fastestLiftOrder` idea.

H. Blockers and Risks

Purpose

Surface active control problems.

UI

Two-column layout.

Left column

Active Blockers

- missing authority source
- registry break
- dependency failure
- artifact family incomplete

Right column

System Risks

- stale governing source
- audit too old
- drift detected
- single point of authority failure

Data model

```
type ActiveBlocker = {
 id: string;
 title: string;
 severity: RiskLevel;
 items: string[];
 affectedLibraries: string[];
}

type SystemRisk = {
 id: string;
 title: string;
```

```
 severity: RiskLevel;
 summary: string;
 affectedLibraries: string[];
}
```

Notes

The current `hardBlockers` section already fits this direction and should be expanded rather than deleted.

knowledge-dashboard

I. Library Governance Rules

Purpose

Show permanent governing rules for the estate.

UI

Stacked rule cards.

Rules to include

- one authoritative source per governed subject
- stale authority cannot silently govern runtime
- no template without source backing
- registry integrity required before trust restoration
- unresolved conflicts force review-required state
- unsafe libraries cannot be marked governed

Data model

```
type GovernanceRule = {
 id: string;
 title: string;
 description: string;
 severity: "info" | "warning" | "strict";
}
```

Notes

This is a repurpose of the current `Cross-Phase Rules` block into evergreen governance rules.

knowledge-dashboard

J. Maintenance Modes

Purpose

Connect library control to maintenance execution.

UI

Card grid or horizontal scroller.

Modes to show first

Use the locked maintenance registry:

- MM-01 Health Check
- MM-03 Coverage Audit
- MM-04 Drift Detection
- MM-12 Rebuild Indexes/Registries
- MM-13 Registry Integrity Cleanup
- MM-17 Dependency Refresh
- MM-18 Workspace Compatibility Check
- MM-20 Failure Triage Mode
- MM-21 Emergency Recovery Mode
  Response: Axion Updates Planning

Data model

```
type MaintenanceModeCard = {
 id: string;                    // "MM-01"
 name: string;
 purpose: string;
 triggerCondition: string;
 suggestedForStates: ControlState[];
 launchable: boolean;
}
```

K. Dependency and Authority Map

Purpose

Visualize cross-library control relationships.

UI

Can be phase 2 of implementation if needed. For v1, a summarized graph/list is enough.

Show

- authoritative library nodes
- dependent library links
- bottlenecks
- orphaned libraries
- disputed authority chains

Data model

```
type DependencyLink = {
 fromLibraryId: string;
 toLibraryId: string;
 relation: "depends-on" | "governs" | "blocks" | "supports";
 severity?: RiskLevel;
}
```

L. Recent Library Activity

Purpose

Show recent estate-control events.

UI

Timeline list.

Events

- audit run
- drift detected
- authority restored
- blocker cleared

- registry rebuilt
- library downgraded to review-required
- library restored to governed

Data model

```
type LibraryActivityEvent = {
 id: string;
 timestamp: string;
 type:
   | "audit-run"
   | "drift-detected"
   | "authority-restored"
   | "blocker-cleared"
   | "registry-rebuilt"
   | "downgraded"
   | "restored";
 libraryId?: string;
 title: string;
 summary: string;
}
```

---

7. Filters and controls

Global filters

Add a filter bar above the Library Estate table.

Filters

- search by library name
- authority status
- control state
- risk
- stale only
- blocked only
- missing authority only
- drift alerts only

Sort options

- risk desc
- dependency pressure desc
- freshness asc
- coverage asc
- last audit oldest
- control score asc

Data model

```
type LibraryControlFilters = {
 query: string;
 authorityStatus?: AuthorityStatus[];
 controlState?: ControlState[];
 risk?: RiskLevel[];
 staleOnly?: boolean;
 blockedOnly?: boolean;
 missingAuthorityOnly?: boolean;
 driftOnly?: boolean;
 sortBy?:
   | "risk"
   | "dependency-pressure"
   | "freshness"
   | "coverage"
   | "last-audit"
   | "control-score";
}
```

---

8. Derived metrics and formulas

Library control score

Use a weighted score to simplify status display.

```
controlScore =
 coveragePercent * 0.30 +
 freshnessPercent * 0.20 +
 integrityPercent * 0.25 +
 (100 - dependencyPressure) * 0.10 +
 authorityScore * 0.15
```

Where:

- authoritative = 100
- partial = 60
- disputed = 30
- missing = 0

Suggested control-state thresholds

- `governed` = 85+
- `review-required` = 70–84
- `degraded` = 50–69
- `blocked` = dependency blocker present
- `unsafe` = missing/disputed authority plus high/critical risk
- `recovery` = active remediation underway

---

9. Suggested source file split

Page file

- `App/src/pages/library-control.tsx`

Data

- `App/src/data/library-control.ts`

Hooks

- `App/src/hooks/use-library-control.ts`

Optional components

- `App/src/components/library-control/health-overview.tsx`
- `App/src/components/library-control/library-estate-table.tsx`
- `App/src/components/library-control/library-detail-panel.tsx`
- `App/src/components/library-control/maintenance-modes.tsx`

Migration note

The current page imports from `../data/upgrade-matrix` and `../hooks/use-upgrade-progress`; that should shift to a control-oriented data source and hook.

---

10. Suggested TypeScript master model

```
export type LibraryControlCenterState = {
 header: EstateHeaderStats;
 metrics: EstateStatusMetrics;
 healthBands: LibraryHealthBand[];
 libraries: LibraryControlRecord[];
 selectedLibrary?: LibraryDetail;
 leverageFixes: LeverageFix[];
 blockers: ActiveBlocker[];
 risks: SystemRisk[];
 governanceRules: GovernanceRule[];
 maintenanceModes: MaintenanceModeCard[];
 dependencyLinks: DependencyLink[];
 recentActivity: LibraryActivityEvent[];
 filters: LibraryControlFilters;
}
```

---

11. Build priority

Phase A — must build first

- Hero Header
- Estate Status Cards
- Control Actions Strip
- Library Health Overview
- Library Estate Table
- Selected Library Detail Panel

Phase B

- Highest Leverage Fixes
- Blockers and Risks

- Library Governance Rules
- Maintenance Modes

Phase C

- Dependency and Authority Map
- Recent Library Activity

---

12. Acceptance criteria

The page is complete when:

- nav label reads `Library Control`
- page title reads `Library Control Center`
- no phase-based upgrade framing remains on the main page
- main table is control-state driven, not phase driven
- clicking a library opens usable detail state
- at least 5 maintenance modes are visible
- blockers and governance rules are visible
- the operator can identify unsafe, stale, and blocked libraries in one pass

---

13. Clean replacement mapping

Current → New

- `Library Upgrade Matrix` → `Library Control Center`
  knowledge-dashboard
- hero progress percent → control score / governed count
  knowledge-dashboard
- phase metric cards → estate status cards
  knowledge-dashboard
- `PhaseTimeline` → Library Health Overview
  knowledge-dashboard
- `All Libraries` table → Library Estate table

- `hardBlockers` → Blockers and Risks
  knowledge-dashboard
- `Cross-Phase Rules` → Library Governance Rules
  knowledge-dashboard
- `fastestLiftOrder` → Highest Leverage Fixes
  knowledge-dashboard

data schema pack

Below is the **data schema pack** for `library-control.ts`.

One constraint up front: the uploaded page shows that the current screen consumes a `libraries` dataset from `../data/upgrade-matrix`, and it confirms there are **16 libraries**, but that source file itself was not uploaded here, so I cannot reliably recover the exact current 16 library names from the file alone. The current page only proves the count and the old upgrade-oriented fields/sections.

knowledge-dashboard

knowledge-dashboard

knowledge-dashboard

So this pack gives you:

- the **full control-center schema**
- **16 starter records**
- a **clean placeholder naming pattern** you can immediately replace with your real library names
- maintenance mode cards aligned to your locked registry
  Response: Axion Updates Planning

App/src/data/library-control.ts

```
// App/src/data/library-control.ts

export type AuthorityStatus = "authoritative" | "partial" | "missing" | "disputed";
export type ControlState =
 | "governed"
 | "review-required"
 | "degraded"
 | "blocked"
 | "unsafe"
 | "recovery";

export type RiskLevel = "low" | "moderate" | "high" | "critical";

export type TrustDecision = "trusted" | "conditional" | "untrusted";

export type HealthBandId =
 | "authoritative"
 | "stable"
```

```
 | "incomplete"
 | "stale"
 | "fragmented"
 | "untrusted";

export type ActivityType =
 | "audit-run"
 | "drift-detected"
 | "authority-restored"
 | "blocker-cleared"
 | "registry-rebuilt"
 | "downgraded"
 | "restored";

export type DependencyRelation = "depends-on" | "governs" | "blocks" | "supports";

export type ControlActionType =
 | "health-check"
 | "coverage-audit"
 | "drift-detection"
 | "rebuild-registries"
 | "review-authority-gaps"
 | "dependency-refresh"
 | "open-unsafe"
 | "resolve-blockers";

export type EstateHeaderStats = {
 controlScore: number;
 governedLibraries: number;
 activeRisks: number;
 blockedDependencies: number;
};

export type EstateStatusMetrics = {
 totalLibraries: number;
 governed: number;
 reviewRequired: number;
 stale: number;
 unsafe: number;
 blocked: number;
 missingAuthority: number;
 driftAlerts: number;
};
```

```
export type ControlAction = {
 id: string;
 label: string;
 icon: string;
 actionType: ControlActionType;
 enabled: boolean;
};

export type LibraryHealthBand = {
 id: HealthBandId;
 label: string;
 count: number;
 libraryIds: string[];
 recommendedAction: string;
};

export type LibraryControlRecord = {
 id: string;
 name: string;
 shortName: string;
 route?: string;
 owner?: string;

 authorityStatus: AuthorityStatus;
 coveragePercent: number;
 freshnessPercent: number;
 integrityPercent: number;
 dependencyPressure: number;

 lastAuditAt?: string;
 risk: RiskLevel;
 controlState: ControlState;
 trustDecision: TrustDecision;

 recommendedAction: string;
 dependencyBlockers: string[];

 driftAlert: boolean;
 stale: boolean;
 missingAuthority: boolean;
 blocked: boolean;
};

export type LibraryDetail = {
```

```
  id: string;
  name: string;
  shortName: string;
  route?: string;
  owner?: string;

  purpose: string;
  estateRole?: string;

  authoritySource?: string;
  authorityStatus: AuthorityStatus;
  trustDecision: TrustDecision;

  coveragePercent: number;
  freshnessPercent: number;
  integrityPercent: number;
  dependencyFitnessPercent: number;

  missingArtifacts: string[];
  staleArtifacts: string[];
  brokenReferences: string[];
  unresolvedConflicts: string[];
  unsupportedDependents: string[];

  upstreamDependencies: string[];
  downstreamDependents: string[];
  blockerImpact: string[];

  recommendedAction: string;
  urgency: RiskLevel;
  suggestedMaintenanceMode?: string;
};

export type LeverageFix = {
  id: string;
  libraryId: string;
  title: string;
  summary: string;
  impactRadius: "local" | "cross-library" | "estate-wide";
  effort: "small" | "medium" | "large";
  payoffScore: number;
  recommendedAction: string;
};
```

```
export type ActiveBlocker = {
 id: string;
 title: string;
 severity: RiskLevel;
 items: string[];
 affectedLibraries: string[];
};

export type SystemRisk = {
 id: string;
 title: string;
 severity: RiskLevel;
 summary: string;
 affectedLibraries: string[];
};

export type GovernanceRule = {
 id: string;
 title: string;
 description: string;
 severity: "info" | "warning" | "strict";
};

export type MaintenanceModeCard = {
 id: string;
 name: string;
 purpose: string;
 triggerCondition: string;
 suggestedForStates: ControlState[];
 launchable: boolean;
};

export type DependencyLink = {
 fromLibraryId: string;
 toLibraryId: string;
 relation: DependencyRelation;
 severity?: RiskLevel;
};

export type LibraryActivityEvent = {
 id: string;
 timestamp: string;
 type: ActivityType;
 libraryId?: string;
```

```
 title: string;
 summary: string;
};

export type LibraryControlFilters = {
 query: string;
 authorityStatus?: AuthorityStatus[];
 controlState?: ControlState[];
 risk?: RiskLevel[];
 staleOnly?: boolean;
 blockedOnly?: boolean;
 missingAuthorityOnly?: boolean;
 driftOnly?: boolean;
 sortBy?:
   | "risk"
   | "dependency-pressure"
   | "freshness"
   | "coverage"
   | "last-audit"
   | "control-score";
};

export type LibraryControlCenterState = {
 header: EstateHeaderStats;
 metrics: EstateStatusMetrics;
 actions: ControlAction[];
 healthBands: LibraryHealthBand[];
 libraries: LibraryControlRecord[];
 libraryDetails: Record<string, LibraryDetail>;
 leverageFixes: LeverageFix[];
 blockers: ActiveBlocker[];
 risks: SystemRisk[];
 governanceRules: GovernanceRule[];
 maintenanceModes: MaintenanceModeCard[];
 dependencyLinks: DependencyLink[];
 recentActivity: LibraryActivityEvent[];
 defaultFilters: LibraryControlFilters;
};
```

Helper utilities

```
const AUTHORITY_SCORE: Record<AuthorityStatus, number> = {
 authoritative: 100,
```

```
 partial: 60,
 disputed: 30,
 missing: 0,
};

export function computeLibraryControlScore(input: {
 authorityStatus: AuthorityStatus;
 coveragePercent: number;
 freshnessPercent: number;
 integrityPercent: number;
 dependencyPressure: number;
}) {
 const authorityScore = AUTHORITY_SCORE[input.authorityStatus];
 return Math.round(
   input.coveragePercent * 0.3 +
     input.freshnessPercent * 0.2 +
     input.integrityPercent * 0.25 +
     (100 - input.dependencyPressure) * 0.1 +
     authorityScore * 0.15
 );
}

export function inferControlState(input: {
 authorityStatus: AuthorityStatus;
 risk: RiskLevel;
 blocked: boolean;
 coveragePercent: number;
 freshnessPercent: number;
 integrityPercent: number;
}) : ControlState {
 if (input.blocked) return "blocked";

 if (
   (input.authorityStatus === "missing" || input.authorityStatus === "disputed") &&
   (input.risk === "high" || input.risk === "critical")
 ) {
   return "unsafe";
 }

 const avg = Math.round(
   (input.coveragePercent + input.freshnessPercent + input.integrityPercent) / 3
 );

 if (avg >= 85 && input.authorityStatus === "authoritative") return "governed";
```

```
 if (avg >= 70) return "review-required";
 if (avg >= 50) return "degraded";
 return "recovery";
}

export function isStale(freshnessPercent: number) {
 return freshnessPercent < 60;
}
```

Master library registry

Because the exact library names are not present in the uploaded `upgrade-matrix` source, this starter pack uses stable placeholders `LIB-01` through `LIB-16`. The page itself confirms there are 16 libraries, so this is safe as a structural starting point.

knowledge-dashboard

```
export const LIBRARY_REGISTRY: Array<{
 id: string;
 name: string;
 shortName: string;
 route: string;
}> = [
 { id: "LIB-01", name: "Library 01", shortName: "L01", route: "/library-01" },
 { id: "LIB-02", name: "Library 02", shortName: "L02", route: "/library-02" },
 { id: "LIB-03", name: "Library 03", shortName: "L03", route: "/library-03" },
 { id: "LIB-04", name: "Library 04", shortName: "L04", route: "/library-04" },
 { id: "LIB-05", name: "Library 05", shortName: "L05", route: "/library-05" },
 { id: "LIB-06", name: "Library 06", shortName: "L06", route: "/library-06" },
 { id: "LIB-07", name: "Library 07", shortName: "L07", route: "/library-07" },
 { id: "LIB-08", name: "Library 08", shortName: "L08", route: "/library-08" },
 { id: "LIB-09", name: "Library 09", shortName: "L09", route: "/library-09" },
 { id: "LIB-10", name: "Library 10", shortName: "L10", route: "/library-10" },
 { id: "LIB-11", name: "Library 11", shortName: "L11", route: "/library-11" },
 { id: "LIB-12", name: "Library 12", shortName: "L12", route: "/library-12" },
 { id: "LIB-13", name: "Library 13", shortName: "L13", route: "/library-13" },
 { id: "LIB-14", name: "Library 14", shortName: "L14", route: "/library-14" },
 { id: "LIB-15", name: "Library 15", shortName: "L15", route: "/library-15" },
 { id: "LIB-16", name: "Library 16", shortName: "L16", route: "/library-16" },
];
```

Starter library records

This gives you balanced initial state across all 16 so the UI has meaningful variety on day one.

```
export const LIBRARIES: LibraryControlRecord[] = [
 {
   id: "LIB-01",
   name: "Library 01",
   shortName: "L01",
   route: "/library-01",
   owner: "Axion Core",
   authorityStatus: "authoritative",
   coveragePercent: 92,
   freshnessPercent: 88,
   integrityPercent: 94,
   dependencyPressure: 78,
   lastAuditAt: "2026-03-09T10:00:00Z",
   risk: "moderate",
   controlState: "governed",
   trustDecision: "trusted",
   recommendedAction: "Monitor dependency pressure and keep audit cadence.",
   dependencyBlockers: [],
   driftAlert: false,
   stale: false,
   missingAuthority: false,
   blocked: false,
 },
 {
   id: "LIB-02",
   name: "Library 02",
   shortName: "L02",
   route: "/library-02",
   owner: "Axion Core",
   authorityStatus: "authoritative",
   coveragePercent: 86,
   freshnessPercent: 80,
   integrityPercent: 83,
   dependencyPressure: 61,
   lastAuditAt: "2026-03-07T14:00:00Z",
   risk: "moderate",
   controlState: "governed",
   trustDecision: "trusted",
   recommendedAction: "Run health check before next dependent update.",
```

```
    dependencyBlockers: [],
    driftAlert: false,
    stale: false,
    missingAuthority: false,
    blocked: false,
  },
  {
    id: "LIB-03",
    name: "Library 03",
    shortName: "L03",
    route: "/library-03",
    owner: "Axion Core",
    authorityStatus: "partial",
    coveragePercent: 78,
    freshnessPercent: 73,
    integrityPercent: 76,
    dependencyPressure: 54,
    lastAuditAt: "2026-02-28T16:30:00Z",
    risk: "moderate",
    controlState: "review-required",
    trustDecision: "conditional",
    recommendedAction: "Close authority gaps and rerun coverage audit.",
    dependencyBlockers: [],
    driftAlert: false,
    stale: false,
    missingAuthority: false,
    blocked: false,
  },
  {
    id: "LIB-04",
    name: "Library 04",
    shortName: "L04",
    route: "/library-04",
    owner: "Axion Core",
    authorityStatus: "partial",
    coveragePercent: 72,
    freshnessPercent: 66,
    integrityPercent: 71,
    dependencyPressure: 67,
    lastAuditAt: "2026-02-24T12:00:00Z",
    risk: "high",
    controlState: "review-required",
    trustDecision: "conditional",
    recommendedAction: "Repair missing authority links and review downstream impact.",
```

```
    dependencyBlockers: [],
    driftAlert: true,
    stale: false,
    missingAuthority: false,
    blocked: false,
  },
  {
    id: "LIB-05",
    name: "Library 05",
    shortName: "L05",
    route: "/library-05",
    owner: "Axion Core",
    authorityStatus: "authoritative",
    coveragePercent: 90,
    freshnessPercent: 55,
    integrityPercent: 91,
    dependencyPressure: 49,
    lastAuditAt: "2026-01-30T08:00:00Z",
    risk: "high",
    controlState: "review-required",
    trustDecision: "conditional",
    recommendedAction: "Run drift detection and refresh stale source material.",
    dependencyBlockers: [],
    driftAlert: true,
    stale: true,
    missingAuthority: false,
    blocked: false,
  },
  {
    id: "LIB-06",
    name: "Library 06",
    shortName: "L06",
    route: "/library-06",
    owner: "Axion Core",
    authorityStatus: "partial",
    coveragePercent: 64,
    freshnessPercent: 58,
    integrityPercent: 69,
    dependencyPressure: 72,
    lastAuditAt: "2026-01-18T11:15:00Z",
    risk: "high",
    controlState: "degraded",
    trustDecision: "conditional",
    recommendedAction: "Run coverage audit and reduce dependency fragility.",
```

```
    dependencyBlockers: ["Registry references incomplete"],
    driftAlert: true,
    stale: true,
    missingAuthority: false,
    blocked: false,
  },
  {
    id: "LIB-07",
    name: "Library 07",
    shortName: "L07",
    route: "/library-07",
    owner: "Axion Core",
    authorityStatus: "missing",
    coveragePercent: 51,
    freshnessPercent: 62,
    integrityPercent: 57,
    dependencyPressure: 43,
    lastAuditAt: "2026-02-10T09:45:00Z",
    risk: "high",
    controlState: "unsafe",
    trustDecision: "untrusted",
    recommendedAction: "Assign authoritative source before further governance use.",
    dependencyBlockers: ["No authoritative governing source"],
    driftAlert: false,
    stale: false,
    missingAuthority: true,
    blocked: false,
  },
  {
    id: "LIB-08",
    name: "Library 08",
    shortName: "L08",
    route: "/library-08",
    owner: "Axion Core",
    authorityStatus: "disputed",
    coveragePercent: 68,
    freshnessPercent: 61,
    integrityPercent: 52,
    dependencyPressure: 64,
    lastAuditAt: "2026-02-06T17:20:00Z",
    risk: "critical",
    controlState: "unsafe",
    trustDecision: "untrusted",
    recommendedAction: "Resolve source conflict and freeze downstream governance.",
```

```
    dependencyBlockers: ["Conflicting authority sources"],
    driftAlert: true,
    stale: false,
    missingAuthority: false,
    blocked: false,
  },
  {
    id: "LIB-09",
    name: "Library 09",
    shortName: "L09",
    route: "/library-09",
    owner: "Axion Core",
    authorityStatus: "authoritative",
    coveragePercent: 87,
    freshnessPercent: 84,
    integrityPercent: 85,
    dependencyPressure: 32,
    lastAuditAt: "2026-03-08T13:00:00Z",
    risk: "low",
    controlState: "governed",
    trustDecision: "trusted",
    recommendedAction: "Maintain current audit cadence.",
    dependencyBlockers: [],
    driftAlert: false,
    stale: false,
    missingAuthority: false,
    blocked: false,
  },
  {
    id: "LIB-10",
    name: "Library 10",
    shortName: "L10",
    route: "/library-10",
    owner: "Axion Core",
    authorityStatus: "partial",
    coveragePercent: 74,
    freshnessPercent: 57,
    integrityPercent: 74,
    dependencyPressure: 58,
    lastAuditAt: "2026-01-26T07:40:00Z",
    risk: "moderate",
    controlState: "review-required",
    trustDecision: "conditional",
    recommendedAction: "Refresh stale sections and rerun integrity cleanup.",
```

```
    dependencyBlockers: [],
    driftAlert: true,
    stale: true,
    missingAuthority: false,
    blocked: false,
  },
  {
    id: "LIB-11",
    name: "Library 11",
    shortName: "L11",
    route: "/library-11",
    owner: "Axion Core",
    authorityStatus: "missing",
    coveragePercent: 46,
    freshnessPercent: 44,
    integrityPercent: 49,
    dependencyPressure: 57,
    lastAuditAt: "2026-01-12T19:10:00Z",
    risk: "critical",
    controlState: "unsafe",
    trustDecision: "untrusted",
    recommendedAction: "Restore authority and rebuild registry entries immediately.",
    dependencyBlockers: ["Missing authority", "Registry entry incomplete"],
    driftAlert: true,
    stale: true,
    missingAuthority: true,
    blocked: false,
  },
  {
    id: "LIB-12",
    name: "Library 12",
    shortName: "L12",
    route: "/library-12",
    owner: "Axion Core",
    authorityStatus: "authoritative",
    coveragePercent: 82,
    freshnessPercent: 79,
    integrityPercent: 88,
    dependencyPressure: 45,
    lastAuditAt: "2026-03-05T15:00:00Z",
    risk: "low",
    controlState: "governed",
    trustDecision: "trusted",
    recommendedAction: "No immediate action required.",
```

```
    dependencyBlockers: [],
    driftAlert: false,
    stale: false,
    missingAuthority: false,
    blocked: false,
  },
  {
    id: "LIB-13",
    name: "Library 13",
    shortName: "L13",
    route: "/library-13",
    owner: "Axion Core",
    authorityStatus: "partial",
    coveragePercent: 59,
    freshnessPercent: 52,
    integrityPercent: 63,
    dependencyPressure: 81,
    lastAuditAt: "2026-01-29T10:20:00Z",
    risk: "high",
    controlState: "degraded",
    trustDecision: "conditional",
    recommendedAction: "Reduce dependency bottleneck and run failure triage.",
    dependencyBlockers: ["High downstream dependency pressure"],
    driftAlert: true,
    stale: true,
    missingAuthority: false,
    blocked: true,
  },
  {
    id: "LIB-14",
    name: "Library 14",
    shortName: "L14",
    route: "/library-14",
    owner: "Axion Core",
    authorityStatus: "authoritative",
    coveragePercent: 91,
    freshnessPercent: 90,
    integrityPercent: 93,
    dependencyPressure: 27,
    lastAuditAt: "2026-03-10T09:30:00Z",
    risk: "low",
    controlState: "governed",
    trustDecision: "trusted",
    recommendedAction: "Use as benchmark library for restoration patterns.",
```

```
    dependencyBlockers: [],
    driftAlert: false,
    stale: false,
    missingAuthority: false,
    blocked: false,
  },
  {
    id: "LIB-15",
    name: "Library 15",
    shortName: "L15",
    route: "/library-15",
    owner: "Axion Core",
    authorityStatus: "disputed",
    coveragePercent: 63,
    freshnessPercent: 67,
    integrityPercent: 59,
    dependencyPressure: 69,
    lastAuditAt: "2026-02-03T06:50:00Z",
    risk: "high",
    controlState: "unsafe",
    trustDecision: "untrusted",
    recommendedAction: "Resolve authority conflict and audit dependent surfaces.",
    dependencyBlockers: ["Authority dispute affects dependent libraries"],
    driftAlert: false,
    stale: false,
    missingAuthority: false,
    blocked: false,
  },
  {
    id: "LIB-16",
    name: "Library 16",
    shortName: "L16",
    route: "/library-16",
    owner: "Axion Core",
    authorityStatus: "partial",
    coveragePercent: 69,
    freshnessPercent: 64,
    integrityPercent: 72,
    dependencyPressure: 75,
    lastAuditAt: "2026-02-18T18:00:00Z",
    risk: "high",
    controlState: "blocked",
    trustDecision: "conditional",
    recommendedAction: "Clear dependency blockers and rerun health check.",
```

```
  dependencyBlockers: ["Upstream registry mismatch", "Dependent library unresolved"],
  driftAlert: true,
  stale: false,
  missingAuthority: false,
  blocked: true,
 },
];
```

Library details map

To keep this readable, use a generator plus targeted overrides.

```
const DEFAULT_MISSING_ARTIFACTS = [
 "Authority source mapping",
 "Registry link verification",
 "Coverage evidence",
];

const DEFAULT_STALE_ARTIFACTS = [
 "Audit snapshot",
 "Dependency review notes",
];

function makeDetail(record: LibraryControlRecord): LibraryDetail {
 return {
  id: record.id,
  name: record.name,
  shortName: record.shortName,
  route: record.route,
  owner: record.owner,
  purpose: `${record.name} governs a defined segment of the Axion library estate.`,
  estateRole: "Library governance unit",

  authoritySource:
    record.authorityStatus === "missing"
      ? undefined
      : `${record.name} canonical source`,
  authorityStatus: record.authorityStatus,
  trustDecision: record.trustDecision,

  coveragePercent: record.coveragePercent,
  freshnessPercent: record.freshnessPercent,
  integrityPercent: record.integrityPercent,
```

```
  dependencyFitnessPercent: Math.max(0, 100 - record.dependencyPressure),

  missingArtifacts:
    record.missingAuthority || record.coveragePercent < 70
      ? DEFAULT_MISSING_ARTIFACTS
      : ["None critical"],
  staleArtifacts:
    record.stale ? DEFAULT_STALE_ARTIFACTS : ["None significant"],
  brokenReferences:
    record.blocked ? ["Registry pointer mismatch"] : [],
  unresolvedConflicts:
    record.authorityStatus === "disputed" ? ["Authority conflict unresolved"] : [],
  unsupportedDependents:
    record.risk === "critical" ? ["Downstream trust should be suspended"] : [],

  upstreamDependencies: [],
  downstreamDependents: [],
  blockerImpact: record.dependencyBlockers,

  recommendedAction: record.recommendedAction,
  urgency: record.risk,
  suggestedMaintenanceMode:
    record.missingAuthority
      ? "MM-13"
      : record.driftAlert
      ? "MM-04"
      : record.blocked
      ? "MM-20"
      : "MM-01",
 };
}

export const LIBRARY_DETAILS: Record<string, LibraryDetail> = Object.fromEntries(
 LIBRARIES.map((lib) => [lib.id, makeDetail(lib)])
);
```

Then add targeted overrides where you want more realism:

```
LIBRARY_DETAILS["LIB-08"] = {
 ...LIBRARY_DETAILS["LIB-08"],
 purpose: "Controls a disputed governance area with conflicting source authority.",
 upstreamDependencies: ["LIB-03", "LIB-05"],
 downstreamDependents: ["LIB-13", "LIB-16"],
 unresolvedConflicts: [
   "Authority source A conflicts with source B",
```

```
  "Dependent trust chain unresolved",
 ],
 blockerImpact: [
  "Dependent libraries should not inherit governance from this library",
 ],
 suggestedMaintenanceMode: "MM-20",
};

LIBRARY_DETAILS["LIB-11"] = {
 ...LIBRARY_DETAILS["LIB-11"],
 purpose: "Represents an authority-critical library missing canonical governance.",
 missingArtifacts: [
  "Canonical authority source",
  "Resolved authority snapshot",
  "Registry record",
  "Audit evidence pack",
 ],
 staleArtifacts: ["Backcompat notes", "Coverage audit output"],
 brokenReferences: ["Registry entry missing", "Broken authority pointer"],
 suggestedMaintenanceMode: "MM-21",
};
```

Control actions

```
export const CONTROL_ACTIONS: ControlAction[] = [
 { id: "act-health", label: "Run Health Check", icon: "HeartPulse", actionType: "health-check",
enabled: true },
 { id: "act-coverage", label: "Run Coverage Audit", icon: "ScanSearch", actionType:
"coverage-audit", enabled: true },
 { id: "act-drift", label: "Run Drift Detection", icon: "GitCompareArrows", actionType:
"drift-detection", enabled: true },
 { id: "act-registry", label: "Rebuild Registries", icon: "DatabaseZap", actionType:
"rebuild-registries", enabled: true },
 { id: "act-authority", label: "Review Authority Gaps", icon: "ShieldAlert", actionType:
"review-authority-gaps", enabled: true },
 { id: "act-refresh", label: "Refresh Dependencies", icon: "RefreshCcw", actionType:
"dependency-refresh", enabled: true },
 { id: "act-unsafe", label: "Open Unsafe Libraries", icon: "TriangleAlert", actionType:
"open-unsafe", enabled: true },
 { id: "act-blockers", label: "Resolve Blockers", icon: "Hammer", actionType: "resolve-blockers",
enabled: true },
];
```

Health bands

```
export const HEALTH_BANDS: LibraryHealthBand[] = [
 {
   id: "authoritative",
   label: "Authoritative",
   count: 4,
   libraryIds: ["LIB-01", "LIB-09", "LIB-12", "LIB-14"],
   recommendedAction: "Preserve audit freshness and use as benchmark libraries.",
 },
 {
   id: "stable",
   label: "Stable",
   count: 2,
   libraryIds: ["LIB-02", "LIB-03"],
   recommendedAction: "Close minor gaps before dependency growth increases risk.",
 },
 {
   id: "incomplete",
   label: "Incomplete",
   count: 3,
   libraryIds: ["LIB-04", "LIB-06", "LIB-10"],
   recommendedAction: "Run coverage and integrity checks to restore confidence.",
 },
 {
   id: "stale",
   label: "Stale",
   count: 3,
   libraryIds: ["LIB-05", "LIB-10", "LIB-13"],
   recommendedAction: "Refresh source material and rerun drift detection.",
 },
 {
   id: "fragmented",
   label: "Fragmented",
   count: 2,
   libraryIds: ["LIB-13", "LIB-16"],
   recommendedAction: "Reduce dependency bottlenecks and rebuild broken links.",
 },
 {
   id: "untrusted",
   label: "Untrusted",
   count: 4,
   libraryIds: ["LIB-07", "LIB-08", "LIB-11", "LIB-15"],
   recommendedAction: "Resolve authority issues before governance use continues.",
```

```
 },
];
```

Leverage fixes

```
export const LEVERAGE_FIXES: LeverageFix[] = [
 {
   id: "fix-01",
   libraryId: "LIB-11",
   title: "Restore authority and registry record",
   summary: "One repair removes a critical trust failure and re-enables governance use.",
   impactRadius: "cross-library",
   effort: "medium",
   payoffScore: 96,
   recommendedAction: "Run MM-13 then MM-21",
 },
 {
   id: "fix-02",
   libraryId: "LIB-08",
   title: "Resolve disputed authority chain",
   summary: "Stops unsafe downstream inheritance across multiple dependent libraries.",
   impactRadius: "estate-wide",
   effort: "medium",
   payoffScore: 92,
   recommendedAction: "Run MM-20 and freeze dependent trust",
 },
 {
   id: "fix-03",
   libraryId: "LIB-13",
   title: "Reduce dependency bottleneck",
   summary: "Clears a blocked path affecting multiple downstream libraries.",
   impactRadius: "cross-library",
   effort: "small",
   payoffScore: 88,
   recommendedAction: "Run MM-17 then rerun MM-01",
 },
 {
   id: "fix-04",
   libraryId: "LIB-05",
   title: "Refresh stale authority surfaces",
   summary: "A freshness repair returns a strong library to reliable governed state.",
   impactRadius: "local",
   effort: "small",
```

```
    payoffScore: 79,
    recommendedAction: "Run MM-04",
  },
  {
    id: "fix-05",
    libraryId: "LIB-07",
    title: "Assign canonical authority source",
    summary: "Converts an unsafe library into a reviewable one.",
    impactRadius: "cross-library",
    effort: "small",
    payoffScore: 85,
    recommendedAction: "Run MM-13",
  },
];
```

Blockers and risks

```
export const ACTIVE_BLOCKERS: ActiveBlocker[] = [
  {
    id: "blk-01",
    title: "Missing authoritative source assignments",
    severity: "critical",
    items: [
      "Governance cannot be trusted where authority is missing",
      "Dependent libraries should not inherit unresolved authority",
    ],
    affectedLibraries: ["LIB-07", "LIB-11"],
  },
  {
    id: "blk-02",
    title: "Authority disputes in active control paths",
    severity: "critical",
    items: [
      "Conflicting sources are present",
      "Trust decisions cannot safely propagate",
    ],
    affectedLibraries: ["LIB-08", "LIB-15"],
  },
  {
    id: "blk-03",
    title: "Dependency bottleneck blocking downstream confidence",
    severity: "high",
    items: [
```

```
    "High dependency pressure is concentrating estate risk",
    "Blocked libraries require triage before expansion",
  ],
  affectedLibraries: ["LIB-13", "LIB-16"],
},
];

export const SYSTEM_RISKS: SystemRisk[] = [
{
  id: "risk-01",
  title: "Stale governing surfaces",
  severity: "high",
  summary: "Several libraries retain strong structure but aging source freshness reduces trust.",
  affectedLibraries: ["LIB-05", "LIB-06", "LIB-10", "LIB-13"],
},
{
  id: "risk-02",
  title: "Registry integrity drift",
  severity: "high",
  summary: "Broken or incomplete registry references threaten cross-library governance
confidence.",
  affectedLibraries: ["LIB-06", "LIB-11", "LIB-16"],
},
{
  id: "risk-03",
  title: "Unsafe downstream reliance",
  severity: "critical",
  summary: "Dependent libraries may be inheriting governance from disputed or missing
authority sources.",
  affectedLibraries: ["LIB-08", "LIB-11", "LIB-13", "LIB-16"],
},
];
```

Governance rules

```
export const GOVERNANCE_RULES: GovernanceRule[] = [
{
  id: "rule-01",
  title: "One authoritative source per governed subject",
  description: "Every governed library area must resolve to one canonical authority source.",
  severity: "strict",
},
{
```

```
  id: "rule-02",
  title: "Stale authority cannot silently govern runtime",
  description: "Freshness decay forces review or remediation before trust continues.",
  severity: "strict",
 },
 {
  id: "rule-03",
  title: "No template or derivative control without source backing",
  description: "Derived governance artifacts require valid authority support.",
  severity: "strict",
 },
 {
  id: "rule-04",
  title: "Registry integrity required before trust restoration",
  description: "Libraries cannot return to governed state with broken registry links.",
  severity: "strict",
 },
 {
  id: "rule-05",
  title: "Unresolved conflicts force review-required or unsafe state",
  description: "Authority disputes must never remain invisible to operators.",
  severity: "warning",
 },
 {
  id: "rule-06",
  title: "Dependency-critical libraries require fresh audit evidence",
  description: "High-pressure libraries need tighter audit cadence than low-pressure libraries.",
  severity: "warning",
 },
];
```

Maintenance modes

These are aligned to your locked maintenance registry.

Response: Axion Updates Planning

```
export const MAINTENANCE_MODES: MaintenanceModeCard[] = [
 {
  id: "MM-01",
  name: "Health Check",
  purpose: "Run a broad estate health pass across libraries.",
  triggerCondition: "Use when you need an overall health baseline.",
```

```
  suggestedForStates: ["governed", "review-required", "degraded"],
  launchable: true,
},
{
  id: "MM-03",
  name: "Coverage Audit",
  purpose: "Validate completeness and identify coverage gaps.",
  triggerCondition: "Use when coverage percent is low or incomplete.",
  suggestedForStates: ["review-required", "degraded", "recovery"],
  launchable: true,
},
{
  id: "MM-04",
  name: "Drift Detection",
  purpose: "Detect divergence between intended governance and current library state.",
  triggerCondition: "Use when freshness declines or drift alerts appear.",
  suggestedForStates: ["review-required", "degraded"],
  launchable: true,
},
{
  id: "MM-12",
  name: "Rebuild Indexes/Registries",
  purpose: "Regenerate indexes and registries supporting control surfaces.",
  triggerCondition: "Use when registry pathing or reference integrity is weak.",
  suggestedForStates: ["degraded", "blocked", "recovery"],
  launchable: true,
},
{
  id: "MM-13",
  name: "Registry Integrity Cleanup",
  purpose: "Repair registry-level integrity issues and broken authority linkages.",
  triggerCondition: "Use when authority is missing or registry references are broken.",
  suggestedForStates: ["review-required", "unsafe", "recovery"],
  launchable: true,
},
{
  id: "MM-17",
  name: "Dependency Refresh",
  purpose: "Refresh dependency assumptions and supporting references.",
  triggerCondition: "Use when dependency pressure or cross-library fragility is high.",
  suggestedForStates: ["degraded", "blocked"],
  launchable: true,
},
{
```

```
  id: "MM-18",
  name: "Workspace Compatibility Check",
  purpose: "Validate that library state remains compatible across connected workspaces.",
  triggerCondition: "Use after major updates or suspected compatibility drift.",
  suggestedForStates: ["review-required", "degraded"],
  launchable: true,
 },
 {
  id: "MM-20",
  name: "Failure Triage Mode",
  purpose: "Contain, inspect, and prioritize active failures.",
  triggerCondition: "Use when blockers or authority disputes are actively affecting governance.",
  suggestedForStates: ["blocked", "unsafe"],
  launchable: true,
 },
 {
  id: "MM-21",
  name: "Emergency Recovery Mode",
  purpose: "Recover control authority after severe integrity or trust failure.",
  triggerCondition: "Use for critical authority or registry failures.",
  suggestedForStates: ["unsafe", "recovery"],
  launchable: true,
 },
];
```

Dependency links

```
export const DEPENDENCY_LINKS: DependencyLink[] = [
 { fromLibraryId: "LIB-03", toLibraryId: "LIB-08", relation: "supports", severity: "moderate" },
 { fromLibraryId: "LIB-05", toLibraryId: "LIB-08", relation: "supports", severity: "moderate" },
 { fromLibraryId: "LIB-08", toLibraryId: "LIB-13", relation: "governs", severity: "critical" },
 { fromLibraryId: "LIB-08", toLibraryId: "LIB-16", relation: "governs", severity: "critical" },
 { fromLibraryId: "LIB-11", toLibraryId: "LIB-13", relation: "blocks", severity: "critical" },
 { fromLibraryId: "LIB-13", toLibraryId: "LIB-16", relation: "blocks", severity: "high" },
 { fromLibraryId: "LIB-01", toLibraryId: "LIB-04", relation: "supports", severity: "low" },
 { fromLibraryId: "LIB-14", toLibraryId: "LIB-02", relation: "supports", severity: "low" },
];
```

Recent activity

```
export const RECENT_ACTIVITY: LibraryActivityEvent[] = [
 {
```

```
  id: "evt-01",
  timestamp: "2026-03-10T09:45:00Z",
  type: "audit-run",
  libraryId: "LIB-14",
  title: "Audit completed for Library 14",
  summary: "Library retained governed state with strong integrity and freshness.",
},
{
  id: "evt-02",
  timestamp: "2026-03-09T12:15:00Z",
  type: "drift-detected",
  libraryId: "LIB-05",
  title: "Drift alert raised on Library 05",
  summary: "Freshness fell below preferred threshold for a governing source surface.",
},
{
  id: "evt-03",
  timestamp: "2026-03-08T17:10:00Z",
  type: "downgraded",
  libraryId: "LIB-13",
  title: "Library 13 downgraded to blocked",
  summary: "Dependency bottleneck exceeded safe threshold and blocked downstream trust.",
},
{
  id: "evt-04",
  timestamp: "2026-03-07T14:20:00Z",
  type: "registry-rebuilt",
  libraryId: "LIB-06",
  title: "Registry rebuild executed for Library 06",
  summary: "Registry structure refreshed; follow-up integrity review still required.",
},
{
  id: "evt-05",
  timestamp: "2026-03-06T11:00:00Z",
  type: "blocker-cleared",
  libraryId: "LIB-02",
  title: "Blocking reference cleared for Library 02",
  summary: "Dependent trust path restored after reference cleanup.",
},
];
```


Default filters

```
export const DEFAULT_FILTERS: LibraryControlFilters = {
 query: "",
 authorityStatus: undefined,
 controlState: undefined,
 risk: undefined,
 staleOnly: false,
 blockedOnly: false,
 missingAuthorityOnly: false,
 driftOnly: false,
 sortBy: "risk",
};
```

Derived estate stats

```
export function getEstateHeaderStats(libraries: LibraryControlRecord[]): EstateHeaderStats {
 const governedLibraries = libraries.filter((l) => l.controlState === "governed").length;
 const activeRisks = libraries.filter((l) => l.risk === "high" || l.risk === "critical").length;
 const blockedDependencies = libraries.filter((l) => l.blocked).length;

 const controlScore = Math.round(
   libraries.reduce((sum, lib) => {
     return sum + computeLibraryControlScore({
       authorityStatus: lib.authorityStatus,
       coveragePercent: lib.coveragePercent,
       freshnessPercent: lib.freshnessPercent,
       integrityPercent: lib.integrityPercent,
       dependencyPressure: lib.dependencyPressure,
     });
   }, 0) / libraries.length
 );

 return {
   controlScore,
   governedLibraries,
   activeRisks,
   blockedDependencies,
 };
}

export function getEstateStatusMetrics(libraries: LibraryControlRecord[]): EstateStatusMetrics {
 return {
   totalLibraries: libraries.length,
   governed: libraries.filter((l) => l.controlState === "governed").length,
```

```
  reviewRequired: libraries.filter((l) => l.controlState === "review-required").length,
  stale: libraries.filter((l) => l.stale).length,
  unsafe: libraries.filter((l) => l.controlState === "unsafe").length,
  blocked: libraries.filter((l) => l.blocked).length,
  missingAuthority: libraries.filter((l) => l.missingAuthority).length,
  driftAlerts: libraries.filter((l) => l.driftAlert).length,
 };
}
```

Export the full page state

```
export const libraryControlCenterState: LibraryControlCenterState = {
 header: getEstateHeaderStats(LIBRARIES),
 metrics: getEstateStatusMetrics(LIBRARIES),
 actions: CONTROL_ACTIONS,
 healthBands: HEALTH_BANDS,
 libraries: LIBRARIES,
 libraryDetails: LIBRARY_DETAILS,
 leverageFixes: LEVERAGE_FIXES,
 blockers: ACTIVE_BLOCKERS,
 risks: SYSTEM_RISKS,
 governanceRules: GOVERNANCE_RULES,
 maintenanceModes: MAINTENANCE_MODES,
 dependencyLinks: DEPENDENCY_LINKS,
 recentActivity: RECENT_ACTIVITY,
 defaultFilters: DEFAULT_FILTERS,
};
```

What to replace first

The first thing I would swap once you have the real source file is:

- `Library 01` → real library name
- `/library-01` → real route
- purpose text
- upstream/downstream dependency links

Everything else can stay structurally intact.

Cleanest migration path

Since the current page is still wired to `../data/upgrade-matrix` and `useUpgradeProgress`, the clean move is:

- create `library-control.ts`
- create `use-library-control.ts`
- port the page to the new schema
- then swap placeholder library records with your actual 16 library records from the old upgrade dataset
  knowledge-dashboard

hook spec

Here's the full **hook spec** for `use-library-control.ts`.

This is replacing the current page behavior layer that is still built around `useUpgradeProgress`, phase toggling, artifact completion toggles, and phase progress helpers. The uploaded page shows the current page depends on `useUpgradeProgress()` and phase-specific UI state like `openPhases`, `togglePhase`, `getOverallProgress`, and `getPhaseProgress`, which is exactly what should be swapped for a control-centered behavior layer.

knowledge-dashboard

knowledge-dashboard

Goal of the hook

`useLibraryControl` should be the **single behavior layer** for the new page.

It should manage:

- current filters
- current sort
- selected library
- filtered/sorted library list
- summary counts
- active quick filters
- derived health views
- action handlers
- detail-panel selection
- lightweight control actions

It should **not** manage upgrade-phase behavior, artifact checkbox toggles, or phase expansion state.

File target

App/src/hooks/use-library-control.ts

Hook contract

```
import { useMemo, useState, useCallback } from "react";
import {
 libraryControlCenterState,
 type AuthorityStatus,
 type ControlState,
 type RiskLevel,
 type LibraryControlFilters,
 type LibraryControlRecord,
 type LibraryDetail,
 type ControlAction,
 type LibraryHealthBand,
 type EstateHeaderStats,
 type EstateStatusMetrics,
 type LeverageFix,
 type ActiveBlocker,
 type SystemRisk,
 type GovernanceRule,
 type MaintenanceModeCard,
 type DependencyLink,
 type LibraryActivityEvent,
 computeLibraryControlScore,
} from "../data/library-control";
```

Return shape

```
export type UseLibraryControlReturn = {
 // raw state
 filters: LibraryControlFilters;
 selectedLibraryId: string | null;

 // derived records
 libraries: LibraryControlRecord[];
 filteredLibraries: LibraryControlRecord[];
 selectedLibrary: LibraryDetail | null;

 // top-page derived state
 header: EstateHeaderStats;
 metrics: EstateStatusMetrics;
 healthBands: LibraryHealthBand[];
```

```
// supporting sections
leverageFixes: LeverageFix[];
blockers: ActiveBlocker[];
risks: SystemRisk[];
governanceRules: GovernanceRule[];
maintenanceModes: MaintenanceModeCard[];
dependencyLinks: DependencyLink[];
recentActivity: LibraryActivityEvent[];
actions: ControlAction[];

// helper views
unsafeLibraries: LibraryControlRecord[];
blockedLibraries: LibraryControlRecord[];
staleLibraries: LibraryControlRecord[];
missingAuthorityLibraries: LibraryControlRecord[];
driftLibraries: LibraryControlRecord[];

// filtering actions
setQuery: (query: string) => void;
setAuthorityFilter: (values?: AuthorityStatus[]) => void;
setControlStateFilter: (values?: ControlState[]) => void;
setRiskFilter: (values?: RiskLevel[]) => void;
setSortBy: (sortBy?: LibraryControlFilters["sortBy"]) => void;

toggleStaleOnly: () => void;
toggleBlockedOnly: () => void;
toggleMissingAuthorityOnly: () => void;
toggleDriftOnly: () => void;

clearFilters: () => void;

// quick filter presets
applyCriticalOnlyPreset: () => void;
applyUnsafePreset: () => void;
applyBlockedPreset: () => void;
applyMissingAuthorityPreset: () => void;
applyDriftPreset: () => void;
applyGovernedPreset: () => void;

// selection
selectLibrary: (libraryId: string) => void;
clearSelectedLibrary: () => void;
selectFirstVisibleLibrary: () => void;
```

```
 // score / badge helpers
 getLibraryControlScore: (library: LibraryControlRecord) => number;
 getRiskWeight: (risk: RiskLevel) => number;

 // control action handlers
 runControlAction: (actionType: ControlAction["actionType"]) => void;
 assignMaintenanceMode: (libraryId: string, modeId: string) => void;

 // convenience predicates
 hasActiveFilters: boolean;
 visibleCount: number;
 totalCount: number;
};
```

State model inside the hook

Internal state

Use only two local state buckets:

```
const [filters, setFilters] = useState<LibraryControlFilters>(
 libraryControlCenterState.defaultFilters
);

const [selectedLibraryId, setSelectedLibraryId] = useState<string | null>(
 libraryControlCenterState.libraries[0]?.id ?? null
);
```

Why

The rest should be derived from the data pack with `useMemo`.

That keeps the behavior layer deterministic and avoids the kind of scattered page-local state the current implementation has with upgrade/phase toggling.

knowledge-dashboard

Core derived selectors

1. Raw libraries

```
const libraries = libraryControlCenterState.libraries;
```

2. Selected library

```
const selectedLibrary = useMemo(() => {
 if (!selectedLibraryId) return null;
 return libraryControlCenterState.libraryDetails[selectedLibraryId] ?? null;
}, [selectedLibraryId]);
```

3. Risk weight helper

```
const getRiskWeight = useCallback((risk: RiskLevel) => {
 switch (risk) {
   case "critical": return 4;
   case "high": return 3;
   case "moderate": return 2;
   case "low": return 1;
 }
}, []);
```

4. Control score helper

```
const getLibraryControlScore = useCallback((library: LibraryControlRecord) => {
 return computeLibraryControlScore({
   authorityStatus: library.authorityStatus,
   coveragePercent: library.coveragePercent,
   freshnessPercent: library.freshnessPercent,
   integrityPercent: library.integrityPercent,
   dependencyPressure: library.dependencyPressure,
 });
}, []);
```

Filtering pipeline

Apply filters in this order:

1. query
2. authority status
3. control state
4. risk

5. stale only
6. blocked only
7. missing authority only
8. drift only
9. sorting

That gives predictable narrowing.

Filter implementation

```
const filteredLibraries = useMemo(() => {
 let result = [...libraries];

 const q = filters.query.trim().toLowerCase();
 if (q) {
   result = result.filter((lib) => {
     return (
       lib.name.toLowerCase().includes(q) ||
       lib.shortName.toLowerCase().includes(q) ||
       lib.owner?.toLowerCase().includes(q) ||
       lib.recommendedAction.toLowerCase().includes(q)
     );
   });
 }

 if (filters.authorityStatus?.length) {
   result = result.filter((lib) =>
     filters.authorityStatus!.includes(lib.authorityStatus)
   );
 }

 if (filters.controlState?.length) {
   result = result.filter((lib) =>
     filters.controlState!.includes(lib.controlState)
   );
 }

 if (filters.risk?.length) {
   result = result.filter((lib) => filters.risk!.includes(lib.risk));
 }

 if (filters.staleOnly) {
   result = result.filter((lib) => lib.stale);
 }
```

```
  if (filters.blockedOnly) {
    result = result.filter((lib) => lib.blocked);
  }

  if (filters.missingAuthorityOnly) {
    result = result.filter((lib) => lib.missingAuthority);
  }

  if (filters.driftOnly) {
    result = result.filter((lib) => lib.driftAlert);
  }

  result.sort((a, b) => {
    switch (filters.sortBy) {
      case "risk":
        return getRiskWeight(b.risk) - getRiskWeight(a.risk);

      case "dependency-pressure":
        return b.dependencyPressure - a.dependencyPressure;

      case "freshness":
        return a.freshnessPercent - b.freshnessPercent;

      case "coverage":
        return a.coveragePercent - b.coveragePercent;

      case "last-audit": {
        const aTime = a.lastAuditAt ? new Date(a.lastAuditAt).getTime() : 0;
        const bTime = b.lastAuditAt ? new Date(b.lastAuditAt).getTime() : 0;
        return aTime - bTime;
      }

      case "control-score":
        return getLibraryControlScore(a) - getLibraryControlScore(b);

      default:
        return getRiskWeight(b.risk) - getRiskWeight(a.risk);
    }
  });

  return result;
}, [libraries, filters, getRiskWeight, getLibraryControlScore]);
```

Convenience subsets

These power chips, quick panels, and shortcut buttons.

```
const unsafeLibraries = useMemo(
 () => libraries.filter((l) => l.controlState === "unsafe"),
 [libraries]
);

const blockedLibraries = useMemo(
 () => libraries.filter((l) => l.blocked),
 [libraries]
);

const staleLibraries = useMemo(
 () => libraries.filter((l) => l.stale),
 [libraries]
);

const missingAuthorityLibraries = useMemo(
 () => libraries.filter((l) => l.missingAuthority),
 [libraries]
);

const driftLibraries = useMemo(
 () => libraries.filter((l) => l.driftAlert),
 [libraries]
);
```

Health bands for current filtered view

You have two valid options:

- keep the static health bands from the data pack
- derive visible counts from the filtered set

Best approach:

- use **static canonical bands** for the dedicated section
- optionally derive **visible counts** for quick chips

For v1, keep it simple:

```
const healthBands = libraryControlCenterState.healthBands;
```

Filter action methods

Query

```
const setQuery = useCallback((query: string) => {
 setFilters((prev) => ({ ...prev, query }));
}, []);
```

Authority filter

```
const setAuthorityFilter = useCallback((values?: AuthorityStatus[]) => {
 setFilters((prev) => ({ ...prev, authorityStatus: values && values.length ? values : undefined }));
}, []);
```

Control-state filter

```
const setControlStateFilter = useCallback((values?: ControlState[]) => {
 setFilters((prev) => ({ ...prev, controlState: values && values.length ? values : undefined }));
}, []);
```

Risk filter

```
const setRiskFilter = useCallback((values?: RiskLevel[]) => {
 setFilters((prev) => ({ ...prev, risk: values && values.length ? values : undefined }));
}, []);
```

Sort

```
const setSortBy = useCallback((sortBy?: LibraryControlFilters["sortBy"]) => {
 setFilters((prev) => ({ ...prev, sortBy }));
}, []);
```

Toggles

```
const toggleStaleOnly = useCallback(() => {
 setFilters((prev) => ({ ...prev, staleOnly: !prev.staleOnly }));
}, []);

const toggleBlockedOnly = useCallback(() => {
```

```
 setFilters((prev) => ({ ...prev, blockedOnly: !prev.blockedOnly }));
}, []);

const toggleMissingAuthorityOnly = useCallback(() => {
 setFilters((prev) => ({ ...prev, missingAuthorityOnly: !prev.missingAuthorityOnly }));
}, []);

const toggleDriftOnly = useCallback(() => {
 setFilters((prev) => ({ ...prev, driftOnly: !prev.driftOnly }));
}, []);
```

Clear all

```
const clearFilters = useCallback(() => {
 setFilters(libraryControlCenterState.defaultFilters);
}, []);
```

Quick presets

These should overwrite only the relevant filters and keep the UX fast.

```
const applyCriticalOnlyPreset = useCallback(() => {
 setFilters({
   ...libraryControlCenterState.defaultFilters,
   risk: ["critical", "high"],
   sortBy: "risk",
 });
}, []);

const applyUnsafePreset = useCallback(() => {
 setFilters({
   ...libraryControlCenterState.defaultFilters,
   controlState: ["unsafe"],
   sortBy: "risk",
 });
}, []);

const applyBlockedPreset = useCallback(() => {
 setFilters({
   ...libraryControlCenterState.defaultFilters,
   blockedOnly: true,
   sortBy: "dependency-pressure",
```

```
 });
}, []);

const applyMissingAuthorityPreset = useCallback(() => {
 setFilters({
   ...libraryControlCenterState.defaultFilters,
   missingAuthorityOnly: true,
   sortBy: "risk",
 });
}, []);

const applyDriftPreset = useCallback(() => {
 setFilters({
   ...libraryControlCenterState.defaultFilters,
   driftOnly: true,
   sortBy: "freshness",
 });
}, []);

const applyGovernedPreset = useCallback(() => {
 setFilters({
   ...libraryControlCenterState.defaultFilters,
   controlState: ["governed"],
   sortBy: "control-score",
 });
}, []);
```

Selection behavior

Select by row click

```
const selectLibrary = useCallback((libraryId: string) => {
 setSelectedLibraryId(libraryId);
}, []);
```

Clear selection

```
const clearSelectedLibrary = useCallback(() => {
 setSelectedLibraryId(null);
}, []);
```

Select first visible

```
const selectFirstVisibleLibrary = useCallback(() => {
 setSelectedLibraryId(filteredLibraries[0]?.id ?? null);
}, [filteredLibraries]);
```

Auto-heal invalid selection

If a filtered result no longer contains the selected library, fall back to first visible.

Use an effect:

```
useEffect(() => {
 if (!selectedLibraryId) return;

 const stillVisible = filteredLibraries.some((lib) => lib.id === selectedLibraryId);
 if (!stillVisible) {
   setSelectedLibraryId(filteredLibraries[0]?.id ?? null);
 }
}, [filteredLibraries, selectedLibraryId]);
```

You'll need to import `useEffect`.

Active filter detection

```
const hasActiveFilters = useMemo(() => {
 return Boolean(
   filters.query ||
     filters.authorityStatus?.length ||
     filters.controlState?.length ||
     filters.risk?.length ||
     filters.staleOnly ||
     filters.blockedOnly ||
     filters.missingAuthorityOnly ||
     filters.driftOnly
 );
}, [filters]);
```

Count helpers

```
const visibleCount = filteredLibraries.length;
const totalCount = libraries.length;
```

Action handlers

The current page is display-heavy; the new hook should expose action handlers even if they are stubbed in v1. The page already has route-capable library rows and action-minded sections like blockers/rules, so the hook should move the UX toward control actions rather than just table rendering.

runControlAction

For v1, this should apply useful filter presets or console/log placeholders.

```
const runControlAction = useCallback((actionType: ControlAction["actionType"]) => {
 switch (actionType) {
   case "health-check":
     clearFilters();
     break;

   case "coverage-audit":
     setFilters({
       ...libraryControlCenterState.defaultFilters,
       controlState: ["review-required", "degraded", "recovery"],
       sortBy: "coverage",
     });
     break;

   case "drift-detection":
     applyDriftPreset();
     break;

   case "rebuild-registries":
     setFilters({
       ...libraryControlCenterState.defaultFilters,
       blockedOnly: true,
       sortBy: "dependency-pressure",
     });
     break;

   case "review-authority-gaps":
```

```
    applyMissingAuthorityPreset();
    break;

  case "dependency-refresh":
    setFilters({
      ...libraryControlCenterState.defaultFilters,
      sortBy: "dependency-pressure",
    });
    break;

  case "open-unsafe":
    applyUnsafePreset();
    break;

  case "resolve-blockers":
    applyBlockedPreset();
    break;
 }
}, [
 clearFilters,
 applyDriftPreset,
 applyMissingAuthorityPreset,
 applyUnsafePreset,
 applyBlockedPreset,
]);
```

assignMaintenanceMode

For v1, keep this as a stub or no-op callback until there is real mutation support.

```
const assignMaintenanceMode = useCallback((libraryId: string, modeId: string) => {
 console.info("Assign maintenance mode", { libraryId, modeId });
}, []);
```

Full hook implementation

```
import { useCallback, useEffect, useMemo, useState } from "react";
import {
 libraryControlCenterState,
 type AuthorityStatus,
 type ControlState,
 type RiskLevel,
```

```
  type LibraryControlFilters,
  type LibraryControlRecord,
  type LibraryDetail,
  type ControlAction,
  type LibraryHealthBand,
  type EstateHeaderStats,
  type EstateStatusMetrics,
  type LeverageFix,
  type ActiveBlocker,
  type SystemRisk,
  type GovernanceRule,
  type MaintenanceModeCard,
  type DependencyLink,
  type LibraryActivityEvent,
  computeLibraryControlScore,
} from "../data/library-control";

export type UseLibraryControlReturn = {
  filters: LibraryControlFilters;
  selectedLibraryId: string | null;

  libraries: LibraryControlRecord[];
  filteredLibraries: LibraryControlRecord[];
  selectedLibrary: LibraryDetail | null;

  header: EstateHeaderStats;
  metrics: EstateStatusMetrics;
  healthBands: LibraryHealthBand[];

  leverageFixes: LeverageFix[];
  blockers: ActiveBlocker[];
  risks: SystemRisk[];
  governanceRules: GovernanceRule[];
  maintenanceModes: MaintenanceModeCard[];
  dependencyLinks: DependencyLink[];
  recentActivity: LibraryActivityEvent[];
  actions: ControlAction[];

  unsafeLibraries: LibraryControlRecord[];
  blockedLibraries: LibraryControlRecord[];
  staleLibraries: LibraryControlRecord[];
  missingAuthorityLibraries: LibraryControlRecord[];
  driftLibraries: LibraryControlRecord[];
```

```
  setQuery: (query: string) => void;
  setAuthorityFilter: (values?: AuthorityStatus[]) => void;
  setControlStateFilter: (values?: ControlState[]) => void;
  setRiskFilter: (values?: RiskLevel[]) => void;
  setSortBy: (sortBy?: LibraryControlFilters["sortBy"]) => void;

  toggleStaleOnly: () => void;
  toggleBlockedOnly: () => void;
  toggleMissingAuthorityOnly: () => void;
  toggleDriftOnly: () => void;

  clearFilters: () => void;

  applyCriticalOnlyPreset: () => void;
  applyUnsafePreset: () => void;
  applyBlockedPreset: () => void;
  applyMissingAuthorityPreset: () => void;
  applyDriftPreset: () => void;
  applyGovernedPreset: () => void;

  selectLibrary: (libraryId: string) => void;
  clearSelectedLibrary: () => void;
  selectFirstVisibleLibrary: () => void;

  getLibraryControlScore: (library: LibraryControlRecord) => number;
  getRiskWeight: (risk: RiskLevel) => number;

  runControlAction: (actionType: ControlAction["actionType"]) => void;
  assignMaintenanceMode: (libraryId: string, modeId: string) => void;

  hasActiveFilters: boolean;
  visibleCount: number;
  totalCount: number;
};

export function useLibraryControl(): UseLibraryControlReturn {
  const [filters, setFilters] = useState<LibraryControlFilters>(
    libraryControlCenterState.defaultFilters
  );

  const [selectedLibraryId, setSelectedLibraryId] = useState<string | null>(
    libraryControlCenterState.libraries[0]?.id ?? null
  );
```

```
const libraries = libraryControlCenterState.libraries;
const header = libraryControlCenterState.header;
const metrics = libraryControlCenterState.metrics;
const healthBands = libraryControlCenterState.healthBands;
const leverageFixes = libraryControlCenterState.leverageFixes;
const blockers = libraryControlCenterState.blockers;
const risks = libraryControlCenterState.risks;
const governanceRules = libraryControlCenterState.governanceRules;
const maintenanceModes = libraryControlCenterState.maintenanceModes;
const dependencyLinks = libraryControlCenterState.dependencyLinks;
const recentActivity = libraryControlCenterState.recentActivity;
const actions = libraryControlCenterState.actions;

const getRiskWeight = useCallback((risk: RiskLevel) => {
  switch (risk) {
    case "critical":
      return 4;
    case "high":
      return 3;
    case "moderate":
      return 2;
    case "low":
      return 1;
  }
}, []);

const getLibraryControlScore = useCallback((library: LibraryControlRecord) => {
  return computeLibraryControlScore({
    authorityStatus: library.authorityStatus,
    coveragePercent: library.coveragePercent,
    freshnessPercent: library.freshnessPercent,
    integrityPercent: library.integrityPercent,
    dependencyPressure: library.dependencyPressure,
  });
}, []);

const filteredLibraries = useMemo(() => {
  let result = [...libraries];

  const q = filters.query.trim().toLowerCase();
  if (q) {
    result = result.filter((lib) => {
      return (
        lib.name.toLowerCase().includes(q) ||
```

```
      lib.shortName.toLowerCase().includes(q) ||
      lib.owner?.toLowerCase().includes(q) ||
      lib.recommendedAction.toLowerCase().includes(q)
    );
  });
}

if (filters.authorityStatus?.length) {
  result = result.filter((lib) =>
    filters.authorityStatus!.includes(lib.authorityStatus)
  );
}

if (filters.controlState?.length) {
  result = result.filter((lib) =>
    filters.controlState!.includes(lib.controlState)
  );
}

if (filters.risk?.length) {
  result = result.filter((lib) => filters.risk!.includes(lib.risk));
}

if (filters.staleOnly) {
  result = result.filter((lib) => lib.stale);
}

if (filters.blockedOnly) {
  result = result.filter((lib) => lib.blocked);
}

if (filters.missingAuthorityOnly) {
  result = result.filter((lib) => lib.missingAuthority);
}

if (filters.driftOnly) {
  result = result.filter((lib) => lib.driftAlert);
}

result.sort((a, b) => {
  switch (filters.sortBy) {
    case "risk":
      return getRiskWeight(b.risk) - getRiskWeight(a.risk);
```

```
      case "dependency-pressure":
        return b.dependencyPressure - a.dependencyPressure;

      case "freshness":
        return a.freshnessPercent - b.freshnessPercent;

      case "coverage":
        return a.coveragePercent - b.coveragePercent;

      case "last-audit": {
        const aTime = a.lastAuditAt ? new Date(a.lastAuditAt).getTime() : 0;
        const bTime = b.lastAuditAt ? new Date(b.lastAuditAt).getTime() : 0;
        return aTime - bTime;
      }

      case "control-score":
        return getLibraryControlScore(a) - getLibraryControlScore(b);

      default:
        return getRiskWeight(b.risk) - getRiskWeight(a.risk);
    }
  });

  return result;
}, [libraries, filters, getRiskWeight, getLibraryControlScore]);

const selectedLibrary = useMemo(() => {
  if (!selectedLibraryId) return null;
  return libraryControlCenterState.libraryDetails[selectedLibraryId] ?? null;
}, [selectedLibraryId]);

const unsafeLibraries = useMemo(
  () => libraries.filter((l) => l.controlState === "unsafe"),
  [libraries]
);

const blockedLibraries = useMemo(
  () => libraries.filter((l) => l.blocked),
  [libraries]
);

const staleLibraries = useMemo(
  () => libraries.filter((l) => l.stale),
  [libraries]
```

```
);

const missingAuthorityLibraries = useMemo(
  () => libraries.filter((l) => l.missingAuthority),
  [libraries]
);

const driftLibraries = useMemo(
  () => libraries.filter((l) => l.driftAlert),
  [libraries]
);

const setQuery = useCallback((query: string) => {
  setFilters((prev) => ({ ...prev, query }));
}, []);

const setAuthorityFilter = useCallback((values?: AuthorityStatus[]) => {
  setFilters((prev) => ({
    ...prev,
    authorityStatus: values && values.length ? values : undefined,
  }));
}, []);

const setControlStateFilter = useCallback((values?: ControlState[]) => {
  setFilters((prev) => ({
    ...prev,
    controlState: values && values.length ? values : undefined,
  }));
}, []);

const setRiskFilter = useCallback((values?: RiskLevel[]) => {
  setFilters((prev) => ({
    ...prev,
    risk: values && values.length ? values : undefined,
  }));
}, []);

const setSortBy = useCallback((sortBy?: LibraryControlFilters["sortBy"]) => {
  setFilters((prev) => ({ ...prev, sortBy }));
}, []);

const toggleStaleOnly = useCallback(() => {
  setFilters((prev) => ({ ...prev, staleOnly: !prev.staleOnly }));
}, []);
```

```
const toggleBlockedOnly = useCallback(() => {
  setFilters((prev) => ({ ...prev, blockedOnly: !prev.blockedOnly }));
}, []);

const toggleMissingAuthorityOnly = useCallback(() => {
  setFilters((prev) => ({
    ...prev,
    missingAuthorityOnly: !prev.missingAuthorityOnly,
  }));
}, []);

const toggleDriftOnly = useCallback(() => {
  setFilters((prev) => ({ ...prev, driftOnly: !prev.driftOnly }));
}, []);

const clearFilters = useCallback(() => {
  setFilters(libraryControlCenterState.defaultFilters);
}, []);

const applyCriticalOnlyPreset = useCallback(() => {
  setFilters({
    ...libraryControlCenterState.defaultFilters,
    risk: ["critical", "high"],
    sortBy: "risk",
  });
}, []);

const applyUnsafePreset = useCallback(() => {
  setFilters({
    ...libraryControlCenterState.defaultFilters,
    controlState: ["unsafe"],
    sortBy: "risk",
  });
}, []);

const applyBlockedPreset = useCallback(() => {
  setFilters({
    ...libraryControlCenterState.defaultFilters,
    blockedOnly: true,
    sortBy: "dependency-pressure",
  });
}, []);
```

```
const applyMissingAuthorityPreset = useCallback(() => {
  setFilters({
    ...libraryControlCenterState.defaultFilters,
    missingAuthorityOnly: true,
    sortBy: "risk",
  });
}, []);

const applyDriftPreset = useCallback(() => {
  setFilters({
    ...libraryControlCenterState.defaultFilters,
    driftOnly: true,
    sortBy: "freshness",
  });
}, []);

const applyGovernedPreset = useCallback(() => {
  setFilters({
    ...libraryControlCenterState.defaultFilters,
    controlState: ["governed"],
    sortBy: "control-score",
  });
}, []);

const selectLibrary = useCallback((libraryId: string) => {
  setSelectedLibraryId(libraryId);
}, []);

const clearSelectedLibrary = useCallback(() => {
  setSelectedLibraryId(null);
}, []);

const selectFirstVisibleLibrary = useCallback(() => {
  setSelectedLibraryId(filteredLibraries[0]?.id ?? null);
}, [filteredLibraries]);

useEffect(() => {
  if (!selectedLibraryId) return;
  const stillVisible = filteredLibraries.some((lib) => lib.id === selectedLibraryId);
  if (!stillVisible) {
    setSelectedLibraryId(filteredLibraries[0]?.id ?? null);
  }
}, [filteredLibraries, selectedLibraryId]);
```

```
const runControlAction = useCallback((actionType: ControlAction["actionType"]) => {
  switch (actionType) {
    case "health-check":
      clearFilters();
      break;
    case "coverage-audit":
      setFilters({
        ...libraryControlCenterState.defaultFilters,
        controlState: ["review-required", "degraded", "recovery"],
        sortBy: "coverage",
      });
      break;
    case "drift-detection":
      applyDriftPreset();
      break;
    case "rebuild-registries":
      setFilters({
        ...libraryControlCenterState.defaultFilters,
        blockedOnly: true,
        sortBy: "dependency-pressure",
      });
      break;
    case "review-authority-gaps":
      applyMissingAuthorityPreset();
      break;
    case "dependency-refresh":
      setFilters({
        ...libraryControlCenterState.defaultFilters,
        sortBy: "dependency-pressure",
      });
      break;
    case "open-unsafe":
      applyUnsafePreset();
      break;
    case "resolve-blockers":
      applyBlockedPreset();
      break;
  }
}, [
  clearFilters,
  applyDriftPreset,
  applyMissingAuthorityPreset,
  applyUnsafePreset,
  applyBlockedPreset,
```

```
]);

const assignMaintenanceMode = useCallback((libraryId: string, modeId: string) => {
  console.info("Assign maintenance mode", { libraryId, modeId });
}, []);

const hasActiveFilters = useMemo(() => {
  return Boolean(
    filters.query ||
      filters.authorityStatus?.length ||
      filters.controlState?.length ||
      filters.risk?.length ||
      filters.staleOnly ||
      filters.blockedOnly ||
      filters.missingAuthorityOnly ||
      filters.driftOnly
  );
}, [filters]);

return {
  filters,
  selectedLibraryId,

  libraries,
  filteredLibraries,
  selectedLibrary,

  header,
  metrics,
  healthBands,

  leverageFixes,
  blockers,
  risks,
  governanceRules,
  maintenanceModes,
  dependencyLinks,
  recentActivity,
  actions,

  unsafeLibraries,
  blockedLibraries,
  staleLibraries,
  missingAuthorityLibraries,
```

```
  driftLibraries,

  setQuery,
  setAuthorityFilter,
  setControlStateFilter,
  setRiskFilter,
  setSortBy,

  toggleStaleOnly,
  toggleBlockedOnly,
  toggleMissingAuthorityOnly,
  toggleDriftOnly,

  clearFilters,

  applyCriticalOnlyPreset,
  applyUnsafePreset,
  applyBlockedPreset,
  applyMissingAuthorityPreset,
  applyDriftPreset,
  applyGovernedPreset,

  selectLibrary,
  clearSelectedLibrary,
  selectFirstVisibleLibrary,

  getLibraryControlScore,
  getRiskWeight,

  runControlAction,
  assignMaintenanceMode,

  hasActiveFilters,
  visibleCount: filteredLibraries.length,
  totalCount: libraries.length,
 };
}
```

What the page should consume from the hook

Your new page should mostly destructure this set:

```
const {
 header,
 metrics,
 actions,
 healthBands,
 filteredLibraries,
 selectedLibrary,
 leverageFixes,
 blockers,
 risks,
 governanceRules,
 maintenanceModes,
 dependencyLinks,
 recentActivity,
 filters,
 setQuery,
 setAuthorityFilter,
 setControlStateFilter,
 setRiskFilter,
 setSortBy,
 toggleStaleOnly,
 toggleBlockedOnly,
 toggleMissingAuthorityOnly,
 toggleDriftOnly,
 clearFilters,
 selectLibrary,
 runControlAction,
 hasActiveFilters,
 visibleCount,
 totalCount,
} = useLibraryControl();
```

Hook responsibilities vs page responsibilities

Hook owns

- filtering logic

- sorting logic
- selected library
- subset derivations
- score derivations
- action-to-filter behaviors
- quick presets

Page owns

- layout
- rendering cards
- rendering table
- rendering detail panel
- rendering buttons and calling returned handlers
- route navigation if a row's `route` is opened

That split is cleaner than the current page, where display structure and upgrade behavior are tightly interwoven.

page skeleton

Here's the `library-control.tsx` **page skeleton** wired to the new hook.

It is structured to replace the old upgrade page with the new **Library Control Center** layout, but it still uses placeholders where your app-specific UI primitives or route helpers may differ.

```
// App/src/pages/library-control.tsx

import { useMemo } from "react";
import {
 AlertTriangle,
 ArrowRight,
 BookOpen,
 Bot,
 Database,
 Filter,
 GitBranch,
 HeartPulse,
 Library,
 RefreshCcw,
 Search,
 Shield,
 ShieldAlert,
 ShieldCheck,
 Siren,
 TriangleAlert,
 Wrench,
} from "lucide-react";

import { useLibraryControl } from "../hooks/use-library-control";

// Reuse your existing UI primitives if these already exist in your app.
// Update import paths as needed.
import { GlassPanel } from "../components/glass-panel";
import { MetricCard } from "../components/metric-card";
import { StatusChip } from "../components/status-chip";

type BadgeTone = "default" | "success" | "warning" | "danger" | "info";

function getRiskTone(risk: string): BadgeTone {
 switch (risk) {
   case "critical":
     return "danger";
   case "high":
     return "warning";
```

```
    case "moderate":
      return "info";
    case "low":
    default:
      return "success";
  }
}

function getControlStateTone(state: string): BadgeTone {
  switch (state) {
    case "unsafe":
      return "danger";
    case "blocked":
      return "warning";
    case "degraded":
      return "warning";
    case "review-required":
      return "info";
    case "recovery":
      return "info";
    case "governed":
    default:
      return "success";
  }
}

function getAuthorityTone(status: string): BadgeTone {
  switch (status) {
    case "authoritative":
      return "success";
    case "partial":
      return "info";
    case "disputed":
      return "warning";
    case "missing":
    default:
      return "danger";
  }
}

function getActionIcon(label: string) {
  if (label.toLowerCase().includes("health")) return HeartPulse;
  if (label.toLowerCase().includes("coverage")) return Search;
  if (label.toLowerCase().includes("drift")) return RefreshCcw;
```

```
 if (label.toLowerCase().includes("registry")) return Database;
 if (label.toLowerCase().includes("authority")) return ShieldAlert;
 if (label.toLowerCase().includes("depend")) return GitBranch;
 if (label.toLowerCase().includes("unsafe")) return TriangleAlert;
 return Wrench;
}

function formatPercent(value: number) {
 return `${Math.round(value)}%`;
}

export default function LibraryControlCenterPage() {
 const {
   header,
   metrics,
   actions,
   healthBands,
   filteredLibraries,
   selectedLibrary,
   leverageFixes,
   blockers,
   risks,
   governanceRules,
   maintenanceModes,
   dependencyLinks,
   recentActivity,

   filters,
   setQuery,
   setAuthorityFilter,
   setControlStateFilter,
   setRiskFilter,
   setSortBy,

   toggleStaleOnly,
   toggleBlockedOnly,
   toggleMissingAuthorityOnly,
   toggleDriftOnly,

   clearFilters,
   selectLibrary,
   runControlAction,
   assignMaintenanceMode,
```

```
  hasActiveFilters,
  visibleCount,
  totalCount,
  getLibraryControlScore,
} = useLibraryControl();

const dependencySummary = useMemo(() => {
  const governedLinks = dependencyLinks.filter((l) => l.relation === "governs").length;
  const blockedLinks = dependencyLinks.filter((l) => l.relation === "blocks").length;
  const supportLinks = dependencyLinks.filter((l) => l.relation === "supports").length;

  return {
    total: dependencyLinks.length,
    governedLinks,
    blockedLinks,
    supportLinks,
  };
}, [dependencyLinks]);

return (
  <div className="space-y-6 p-6">
    {/* Hero Header */}
    <GlassPanel className="rounded-3xl p-6">
      <div className="flex flex-col gap-6 lg:flex-row lg:items-start lg:justify-between">
        <div className="space-y-3">
          <div className="flex items-center gap-3">
            <div className="rounded-2xl border border-white/10 bg-white/5 p-3">
              <Library className="h-6 w-6" />
            </div>
            <div>
              <div className="text-xs uppercase tracking-[0.25em] text-white/50">
                Control Authority
              </div>
              <h1 className="text-3xl font-semibold tracking-tight">
                Library Control Center
              </h1>
            </div>
          </div>

          <p className="max-w-3xl text-sm text-white/70">
            Central authority for governing, inspecting, maintaining, and directing
            the Axion library system.
          </p>
        </div>
```

```
    <div className="grid min-w-[320px] grid-cols-2 gap-3">
      <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
        <div className="text-xs uppercase tracking-wide text-white/50">
          Control Score
        </div>
        <div className="mt-2 text-2xl font-semibold">{header.controlScore}</div>
      </div>
      <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
        <div className="text-xs uppercase tracking-wide text-white/50">
          Governed Libraries
        </div>
        <div className="mt-2 text-2xl font-semibold">{header.governedLibraries}</div>
      </div>
      <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
        <div className="text-xs uppercase tracking-wide text-white/50">
          Active Risks
        </div>
        <div className="mt-2 text-2xl font-semibold">{header.activeRisks}</div>
      </div>
      <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
        <div className="text-xs uppercase tracking-wide text-white/50">
          Blocked Dependencies
        </div>
        <div className="mt-2 text-2xl font-semibold">
          {header.blockedDependencies}
        </div>
      </div>
    </div>
  </div>
</GlassPanel>

{/* Estate Status Cards */}
<section className="space-y-3">
  <div className="flex items-center justify-between">
    <h2 className="text-lg font-semibold">Estate Status</h2>
    <div className="text-sm text-white/50">
      {visibleCount} visible / {totalCount} total
    </div>
  </div>

  <div className="grid gap-4 sm:grid-cols-2 xl:grid-cols-4">
    <MetricCard title="Total Libraries" value={metrics.totalLibraries} icon={BookOpen} />
    <MetricCard title="Governed" value={metrics.governed} icon={ShieldCheck} />
```

```
        <MetricCard title="Review Required" value={metrics.reviewRequired} icon={Shield} />
        <MetricCard title="Stale" value={metrics.stale} icon={RefreshCcw} />
        <MetricCard title="Unsafe" value={metrics.unsafe} icon={TriangleAlert} />
        <MetricCard title="Blocked" value={metrics.blocked} icon={AlertTriangle} />
        <MetricCard title="Missing Authority" value={metrics.missingAuthority} icon={ShieldAlert}
/>
        <MetricCard title="Drift Alerts" value={metrics.driftAlerts} icon={Siren} />
      </div>
    </section>

    {/* Control Actions */}
    <GlassPanel className="rounded-3xl p-5">
      <div className="mb-4 flex items-center gap-2">
        <Bot className="h-5 w-5" />
        <h2 className="text-lg font-semibold">Control Actions</h2>
      </div>

      <div className="grid gap-3 sm:grid-cols-2 xl:grid-cols-4">
        {actions.map((action) => {
          const Icon = getActionIcon(action.label);
          return (
            <button
              key={action.id}
              type="button"
              onClick={() => runControlAction(action.actionType)}
              disabled={!action.enabled}
              className="flex items-center justify-between rounded-2xl border border-white/10
bg-white/5 p-4 text-left transition hover:bg-white/10 disabled:cursor-not-allowed
disabled:opacity-50"
            >
              <div className="flex items-start gap-3">
                <div className="rounded-xl border border-white/10 bg-white/5 p-2">
                  <Icon className="h-4 w-4" />
                </div>
                <div>
                  <div className="font-medium">{action.label}</div>
                  <div className="text-sm text-white/50">Launch control workflow</div>
                </div>
              </div>
              <ArrowRight className="h-4 w-4 text-white/40" />
            </button>
          );
        })}
      </div>
```

```
    </GlassPanel>

    {/* Library Health Overview */}
    <section className="space-y-3">
      <h2 className="text-lg font-semibold">Library Health Overview</h2>

      <div className="grid gap-4 md:grid-cols-2 xl:grid-cols-3">
        {healthBands.map((band) => (
          <GlassPanel key={band.id} className="rounded-3xl p-5">
            <div className="flex items-start justify-between gap-4">
              <div>
                <div className="text-sm uppercase tracking-wide text-white/50">
                  {band.label}
                </div>
                <div className="mt-1 text-2xl font-semibold">{band.count}</div>
              </div>
              <StatusChip label={band.id} tone="info" />
            </div>

            <div className="mt-4 space-y-2">
              <div className="text-sm text-white/60">
                Recommended action
              </div>
              <p className="text-sm text-white/80">{band.recommendedAction}</p>
            </div>

            <div className="mt-4 flex flex-wrap gap-2">
              {band.libraryIds.map((id) => (
                <span
                  key={id}
                  className="rounded-full border border-white/10 bg-white/5 px-3 py-1 text-xs
text-white/70"
                >
                  {id}
                </span>
              ))}
            </div>
          </GlassPanel>
        ))}
      </div>
    </section>

    {/* Filters + Table */}
    <GlassPanel className="rounded-3xl p-5">
```

```
      <div className="mb-5 flex flex-col gap-4 xl:flex-row xl:items-center xl:justify-between">
        <div>
          <h2 className="text-lg font-semibold">Library Estate</h2>
          <p className="text-sm text-white/50">
            Inspect authority, health, risk, and recommended actions across all libraries.
          </p>
        </div>

        {hasActiveFilters && (
          <button
            type="button"
            onClick={clearFilters}
            className="rounded-xl border border-white/10 bg-white/5 px-4 py-2 text-sm transition
hover:bg-white/10"
          >
            Clear Filters
          </button>
        )}
      </div>

      {/* Filter Bar */}
      <div className="mb-5 grid gap-3 xl:grid-cols-12">
        <div className="xl:col-span-4">
          <label className="mb-2 block text-xs uppercase tracking-wide text-white/50">
            Search
          </label>
          <div className="flex items-center gap-2 rounded-2xl border border-white/10 bg-white/5
px-3 py-2">
            <Search className="h-4 w-4 text-white/40" />
            <input
              value={filters.query}
              onChange={(e) => setQuery(e.target.value)}
              placeholder="Search libraries, owners, or actions"
              className="w-full bg-transparent text-sm outline-none placeholder:text-white/30"
            />
          </div>
        </div>

        <div className="xl:col-span-2">
          <label className="mb-2 block text-xs uppercase tracking-wide text-white/50">
            Authority
          </label>
          <select
```

```
          className="w-full rounded-2xl border border-white/10 bg-white/5 px-3 py-2 text-sm
outline-none"
          onChange={(e) =>
            setAuthorityFilter(
              e.target.value ? [e.target.value as any] : undefined
            )
          }
          value={filters.authorityStatus?.[0] ?? ""}
        >
          <option value="">All</option>
          <option value="authoritative">Authoritative</option>
          <option value="partial">Partial</option>
          <option value="missing">Missing</option>
          <option value="disputed">Disputed</option>
        </select>
      </div>

      <div className="xl:col-span-2">
        <label className="mb-2 block text-xs uppercase tracking-wide text-white/50">
          Control State
        </label>
        <select
          className="w-full rounded-2xl border border-white/10 bg-white/5 px-3 py-2 text-sm
outline-none"
          onChange={(e) =>
            setControlStateFilter(
              e.target.value ? [e.target.value as any] : undefined
            )
          }
          value={filters.controlState?.[0] ?? ""}
        >
          <option value="">All</option>
          <option value="governed">Governed</option>
          <option value="review-required">Review Required</option>
          <option value="degraded">Degraded</option>
          <option value="blocked">Blocked</option>
          <option value="unsafe">Unsafe</option>
          <option value="recovery">Recovery</option>
        </select>
      </div>

      <div className="xl:col-span-2">
        <label className="mb-2 block text-xs uppercase tracking-wide text-white/50">
          Risk
```

```
          </label>
          <select
            className="w-full rounded-2xl border border-white/10 bg-white/5 px-3 py-2 text-sm
outline-none"
            onChange={(e) =>
              setRiskFilter(e.target.value ? [e.target.value as any] : undefined)
            }
            value={filters.risk?.[0] ?? ""}
          >
            <option value="">All</option>
            <option value="critical">Critical</option>
            <option value="high">High</option>
            <option value="moderate">Moderate</option>
            <option value="low">Low</option>
          </select>
        </div>

        <div className="xl:col-span-2">
          <label className="mb-2 block text-xs uppercase tracking-wide text-white/50">
            Sort
          </label>
          <select
            className="w-full rounded-2xl border border-white/10 bg-white/5 px-3 py-2 text-sm
outline-none"
            onChange={(e) => setSortBy((e.target.value || undefined) as any)}
            value={filters.sortBy ?? ""}
          >
            <option value="risk">Risk</option>
            <option value="dependency-pressure">Dependency Pressure</option>
            <option value="freshness">Freshness</option>
            <option value="coverage">Coverage</option>
            <option value="last-audit">Last Audit</option>
            <option value="control-score">Control Score</option>
          </select>
        </div>
      </div>

      {/* Quick Toggle Filters */}
      <div className="mb-5 flex flex-wrap items-center gap-2">
        <div className="mr-2 flex items-center gap-2 text-xs uppercase tracking-wide
text-white/50">
          <Filter className="h-3.5 w-3.5" />
          Quick Filters
        </div>
```

```
  <button
    type="button"
    onClick={toggleStaleOnly}
    className={`rounded-full px-3 py-1.5 text-xs ${
      filters.staleOnly ? "bg-white text-black" : "border border-white/10 bg-white/5"
    }`}
  >
    Stale Only
  </button>

  <button
    type="button"
    onClick={toggleBlockedOnly}
    className={`rounded-full px-3 py-1.5 text-xs ${
      filters.blockedOnly ? "bg-white text-black" : "border border-white/10 bg-white/5"
    }`}
  >
    Blocked Only
  </button>

  <button
    type="button"
    onClick={toggleMissingAuthorityOnly}
    className={`rounded-full px-3 py-1.5 text-xs ${
      filters.missingAuthorityOnly
        ? "bg-white text-black"
        : "border border-white/10 bg-white/5"
    }`}
  >
    Missing Authority
  </button>

  <button
    type="button"
    onClick={toggleDriftOnly}
    className={`rounded-full px-3 py-1.5 text-xs ${
      filters.driftOnly ? "bg-white text-black" : "border border-white/10 bg-white/5"
    }`}
  >
    Drift Only
  </button>
</div>
```

```
      {/* Library Table */}
      <div className="overflow-x-auto">
        <table className="min-w-full border-separate border-spacing-y-2">
          <thead>
            <tr className="text-left text-xs uppercase tracking-wide text-white/45">
              <th className="px-3 py-2">Library</th>
              <th className="px-3 py-2">Authority</th>
              <th className="px-3 py-2">Coverage</th>
              <th className="px-3 py-2">Freshness</th>
              <th className="px-3 py-2">Integrity</th>
              <th className="px-3 py-2">Dependency Pressure</th>
              <th className="px-3 py-2">Risk</th>
              <th className="px-3 py-2">Control State</th>
              <th className="px-3 py-2">Score</th>
              <th className="px-3 py-2">Recommended Action</th>
            </tr>
          </thead>
          <tbody>
            {filteredLibraries.map((library) => (
              <tr
                key={library.id}
                onClick={() => selectLibrary(library.id)}
                className="cursor-pointer rounded-2xl border border-white/10 bg-white/5 transition
hover:bg-white/10"
              >
                <td className="rounded-l-2xl px-3 py-4">
                  <div className="font-medium">{library.name}</div>
                  <div className="text-sm text-white/50">{library.shortName}</div>
                </td>

                <td className="px-3 py-4">
                  <StatusChip
                    label={library.authorityStatus}
                    tone={getAuthorityTone(library.authorityStatus)}
                  />
                </td>

                <td className="px-3 py-4 text-sm">{formatPercent(library.coveragePercent)}</td>
                <td className="px-3 py-4 text-sm">{formatPercent(library.freshnessPercent)}</td>
                <td className="px-3 py-4 text-sm">{formatPercent(library.integrityPercent)}</td>
                <td className="px-3 py-4
text-sm">{formatPercent(library.dependencyPressure)}</td>

                <td className="px-3 py-4">
```

```
              <StatusChip label={library.risk} tone={getRiskTone(library.risk)} />
            </td>

            <td className="px-3 py-4">
              <StatusChip
                label={library.controlState}
                tone={getControlStateTone(library.controlState)}
              />
            </td>

            <td className="px-3 py-4 text-sm">{getLibraryControlScore(library)}</td>

            <td className="rounded-r-2xl px-3 py-4">
              <div className="max-w-[260px] text-sm text-white/75">
                {library.recommendedAction}
              </div>
            </td>
          </tr>
        ))}
      </tbody>
    </table>

    {filteredLibraries.length === 0 && (
      <div className="rounded-2xl border border-dashed border-white/10 bg-white/5 p-8
text-center text-sm text-white/50">
        No libraries match the current filters.
      </div>
    )}
  </div>
</GlassPanel>

{/* Selected Library Detail */}
<GlassPanel className="rounded-3xl p-5">
  <div className="mb-4 flex items-center gap-2">
    <Shield className="h-5 w-5" />
    <h2 className="text-lg font-semibold">Selected Library Detail</h2>
  </div>

  {!selectedLibrary ? (
    <div className="rounded-2xl border border-dashed border-white/10 bg-white/5 p-8
text-sm text-white/50">
      Select a library to inspect governance state, gaps, dependencies, and actions.
    </div>
  ) : (
```

```
        <div className="grid gap-5 xl:grid-cols-12">
          <div className="space-y-5 xl:col-span-4">
            <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
              <div className="text-xs uppercase tracking-wide text-white/50">Identity</div>
              <div className="mt-2 text-xl font-semibold">{selectedLibrary.name}</div>
              <div className="mt-1 text-sm text-white/50">{selectedLibrary.shortName}</div>
              <div className="mt-4 space-y-2 text-sm text-white/75">
                <div><span className="text-white/50">Owner:</span> {selectedLibrary.owner ??
"—"}</div>
                <div><span className="text-white/50">Role:</span> {selectedLibrary.estateRole ??
"—"}</div>
                <div><span className="text-white/50">Route:</span> {selectedLibrary.route ??
"—"}</div>
              </div>
            </div>

            <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
              <div className="text-xs uppercase tracking-wide text-white/50">Governance
State</div>
              <div className="mt-3 flex flex-wrap gap-2">
                <StatusChip
                  label={selectedLibrary.authorityStatus}
                  tone={getAuthorityTone(selectedLibrary.authorityStatus)}
                />
                <StatusChip
                  label={selectedLibrary.trustDecision}
                  tone={selectedLibrary.trustDecision === "trusted" ? "success" :
selectedLibrary.trustDecision === "conditional" ? "info" : "danger"}
                />
              </div>

              <div className="mt-4 text-sm text-white/75">
                <div className="mb-2 text-white/50">Authority Source</div>
                <div>{selectedLibrary.authoritySource ?? "No authoritative source assigned."}</div>
              </div>
            </div>

            <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
              <div className="text-xs uppercase tracking-wide text-white/50">Recommended
Action</div>
              <p className="mt-3 text-sm
text-white/80">{selectedLibrary.recommendedAction}</p>

              <div className="mt-4">
```

```
              <div className="mb-2 text-xs uppercase tracking-wide text-white/50">
                Suggested Maintenance Mode
              </div>
              <div className="flex items-center justify-between rounded-xl border border-white/10
bg-black/20 px-3 py-2">
                <span className="text-sm">{selectedLibrary.suggestedMaintenanceMode ??
"—"}</span>
                <button
                  type="button"
                  className="rounded-lg border border-white/10 bg-white/5 px-3 py-1.5 text-xs
hover:bg-white/10"
                  onClick={() =>
                    selectedLibrary.suggestedMaintenanceMode &&
                    assignMaintenanceMode(
                      selectedLibrary.id,
                      selectedLibrary.suggestedMaintenanceMode
                    )
                  }
                >
                  Assign
                </button>
              </div>
            </div>
          </div>
        </div>

        <div className="space-y-5 xl:col-span-8">
          <div className="grid gap-4 md:grid-cols-4">
            <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
              <div className="text-xs uppercase tracking-wide text-white/50">Coverage</div>
              <div className="mt-2 text-2xl font-semibold">
                {formatPercent(selectedLibrary.coveragePercent)}
              </div>
            </div>
            <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
              <div className="text-xs uppercase tracking-wide text-white/50">Freshness</div>
              <div className="mt-2 text-2xl font-semibold">
                {formatPercent(selectedLibrary.freshnessPercent)}
              </div>
            </div>
            <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
              <div className="text-xs uppercase tracking-wide text-white/50">Integrity</div>
              <div className="mt-2 text-2xl font-semibold">
                {formatPercent(selectedLibrary.integrityPercent)}
```

```
            </div>
          </div>
          <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
            <div className="text-xs uppercase tracking-wide text-white/50">Dependency
Fitness</div>
            <div className="mt-2 text-2xl font-semibold">
              {formatPercent(selectedLibrary.dependencyFitnessPercent)}
            </div>
          </div>
        </div>

        <div className="grid gap-4 md:grid-cols-2">
          <DetailListCard title="Missing Artifacts" items={selectedLibrary.missingArtifacts} />
          <DetailListCard title="Stale Artifacts" items={selectedLibrary.staleArtifacts} />
          <DetailListCard title="Broken References" items={selectedLibrary.brokenReferences}
/>
          <DetailListCard title="Unresolved Conflicts"
items={selectedLibrary.unresolvedConflicts} />
          <DetailListCard title="Upstream Dependencies"
items={selectedLibrary.upstreamDependencies} />
          <DetailListCard title="Downstream Dependents"
items={selectedLibrary.downstreamDependents} />
        </div>
      </div>
    </div>
  )}
</GlassPanel>

{/* Lower Grid */}
<div className="grid gap-6 xl:grid-cols-12">
  <div className="space-y-6 xl:col-span-7">
    {/* Highest Leverage Fixes */}
    <GlassPanel className="rounded-3xl p-5">
      <h2 className="mb-4 text-lg font-semibold">Highest Leverage Fixes</h2>

      <div className="space-y-3">
        {leverageFixes.map((fix) => (
          <div
            key={fix.id}
            className="rounded-2xl border border-white/10 bg-white/5 p-4"
          >
            <div className="flex items-start justify-between gap-4">
              <div>
                <div className="font-medium">{fix.title}</div>
```

```
                <div className="mt-1 text-sm text-white/60">{fix.summary}</div>
              </div>
              <div className="rounded-full border border-white/10 bg-black/20 px-3 py-1 text-xs
text-white/60">
                {fix.payoffScore}
              </div>
            </div>

            <div className="mt-4 flex flex-wrap gap-2">
              <StatusChip label={fix.impactRadius} tone="info" />
              <StatusChip label={fix.effort} tone="warning" />
            </div>

            <div className="mt-4 text-sm text-white/75">
              {fix.recommendedAction}
            </div>
          </div>
        ))}
      </div>
    </GlassPanel>

    {/* Blockers and Risks */}
    <GlassPanel className="rounded-3xl p-5">
      <h2 className="mb-4 text-lg font-semibold">Blockers and Risks</h2>

      <div className="grid gap-4 md:grid-cols-2">
        <div className="space-y-3">
          <div className="text-sm font-medium text-white/80">Active Blockers</div>
          {blockers.map((blocker) => (
            <div
              key={blocker.id}
              className="rounded-2xl border border-white/10 bg-white/5 p-4"
            >
              <div className="flex items-center justify-between gap-3">
                <div className="font-medium">{blocker.title}</div>
                <StatusChip label={blocker.severity} tone={getRiskTone(blocker.severity)} />
              </div>

              <ul className="mt-3 space-y-2 text-sm text-white/70">
                {blocker.items.map((item) => (
                  <li key={item}>• {item}</li>
                ))}
              </ul>
            </div>
```

```
      ))}
    </div>

    <div className="space-y-3">
      <div className="text-sm font-medium text-white/80">System Risks</div>
      {risks.map((risk) => (
        <div
          key={risk.id}
          className="rounded-2xl border border-white/10 bg-white/5 p-4"
        >
          <div className="flex items-center justify-between gap-3">
            <div className="font-medium">{risk.title}</div>
            <StatusChip label={risk.severity} tone={getRiskTone(risk.severity)} />
          </div>
          <p className="mt-3 text-sm text-white/70">{risk.summary}</p>
        </div>
      ))}
    </div>
  </div>
</GlassPanel>

{/* Governance Rules */}
<GlassPanel className="rounded-3xl p-5">
  <h2 className="mb-4 text-lg font-semibold">Library Governance Rules</h2>

  <div className="space-y-3">
    {governanceRules.map((rule) => (
      <div
        key={rule.id}
        className="rounded-2xl border border-white/10 bg-white/5 p-4"
      >
        <div className="flex items-center justify-between gap-3">
          <div className="font-medium">{rule.title}</div>
          <StatusChip label={rule.severity} tone="info" />
        </div>
        <p className="mt-2 text-sm text-white/70">{rule.description}</p>
      </div>
    ))}
  </div>
</GlassPanel>
</div>

<div className="space-y-6 xl:col-span-5">
  {/* Maintenance Modes */}
```

```
<GlassPanel className="rounded-3xl p-5">
  <h2 className="mb-4 text-lg font-semibold">Maintenance Modes</h2>

  <div className="space-y-3">
    {maintenanceModes.map((mode) => (
      <div
        key={mode.id}
        className="rounded-2xl border border-white/10 bg-white/5 p-4"
      >
        <div className="flex items-center justify-between gap-3">
          <div>
            <div className="font-medium">
              {mode.id} — {mode.name}
            </div>
            <div className="mt-1 text-sm text-white/60">{mode.purpose}</div>
          </div>
          <StatusChip
            label={mode.launchable ? "launchable" : "locked"}
            tone={mode.launchable ? "success" : "warning"}
          />
        </div>

        <div className="mt-3 text-sm text-white/75">
          {mode.triggerCondition}
        </div>
      </div>
    ))}
  </div>
</GlassPanel>

{/* Dependency and Authority Map Summary */}
<GlassPanel className="rounded-3xl p-5">
  <h2 className="mb-4 text-lg font-semibold">Dependency and Authority Map</h2>

  <div className="grid gap-3 sm:grid-cols-2">
    <MiniStat label="Total Links" value={dependencySummary.total} />
    <MiniStat label="Governance Links" value={dependencySummary.governedLinks} />
    <MiniStat label="Blocked Links" value={dependencySummary.blockedLinks} />
    <MiniStat label="Support Links" value={dependencySummary.supportLinks} />
  </div>

  <div className="mt-4 space-y-2">
    {dependencyLinks.slice(0, 8).map((link, index) => (
      <div
```

```
            key={`${link.fromLibraryId}-${link.toLibraryId}-${index}`}
            className="flex items-center justify-between rounded-xl border border-white/10
bg-white/5 px-3 py-2 text-sm"
          >
            <div className="text-white/75">
              {link.fromLibraryId} → {link.toLibraryId}
            </div>
            <div className="flex items-center gap-2">
              <StatusChip label={link.relation} tone="info" />
              {link.severity && (
                <StatusChip label={link.severity} tone={getRiskTone(link.severity)} />
              )}
            </div>
          </div>
        ))}
      </div>
    </GlassPanel>

    {/* Recent Activity */}
    <GlassPanel className="rounded-3xl p-5">
      <h2 className="mb-4 text-lg font-semibold">Recent Library Activity</h2>

      <div className="space-y-3">
        {recentActivity.map((event) => (
          <div
            key={event.id}
            className="rounded-2xl border border-white/10 bg-white/5 p-4"
          >
            <div className="flex items-center justify-between gap-3">
              <div className="font-medium">{event.title}</div>
              <div className="text-xs text-white/45">{event.timestamp}</div>
            </div>

            <p className="mt-2 text-sm text-white/70">{event.summary}</p>

            <div className="mt-3 flex items-center gap-2">
              <StatusChip label={event.type} tone="info" />
              {event.libraryId && <StatusChip label={event.libraryId} tone="default" />}
            </div>
          </div>
        ))}
      </div>
    </GlassPanel>
  </div>
```

```
      </div>
    </div>
  );
}

function DetailListCard({
  title,
  items,
}: {
  title: string;
  items: string[];
}) {
  return (
    <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
      <div className="text-sm font-medium">{title}</div>
      <div className="mt-3">
        {items.length === 0 ? (
          <div className="text-sm text-white/45">None</div>
        ) : (
          <ul className="space-y-2 text-sm text-white/70">
            {items.map((item) => (
              <li key={item}>• {item}</li>
            ))}
          </ul>
        )}
      </div>
    </div>
  );
}

function MiniStat({
  label,
  value,
}: {
  label: string;
  value: number;
}) {
  return (
    <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
      <div className="text-xs uppercase tracking-wide text-white/50">{label}</div>
      <div className="mt-2 text-2xl font-semibold">{value}</div>
    </div>
  );
}
```

What this gives you

This skeleton already includes the full repurpose structure:

- **Hero header**
- **Estate status cards**
- **Control actions**
- **Library health overview**
- **Filter bar**
- **Library estate table**
- **Selected library detail panel**
- **Highest leverage fixes**
- **Blockers and risks**
- **Governance rules**
- **Maintenance modes**
- **Dependency map summary**
- **Recent activity**

What you will likely need to adjust

The main things you'll probably swap immediately are:

- `GlassPanel`, `MetricCard`, `StatusChip` import paths
- icon choices
- your app's table styling conventions
- route navigation when a library row is clicked

- exact tone props for `StatusChip`
- date formatting for `event.timestamp`

component extraction

Component extraction outline

1. Target file structure

App/src/pages/library-control.tsx

App/src/components/library-control/
 library-control-header.tsx
 estate-status-grid.tsx
 control-actions-strip.tsx
 library-health-overview.tsx
 library-estate-filters.tsx
 library-estate-table.tsx
 selected-library-detail.tsx
 leverage-fixes-panel.tsx
 blockers-risks-panel.tsx
 governance-rules-panel.tsx
 maintenance-modes-panel.tsx
 dependency-map-panel.tsx
 recent-library-activity-panel.tsx
 shared.tsx

---

2. Page responsibility after extraction

`library-control.tsx` should only:

- call `useLibraryControl()`
- assemble sections in order
- pass props down
- keep no extra UI logic beyond layout

That page should become a clean orchestration file.

---

3. Main page skeleton after extraction

```
// App/src/pages/library-control.tsx

import { useLibraryControl } from "../hooks/use-library-control";

import { LibraryControlHeader } from "../components/library-control/library-control-header";
import { EstateStatusGrid } from "../components/library-control/estate-status-grid";
import { ControlActionsStrip } from "../components/library-control/control-actions-strip";
import { LibraryHealthOverview } from "../components/library-control/library-health-overview";
import { LibraryEstateFilters } from "../components/library-control/library-estate-filters";
import { LibraryEstateTable } from "../components/library-control/library-estate-table";
import { SelectedLibraryDetail } from "../components/library-control/selected-library-detail";
import { LeverageFixesPanel } from "../components/library-control/leverage-fixes-panel";
import { BlockersRisksPanel } from "../components/library-control/blockers-risks-panel";
import { GovernanceRulesPanel } from "../components/library-control/governance-rules-panel";
import { MaintenanceModesPanel } from
"../components/library-control/maintenance-modes-panel";
import { DependencyMapPanel } from "../components/library-control/dependency-map-panel";
import { RecentLibraryActivityPanel } from
"../components/library-control/recent-library-activity-panel";
import { GlassPanel } from "../components/glass-panel";

export default function LibraryControlCenterPage() {
 const control = useLibraryControl();

 return (
   <div className="space-y-6 p-6">
     <LibraryControlHeader header={control.header} />

     <EstateStatusGrid
       metrics={control.metrics}
       visibleCount={control.visibleCount}
       totalCount={control.totalCount}
     />

     <ControlActionsStrip
       actions={control.actions}
       onRunAction={control.runControlAction}
     />

     <LibraryHealthOverview healthBands={control.healthBands} />

     <GlassPanel className="rounded-3xl p-5">
```

```
      <LibraryEstateFilters
        filters={control.filters}
        hasActiveFilters={control.hasActiveFilters}
        onQueryChange={control.setQuery}
        onAuthorityFilterChange={control.setAuthorityFilter}
        onControlStateFilterChange={control.setControlStateFilter}
        onRiskFilterChange={control.setRiskFilter}
        onSortChange={control.setSortBy}
        onToggleStaleOnly={control.toggleStaleOnly}
        onToggleBlockedOnly={control.toggleBlockedOnly}
        onToggleMissingAuthorityOnly={control.toggleMissingAuthorityOnly}
        onToggleDriftOnly={control.toggleDriftOnly}
        onClearFilters={control.clearFilters}
      />

      <LibraryEstateTable
        libraries={control.filteredLibraries}
        onSelectLibrary={control.selectLibrary}
        getLibraryControlScore={control.getLibraryControlScore}
      />
    </GlassPanel>

    <SelectedLibraryDetail
      library={control.selectedLibrary}
      onAssignMaintenanceMode={control.assignMaintenanceMode}
    />

    <div className="grid gap-6 xl:grid-cols-12">
      <div className="space-y-6 xl:col-span-7">
        <LeverageFixesPanel fixes={control.leverageFixes} />
        <BlockersRisksPanel blockers={control.blockers} risks={control.risks} />
        <GovernanceRulesPanel rules={control.governanceRules} />
      </div>

      <div className="space-y-6 xl:col-span-5">
        <MaintenanceModesPanel modes={control.maintenanceModes} />
        <DependencyMapPanel links={control.dependencyLinks} />
        <RecentLibraryActivityPanel events={control.recentActivity} />
      </div>
    </div>
  </div>
);
}
```

4. Component-by-component contract

A. `library-control-header.tsx`

Job

Render hero/header only.

Props

```
import { EstateHeaderStats } from "../../data/library-control";

type LibraryControlHeaderProps = {
 header: EstateHeaderStats;
};
```

Contents

- icon
- title
- subtitle
- 4 summary tiles

Keep inside component

- icon choice
- hero layout
- stat tile markup

Do not keep inside component

- filters
- page actions

- table behavior

B. `estate-status-grid.tsx`

Job

Render top status cards.

Props

```
import { EstateStatusMetrics } from "../../data/library-control";

type EstateStatusGridProps = {
 metrics: EstateStatusMetrics;
 visibleCount: number;
 totalCount: number;
};
```

Contents

- section title
- visible/total count
- 8 status cards

C. `control-actions-strip.tsx`

Job

Render action buttons for control actions.

Props

```
import { ControlAction } from "../../data/library-control";

type ControlActionsStripProps = {
 actions: ControlAction[];
```

```
 onRunAction: (actionType: ControlAction["actionType"]) => void;
};
```

Contents

- action cards/buttons
- icon mapping
- click handling

Keep in component

- button rendering
- icon resolver for action labels

D. `library-health-overview.tsx`

Job

Render health band cards.

Props

```
import { LibraryHealthBand } from "../../data/library-control";

type LibraryHealthOverviewProps = {
 healthBands: LibraryHealthBand[];
};
```

Contents

- section title
- one card per band
- count

- recommended action
- library ID pills

E. `library-estate-filters.tsx`

Job

Render the whole filter/search/sort UI.

Props

```
import {
 AuthorityStatus,
 ControlState,
 LibraryControlFilters,
 RiskLevel,
} from "../../data/library-control";

type LibraryEstateFiltersProps = {
 filters: LibraryControlFilters;
 hasActiveFilters: boolean;
 onQueryChange: (query: string) => void;
 onAuthorityFilterChange: (values?: AuthorityStatus[]) => void;
 onControlStateFilterChange: (values?: ControlState[]) => void;
 onRiskFilterChange: (values?: RiskLevel[]) => void;
 onSortChange: (sortBy?: LibraryControlFilters["sortBy"]) => void;
 onToggleStaleOnly: () => void;
 onToggleBlockedOnly: () => void;
 onToggleMissingAuthorityOnly: () => void;
 onToggleDriftOnly: () => void;
 onClearFilters: () => void;
};
```

Contents

- heading/subheading
- search input

- 4 selects
- quick toggle pills
- clear filters button

Reason to isolate

This is one of the easiest areas to bloat.

F. `library-estate-table.tsx`

Job

Render the main library table.

Props

```
import { LibraryControlRecord } from "../../data/library-control";

type LibraryEstateTableProps = {
 libraries: LibraryControlRecord[];
 onSelectLibrary: (libraryId: string) => void;
 getLibraryControlScore: (library: LibraryControlRecord) => number;
};
```

Contents

- table head
- rows
- status chips
- empty state

Keep in component

- row click

- formatting
- tone mapping helpers

Optional split later

If this gets larger, split table row into:

- `library-estate-row.tsx`

G. `selected-library-detail.tsx`

Job

Render the currently selected library's full detail state.

Props

```
import { LibraryDetail } from "../../data/library-control";

type SelectedLibraryDetailProps = {
 library: LibraryDetail | null;
 onAssignMaintenanceMode: (libraryId: string, modeId: string) => void;
};
```

Contents

- empty state if no library selected
- identity card
- governance card
- recommended action card
- health metric tiles
- list cards for:

    - missing artifacts
    - stale artifacts
    - broken refs
    - unresolved conflicts
    - upstream dependencies
    - downstream dependents

Good internal helper

- `DetailListCard`

H. `leverage-fixes-panel.tsx`

Job

Render ranked leverage fixes.

Props

```
import { LeverageFix } from "../../data/library-control";

type LeverageFixesPanelProps = {
 fixes: LeverageFix[];
};
```

Contents

- title
- list of ranked fix cards
- payoff score
- impact radius

- effort
- action text

I. `blockers-risks-panel.tsx`

Job

Render blockers and system risks side by side.

Props

```
import { ActiveBlocker, SystemRisk } from "../../data/library-control";

type BlockersRisksPanelProps = {
 blockers: ActiveBlocker[];
 risks: SystemRisk[];
};
```

Contents

- left column: blockers
- right column: risks
- severity chips

J. `governance-rules-panel.tsx`

Job

Render governance rules.

Props

```
import { GovernanceRule } from "../../data/library-control";

type GovernanceRulesPanelProps = {
```

```
 rules: GovernanceRule[];
};
```

Contents

- rule cards
- title
- severity chip
- description

K. `maintenance-modes-panel.tsx`

Job

Render maintenance mode cards.

Props

```
import { MaintenanceModeCard } from "../../data/library-control";

type MaintenanceModesPanelProps = {
 modes: MaintenanceModeCard[];
};
```

Contents

- list/grid of modes
- mode ID + name
- purpose
- trigger condition
- launchable chip

L. `dependency-map-panel.tsx`

Job

Render dependency summary and a lightweight link list.

Props

```
import { DependencyLink } from "../../data/library-control";

type DependencyMapPanelProps = {
 links: DependencyLink[];
};
```

Contents

- summary mini stats
- top few links
- relation/risk chips

Internal helper

- `MiniStat`

M. `recent-library-activity-panel.tsx`

Job

Render recent events timeline/list.

Props

```
import { LibraryActivityEvent } from "../../data/library-control";

type RecentLibraryActivityPanelProps = {
 events: LibraryActivityEvent[];
};
```

Contents

- title
- event cards
- timestamp
- summary
- event type chip
- library ID chip

N. `shared.tsx`

Job

Hold tiny shared helpers used by multiple library-control components.

Good candidates

- `getRiskTone`
- `getControlStateTone`
- `getAuthorityTone`
- `formatPercent`
- `MiniStat`
- `DetailListCard` if reused
- maybe `SectionHeader`

Example

```
export type BadgeTone = "default" | "success" | "warning" | "danger" | "info";

export function getRiskTone(risk: string): BadgeTone {
 switch (risk) {
   case "critical":
     return "danger";
   case "high":
     return "warning";
   case "moderate":
     return "info";
   case "low":
   default:
     return "success";
 }
}
```

---

5. Extraction order

Best order:

Pass 1

- `library-control-header.tsx`
- `estate-status-grid.tsx`
- `control-actions-strip.tsx`

Pass 2

- `library-estate-filters.tsx`
- `library-estate-table.tsx`
- `selected-library-detail.tsx`

Pass 3

- `leverage-fixes-panel.tsx`
- `blockers-risks-panel.tsx`
- `governance-rules-panel.tsx`

Pass 4

- `maintenance-modes-panel.tsx`
- `dependency-map-panel.tsx`
- `recent-library-activity-panel.tsx`
- `shared.tsx`

That keeps the most important UI extracted first.

---

6. Rules for the extraction

Use these rules so the page stays clean:

- page file should not contain tone helpers
- page file should not contain list subcomponents
- section-specific formatting stays in the section component
- shared helpers only go in `shared.tsx` if at least 2 components use them
- avoid over-splitting on day one

---

7. Best stopping point

A good maintainable endpoint is:

- one page file
- 10–12 section components
- one shared helper file

That is enough structure without turning this into component sprawl.

8. What comes after this

The next clean step is the **first extraction batch code**, starting with:

- `library-control-header.tsx`
- `estate-status-grid.tsx`
- `control-actions-strip.tsx`
- `shared.tsx`

extraction batch 1

Here is **extraction batch 1**.

This batch gives you:

- `shared.tsx`
- `library-control-header.tsx`
- `estate-status-grid.tsx`
- `control-actions-strip.tsx`

It keeps the page cleaner without over-splitting, and the action strip remains aligned with your locked maintenance/control model direction.

Response: Axion Updates Planning

App/src/components/library-control/shared.tsx

```
import { ReactNode } from "react";

export type BadgeTone = "default" | "success" | "warning" | "danger" | "info";

export function getRiskTone(risk: string): BadgeTone {
 switch (risk) {
   case "critical":
     return "danger";
   case "high":
     return "warning";
   case "moderate":
     return "info";
   case "low":
   default:
     return "success";
 }
}

export function getControlStateTone(state: string): BadgeTone {
 switch (state) {
   case "unsafe":
     return "danger";
   case "blocked":
     return "warning";
   case "degraded":
```

```
      return "warning";
    case "review-required":
      return "info";
    case "recovery":
      return "info";
    case "governed":
    default:
      return "success";
  }
}

export function getAuthorityTone(status: string): BadgeTone {
  switch (status) {
    case "authoritative":
      return "success";
    case "partial":
      return "info";
    case "disputed":
      return "warning";
    case "missing":
    default:
      return "danger";
  }
}

export function formatPercent(value: number) {
  return `${Math.round(value)}%`;
}

export function MiniStat({
  label,
  value,
}: {
  label: string;
  value: number;
}) {
  return (
    <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
      <div className="text-xs uppercase tracking-wide text-white/50">{label}</div>
      <div className="mt-2 text-2xl font-semibold">{value}</div>
    </div>
  );
}
```

```
export function SectionTitle({
 title,
 subtitle,
 right,
}: {
 title: string;
 subtitle?: string;
 right?: ReactNode;
}) {
 return (
   <div className="flex flex-col gap-3 md:flex-row md:items-center md:justify-between">
     <div>
       <h2 className="text-lg font-semibold">{title}</h2>
       {subtitle ? <p className="text-sm text-white/50">{subtitle}</p> : null}
     </div>
     {right ? <div>{right}</div> : null}
   </div>
 );
}
```

App/src/components/library-control/library-control-header.tsx

```
import { Library } from "lucide-react";
import type { EstateHeaderStats } from "../../data/library-control";
import { GlassPanel } from "../glass-panel";

type LibraryControlHeaderProps = {
 header: EstateHeaderStats;
};

export function LibraryControlHeader({
 header,
}: LibraryControlHeaderProps) {
 return (
   <GlassPanel className="rounded-3xl p-6">
     <div className="flex flex-col gap-6 lg:flex-row lg:items-start lg:justify-between">
       <div className="space-y-3">
         <div className="flex items-center gap-3">
           <div className="rounded-2xl border border-white/10 bg-white/5 p-3">
             <Library className="h-6 w-6" />
           </div>
```

```
          <div>
            <div className="text-xs uppercase tracking-[0.25em] text-white/50">
              Control Authority
            </div>
            <h1 className="text-3xl font-semibold tracking-tight">
              Library Control Center
            </h1>
          </div>
        </div>

        <p className="max-w-3xl text-sm text-white/70">
          Central authority for governing, inspecting, maintaining, and directing
          the Axion library system.
        </p>
      </div>

      <div className="grid min-w-[320px] grid-cols-2 gap-3">
        <HeaderStat label="Control Score" value={header.controlScore} />
        <HeaderStat label="Governed Libraries" value={header.governedLibraries} />
        <HeaderStat label="Active Risks" value={header.activeRisks} />
        <HeaderStat
          label="Blocked Dependencies"
          value={header.blockedDependencies}
        />
      </div>
    </div>
  </GlassPanel>
 );
}

function HeaderStat({
 label,
 value,
}: {
 label: string;
 value: number;
}) {
 return (
   <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
     <div className="text-xs uppercase tracking-wide text-white/50">{label}</div>
     <div className="mt-2 text-2xl font-semibold">{value}</div>
   </div>
 );
```

```
}
```

App/src/components/library-control/estate-status-grid.tsx

```
import {
 AlertTriangle,
 BookOpen,
 RefreshCcw,
 Shield,
 ShieldAlert,
 ShieldCheck,
 Siren,
 TriangleAlert,
} from "lucide-react";

import type { EstateStatusMetrics } from "../../data/library-control";
import { MetricCard } from "../metric-card";
import { SectionTitle } from "./shared";

type EstateStatusGridProps = {
 metrics: EstateStatusMetrics;
 visibleCount: number;
 totalCount: number;
};

export function EstateStatusGrid({
 metrics,
 visibleCount,
 totalCount,
}: EstateStatusGridProps) {
 return (
   <section className="space-y-3">
     <SectionTitle
       title="Estate Status"
       right={
         <div className="text-sm text-white/50">
           {visibleCount} visible / {totalCount} total
         </div>
       }
     />
```

```
    <div className="grid gap-4 sm:grid-cols-2 xl:grid-cols-4">
      <MetricCard title="Total Libraries" value={metrics.totalLibraries} icon={BookOpen} />
      <MetricCard title="Governed" value={metrics.governed} icon={ShieldCheck} />
      <MetricCard title="Review Required" value={metrics.reviewRequired} icon={Shield} />
      <MetricCard title="Stale" value={metrics.stale} icon={RefreshCcw} />
      <MetricCard title="Unsafe" value={metrics.unsafe} icon={TriangleAlert} />
      <MetricCard title="Blocked" value={metrics.blocked} icon={AlertTriangle} />
      <MetricCard title="Missing Authority" value={metrics.missingAuthority} icon={ShieldAlert} />
      <MetricCard title="Drift Alerts" value={metrics.driftAlerts} icon={Siren} />
    </div>
  </section>
 );
}
```

App/src/components/library-control/control-actions-strip.tsx

```
import {
 ArrowRight,
 Database,
 HeartPulse,
 RefreshCcw,
 Search,
 ShieldAlert,
 TriangleAlert,
 Wrench,
 GitBranch,
} from "lucide-react";

import type { ControlAction } from "../../data/library-control";
import { GlassPanel } from "../glass-panel";

type ControlActionsStripProps = {
 actions: ControlAction[];
 onRunAction: (actionType: ControlAction["actionType"]) => void;
};

export function ControlActionsStrip({
 actions,
 onRunAction,
}: ControlActionsStripProps) {
 return (
```

```
  <GlassPanel className="rounded-3xl p-5">
    <div className="mb-4 flex items-center gap-2">
      <Wrench className="h-5 w-5" />
      <h2 className="text-lg font-semibold">Control Actions</h2>
    </div>

    <div className="grid gap-3 sm:grid-cols-2 xl:grid-cols-4">
      {actions.map((action) => {
        const Icon = getActionIcon(action.actionType);

        return (
          <button
            key={action.id}
            type="button"
            onClick={() => onRunAction(action.actionType)}
            disabled={!action.enabled}
            className="flex items-center justify-between rounded-2xl border border-white/10
bg-white/5 p-4 text-left transition hover:bg-white/10 disabled:cursor-not-allowed
disabled:opacity-50"
          >
            <div className="flex items-start gap-3">
              <div className="rounded-xl border border-white/10 bg-white/5 p-2">
                <Icon className="h-4 w-4" />
              </div>

              <div>
                <div className="font-medium">{action.label}</div>
                <div className="text-sm text-white/50">
                  Launch control workflow
                </div>
              </div>
            </div>

            <ArrowRight className="h-4 w-4 text-white/40" />
          </button>
        );
      })}
    </div>
  </GlassPanel>
 );
}

function getActionIcon(actionType: ControlAction["actionType"]) {
 switch (actionType) {
```

```
    case "health-check":
      return HeartPulse;
    case "coverage-audit":
      return Search;
    case "drift-detection":
      return RefreshCcw;
    case "rebuild-registries":
      return Database;
    case "review-authority-gaps":
      return ShieldAlert;
    case "dependency-refresh":
      return GitBranch;
    case "open-unsafe":
      return TriangleAlert;
    case "resolve-blockers":
    default:
      return Wrench;
  }
}
```

Updated `App/src/pages/library-control.tsx` for batch 1

This is the reduced page after pulling out the first three sections.

```
import { useMemo } from "react";
import {
  AlertTriangle,
  ArrowRight,
  Bot,
  Database,
  Filter,
  GitBranch,
  HeartPulse,
  Library,
  RefreshCcw,
  Search,
  Shield,
  ShieldAlert,
  ShieldCheck,
  Siren,
  TriangleAlert,
```

```
 Wrench,
} from "lucide-react";

import { useLibraryControl } from "../hooks/use-library-control";

import { GlassPanel } from "../components/glass-panel";
import { MetricCard } from "../components/metric-card";
import { StatusChip } from "../components/status-chip";

import { LibraryControlHeader } from "../components/library-control/library-control-header";
import { EstateStatusGrid } from "../components/library-control/estate-status-grid";
import { ControlActionsStrip } from "../components/library-control/control-actions-strip";
import { MiniStat } from "../components/library-control/shared";

// keep the rest of your existing helpers/components here for now
type BadgeTone = "default" | "success" | "warning" | "danger" | "info";

function getRiskTone(risk: string): BadgeTone {
 switch (risk) {
   case "critical":
     return "danger";
   case "high":
     return "warning";
   case "moderate":
     return "info";
   case "low":
   default:
     return "success";
 }
}

function getControlStateTone(state: string): BadgeTone {
 switch (state) {
   case "unsafe":
     return "danger";
   case "blocked":
     return "warning";
   case "degraded":
     return "warning";
   case "review-required":
     return "info";
   case "recovery":
     return "info";
   case "governed":
```

```
    default:
      return "success";
  }
}

function getAuthorityTone(status: string): BadgeTone {
  switch (status) {
    case "authoritative":
      return "success";
    case "partial":
      return "info";
    case "disputed":
      return "warning";
    case "missing":
    default:
      return "danger";
  }
}

function formatPercent(value: number) {
  return `${Math.round(value)}%`;
}

export default function LibraryControlCenterPage() {
  const {
    header,
    metrics,
    actions,
    healthBands,
    filteredLibraries,
    selectedLibrary,
    leverageFixes,
    blockers,
    risks,
    governanceRules,
    maintenanceModes,
    dependencyLinks,
    recentActivity,

    filters,
    setQuery,
    setAuthorityFilter,
    setControlStateFilter,
    setRiskFilter,
```

```
  setSortBy,

  toggleStaleOnly,
  toggleBlockedOnly,
  toggleMissingAuthorityOnly,
  toggleDriftOnly,

  clearFilters,
  selectLibrary,
  runControlAction,
  assignMaintenanceMode,

  hasActiveFilters,
  visibleCount,
  totalCount,
  getLibraryControlScore,
} = useLibraryControl();

const dependencySummary = useMemo(() => {
  const governedLinks = dependencyLinks.filter((l) => l.relation === "governs").length;
  const blockedLinks = dependencyLinks.filter((l) => l.relation === "blocks").length;
  const supportLinks = dependencyLinks.filter((l) => l.relation === "supports").length;

  return {
    total: dependencyLinks.length,
    governedLinks,
    blockedLinks,
    supportLinks,
  };
}, [dependencyLinks]);

return (
  <div className="space-y-6 p-6">
    <LibraryControlHeader header={header} />

    <EstateStatusGrid
      metrics={metrics}
      visibleCount={visibleCount}
      totalCount={totalCount}
    />

    <ControlActionsStrip
      actions={actions}
      onRunAction={runControlAction}
```

```
      />

      {/* keep remaining sections in page for now */}
      {/* ...rest of page unchanged from previous skeleton... */}
    </div>
  );
}

function DetailListCard({
  title,
  items,
}: {
  title: string;
  items: string[];
}) {
  return (
    <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
      <div className="text-sm font-medium">{title}</div>
      <div className="mt-3">
        {items.length === 0 ? (
          <div className="text-sm text-white/45">None</div>
        ) : (
          <ul className="space-y-2 text-sm text-white/70">
            {items.map((item) => (
              <li key={item}>• {item}</li>
            ))}
          </ul>
        )}
      </div>
    </div>
  );
}
```

What changed in this batch

The page now hands off:

- hero/header
- estate status grid
- control actions strip

extraction batch 2

Here is **extraction batch 2**.

This batch pulls out the main middle work surface:

- `library-health-overview.tsx`
- `library-estate-filters.tsx`
- `library-estate-table.tsx`

That removes the heaviest table/filter UI from the page file and gets the control page much closer to a clean orchestration shell.

App/src/components/library-control/library-health-overview.tsx

```
import type { LibraryHealthBand } from "../../data/library-control";
import { GlassPanel } from "../glass-panel";
import { SectionTitle } from "./shared";

type LibraryHealthOverviewProps = {
 healthBands: LibraryHealthBand[];
};

export function LibraryHealthOverview({
 healthBands,
}: LibraryHealthOverviewProps) {
 return (
   <section className="space-y-3">
     <SectionTitle title="Library Health Overview" />

     <div className="grid gap-4 md:grid-cols-2 xl:grid-cols-3">
       {healthBands.map((band) => (
         <GlassPanel key={band.id} className="rounded-3xl p-5">
           <div className="flex items-start justify-between gap-4">
             <div>
               <div className="text-sm uppercase tracking-wide text-white/50">
                 {band.label}
               </div>
```

```
            <div className="mt-1 text-2xl font-semibold">{band.count}</div>
          </div>

          <div className="rounded-full border border-white/10 bg-white/5 px-3 py-1 text-xs
text-white/70">
            {band.id}
          </div>
        </div>

        <div className="mt-4 space-y-2">
          <div className="text-sm text-white/60">Recommended action</div>
          <p className="text-sm text-white/80">{band.recommendedAction}</p>
        </div>

        <div className="mt-4 flex flex-wrap gap-2">
          {band.libraryIds.map((id) => (
            <span
              key={id}
              className="rounded-full border border-white/10 bg-white/5 px-3 py-1 text-xs
text-white/70"
            >
              {id}
            </span>
          ))}
        </div>
      </GlassPanel>
    ))}
  </div>
 </section>
);
}
```

App/src/components/library-control/library-estate-filters.tsx

```
import { Filter, Search } from "lucide-react";
import type {
 AuthorityStatus,
 ControlState,
 LibraryControlFilters,
 RiskLevel,
} from "../../data/library-control";
```

```
import { SectionTitle } from "./shared";

type LibraryEstateFiltersProps = {
 filters: LibraryControlFilters;
 hasActiveFilters: boolean;
 onQueryChange: (query: string) => void;
 onAuthorityFilterChange: (values?: AuthorityStatus[]) => void;
 onControlStateFilterChange: (values?: ControlState[]) => void;
 onRiskFilterChange: (values?: RiskLevel[]) => void;
 onSortChange: (sortBy?: LibraryControlFilters["sortBy"]) => void;
 onToggleStaleOnly: () => void;
 onToggleBlockedOnly: () => void;
 onToggleMissingAuthorityOnly: () => void;
 onToggleDriftOnly: () => void;
 onClearFilters: () => void;
};

export function LibraryEstateFilters({
 filters,
 hasActiveFilters,
 onQueryChange,
 onAuthorityFilterChange,
 onControlStateFilterChange,
 onRiskFilterChange,
 onSortChange,
 onToggleStaleOnly,
 onToggleBlockedOnly,
 onToggleMissingAuthorityOnly,
 onToggleDriftOnly,
 onClearFilters,
}: LibraryEstateFiltersProps) {
 return (
   <>
     <div className="mb-5">
       <SectionTitle
         title="Library Estate"
         subtitle="Inspect authority, health, risk, and recommended actions across all libraries."
         right={
           hasActiveFilters ? (
             <button
               type="button"
               onClick={onClearFilters}
               className="rounded-xl border border-white/10 bg-white/5 px-4 py-2 text-sm transition
hover:bg-white/10"
```

```
          >
            Clear Filters
          </button>
        ) : null
      }
    />
  </div>

  <div className="mb-5 grid gap-3 xl:grid-cols-12">
    <div className="xl:col-span-4">
      <label className="mb-2 block text-xs uppercase tracking-wide text-white/50">
        Search
      </label>
      <div className="flex items-center gap-2 rounded-2xl border border-white/10 bg-white/5
px-3 py-2">
        <Search className="h-4 w-4 text-white/40" />
        <input
          value={filters.query}
          onChange={(e) => onQueryChange(e.target.value)}
          placeholder="Search libraries, owners, or actions"
          className="w-full bg-transparent text-sm outline-none placeholder:text-white/30"
        />
      </div>
    </div>

    <div className="xl:col-span-2">
      <label className="mb-2 block text-xs uppercase tracking-wide text-white/50">
        Authority
      </label>
      <select
        className="w-full rounded-2xl border border-white/10 bg-white/5 px-3 py-2 text-sm
outline-none"
        onChange={(e) =>
          onAuthorityFilterChange(
            e.target.value ? [e.target.value as AuthorityStatus] : undefined
          )
        }
        value={filters.authorityStatus?.[0] ?? ""}
      >
        <option value="">All</option>
        <option value="authoritative">Authoritative</option>
        <option value="partial">Partial</option>
        <option value="missing">Missing</option>
        <option value="disputed">Disputed</option>
```

```
        </select>
      </div>

      <div className="xl:col-span-2">
        <label className="mb-2 block text-xs uppercase tracking-wide text-white/50">
          Control State
        </label>
        <select
          className="w-full rounded-2xl border border-white/10 bg-white/5 px-3 py-2 text-sm
outline-none"
          onChange={(e) =>
            onControlStateFilterChange(
              e.target.value ? [e.target.value as ControlState] : undefined
            )
          }
          value={filters.controlState?.[0] ?? ""}
        >
          <option value="">All</option>
          <option value="governed">Governed</option>
          <option value="review-required">Review Required</option>
          <option value="degraded">Degraded</option>
          <option value="blocked">Blocked</option>
          <option value="unsafe">Unsafe</option>
          <option value="recovery">Recovery</option>
        </select>
      </div>

      <div className="xl:col-span-2">
        <label className="mb-2 block text-xs uppercase tracking-wide text-white/50">
          Risk
        </label>
        <select
          className="w-full rounded-2xl border border-white/10 bg-white/5 px-3 py-2 text-sm
outline-none"
          onChange={(e) =>
            onRiskFilterChange(
              e.target.value ? [e.target.value as RiskLevel] : undefined
            )
          }
          value={filters.risk?.[0] ?? ""}
        >
          <option value="">All</option>
          <option value="critical">Critical</option>
          <option value="high">High</option>
```

```
          <option value="moderate">Moderate</option>
          <option value="low">Low</option>
        </select>
      </div>

      <div className="xl:col-span-2">
        <label className="mb-2 block text-xs uppercase tracking-wide text-white/50">
          Sort
        </label>
        <select
          className="w-full rounded-2xl border border-white/10 bg-white/5 px-3 py-2 text-sm
outline-none"
          onChange={(e) =>
            onSortChange(
              (e.target.value || undefined) as LibraryControlFilters["sortBy"]
            )
          }
          value={filters.sortBy ?? ""}
        >
          <option value="risk">Risk</option>
          <option value="dependency-pressure">Dependency Pressure</option>
          <option value="freshness">Freshness</option>
          <option value="coverage">Coverage</option>
          <option value="last-audit">Last Audit</option>
          <option value="control-score">Control Score</option>
        </select>
      </div>
    </div>

    <div className="mb-5 flex flex-wrap items-center gap-2">
      <div className="mr-2 flex items-center gap-2 text-xs uppercase tracking-wide
text-white/50">
        <Filter className="h-3.5 w-3.5" />
        Quick Filters
      </div>

      <FilterPill
        active={Boolean(filters.staleOnly)}
        label="Stale Only"
        onClick={onToggleStaleOnly}
      />

      <FilterPill
        active={Boolean(filters.blockedOnly)}
```

```
        label="Blocked Only"
        onClick={onToggleBlockedOnly}
      />

      <FilterPill
        active={Boolean(filters.missingAuthorityOnly)}
        label="Missing Authority"
        onClick={onToggleMissingAuthorityOnly}
      />

      <FilterPill
        active={Boolean(filters.driftOnly)}
        label="Drift Only"
        onClick={onToggleDriftOnly}
      />
    </div>
  </>
 );
}

function FilterPill({
 active,
 label,
 onClick,
}: {
 active: boolean;
 label: string;
 onClick: () => void;
}) {
 return (
   <button
     type="button"
     onClick={onClick}
     className={`rounded-full px-3 py-1.5 text-xs transition ${
       active ? "bg-white text-black" : "border border-white/10 bg-white/5"
     }`}
   >
     {label}
   </button>
 );
}
```

App/src/components/library-control/library-estate-table.tsx

```
import type { LibraryControlRecord } from "../../data/library-control";
import { StatusChip } from "../status-chip";
import {
 formatPercent,
 getAuthorityTone,
 getControlStateTone,
 getRiskTone,
} from "./shared";

type LibraryEstateTableProps = {
 libraries: LibraryControlRecord[];
 onSelectLibrary: (libraryId: string) => void;
 getLibraryControlScore: (library: LibraryControlRecord) => number;
};

export function LibraryEstateTable({
 libraries,
 onSelectLibrary,
 getLibraryControlScore,
}: LibraryEstateTableProps) {
 return (
   <div className="overflow-x-auto">
     <table className="min-w-full border-separate border-spacing-y-2">
       <thead>
         <tr className="text-left text-xs uppercase tracking-wide text-white/45">
           <th className="px-3 py-2">Library</th>
           <th className="px-3 py-2">Authority</th>
           <th className="px-3 py-2">Coverage</th>
           <th className="px-3 py-2">Freshness</th>
           <th className="px-3 py-2">Integrity</th>
           <th className="px-3 py-2">Dependency Pressure</th>
           <th className="px-3 py-2">Risk</th>
           <th className="px-3 py-2">Control State</th>
           <th className="px-3 py-2">Score</th>
           <th className="px-3 py-2">Recommended Action</th>
         </tr>
       </thead>

       <tbody>
         {libraries.map((library) => (
```

```
          <tr
            key={library.id}
            onClick={() => onSelectLibrary(library.id)}
            className="cursor-pointer rounded-2xl border border-white/10 bg-white/5 transition
hover:bg-white/10"
          >
            <td className="rounded-l-2xl px-3 py-4">
              <div className="font-medium">{library.name}</div>
              <div className="text-sm text-white/50">{library.shortName}</div>
            </td>

            <td className="px-3 py-4">
              <StatusChip
                label={library.authorityStatus}
                tone={getAuthorityTone(library.authorityStatus)}
              />
            </td>

            <td className="px-3 py-4 text-sm">
              {formatPercent(library.coveragePercent)}
            </td>

            <td className="px-3 py-4 text-sm">
              {formatPercent(library.freshnessPercent)}
            </td>

            <td className="px-3 py-4 text-sm">
              {formatPercent(library.integrityPercent)}
            </td>

            <td className="px-3 py-4 text-sm">
              {formatPercent(library.dependencyPressure)}
            </td>

            <td className="px-3 py-4">
              <StatusChip
                label={library.risk}
                tone={getRiskTone(library.risk)}
              />
            </td>

            <td className="px-3 py-4">
              <StatusChip
                label={library.controlState}
```

```
            tone={getControlStateTone(library.controlState)}
          />
        </td>

        <td className="px-3 py-4 text-sm">
          {getLibraryControlScore(library)}
        </td>

        <td className="rounded-r-2xl px-3 py-4">
          <div className="max-w-[260px] text-sm text-white/75">
            {library.recommendedAction}
          </div>
        </td>
      </tr>
    ))}
  </tbody>
</table>

{libraries.length === 0 && (
  <div className="rounded-2xl border border-dashed border-white/10 bg-white/5 p-8
text-center text-sm text-white/50">
    No libraries match the current filters.
  </div>
)}
</div>
);
}
```

Updated `App/src/pages/library-control.tsx` after batch 2

This is the page shell after wiring in the new extracted middle section.

```
import { useMemo } from "react";
import { Shield } from "lucide-react";

import { useLibraryControl } from "../hooks/use-library-control";

import { GlassPanel } from "../components/glass-panel";
import { StatusChip } from "../components/status-chip";

import { LibraryControlHeader } from "../components/library-control/library-control-header";
```

```
import { EstateStatusGrid } from "../components/library-control/estate-status-grid";
import { ControlActionsStrip } from "../components/library-control/control-actions-strip";
import { LibraryHealthOverview } from "../components/library-control/library-health-overview";
import { LibraryEstateFilters } from "../components/library-control/library-estate-filters";
import { LibraryEstateTable } from "../components/library-control/library-estate-table";
import {
 MiniStat,
 formatPercent,
 getAuthorityTone,
 getControlStateTone,
 getRiskTone,
} from "../components/library-control/shared";

export default function LibraryControlCenterPage() {
 const {
   header,
   metrics,
   actions,
   healthBands,
   filteredLibraries,
   selectedLibrary,
   leverageFixes,
   blockers,
   risks,
   governanceRules,
   maintenanceModes,
   dependencyLinks,
   recentActivity,

   filters,
   setQuery,
   setAuthorityFilter,
   setControlStateFilter,
   setRiskFilter,
   setSortBy,

   toggleStaleOnly,
   toggleBlockedOnly,
   toggleMissingAuthorityOnly,
   toggleDriftOnly,

   clearFilters,
   selectLibrary,
   runControlAction,
```

```
  assignMaintenanceMode,

  hasActiveFilters,
  visibleCount,
  totalCount,
  getLibraryControlScore,
} = useLibraryControl();

const dependencySummary = useMemo(() => {
  const governedLinks = dependencyLinks.filter((l) => l.relation === "governs").length;
  const blockedLinks = dependencyLinks.filter((l) => l.relation === "blocks").length;
  const supportLinks = dependencyLinks.filter((l) => l.relation === "supports").length;

  return {
    total: dependencyLinks.length,
    governedLinks,
    blockedLinks,
    supportLinks,
  };
}, [dependencyLinks]);

return (
  <div className="space-y-6 p-6">
    <LibraryControlHeader header={header} />

    <EstateStatusGrid
      metrics={metrics}
      visibleCount={visibleCount}
      totalCount={totalCount}
    />

    <ControlActionsStrip
      actions={actions}
      onRunAction={runControlAction}
    />

    <LibraryHealthOverview healthBands={healthBands} />

    <GlassPanel className="rounded-3xl p-5">
      <LibraryEstateFilters
        filters={filters}
        hasActiveFilters={hasActiveFilters}
        onQueryChange={setQuery}
        onAuthorityFilterChange={setAuthorityFilter}
```

```
        onControlStateFilterChange={setControlStateFilter}
        onRiskFilterChange={setRiskFilter}
        onSortChange={setSortBy}
        onToggleStaleOnly={toggleStaleOnly}
        onToggleBlockedOnly={toggleBlockedOnly}
        onToggleMissingAuthorityOnly={toggleMissingAuthorityOnly}
        onToggleDriftOnly={toggleDriftOnly}
        onClearFilters={clearFilters}
      />

      <LibraryEstateTable
        libraries={filteredLibraries}
        onSelectLibrary={selectLibrary}
        getLibraryControlScore={getLibraryControlScore}
      />
    </GlassPanel>

    {/* keep remaining sections in page for now */}
    <GlassPanel className="rounded-3xl p-5">
      <div className="mb-4 flex items-center gap-2">
        <Shield className="h-5 w-5" />
        <h2 className="text-lg font-semibold">Selected Library Detail</h2>
      </div>

      {!selectedLibrary ? (
        <div className="rounded-2xl border border-dashed border-white/10 bg-white/5 p-8
text-sm text-white/50">
          Select a library to inspect governance state, gaps, dependencies, and actions.
        </div>
      ) : (
        <div className="grid gap-5 xl:grid-cols-12">
          <div className="space-y-5 xl:col-span-4">
            <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
              <div className="text-xs uppercase tracking-wide text-white/50">Identity</div>
              <div className="mt-2 text-xl font-semibold">{selectedLibrary.name}</div>
              <div className="mt-1 text-sm text-white/50">{selectedLibrary.shortName}</div>
              <div className="mt-4 space-y-2 text-sm text-white/75">
                <div><span className="text-white/50">Owner:</span> {selectedLibrary.owner ??
"—"}</div>
                <div><span className="text-white/50">Role:</span> {selectedLibrary.estateRole ??
"—"}</div>
                <div><span className="text-white/50">Route:</span> {selectedLibrary.route ??
"—"}</div>
              </div>
```

```
          </div>

          <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
            <div className="text-xs uppercase tracking-wide text-white/50">Governance
State</div>
            <div className="mt-3 flex flex-wrap gap-2">
              <StatusChip
                label={selectedLibrary.authorityStatus}
                tone={getAuthorityTone(selectedLibrary.authorityStatus)}
              />
              <StatusChip
                label={selectedLibrary.trustDecision}
                tone={
                  selectedLibrary.trustDecision === "trusted"
                    ? "success"
                    : selectedLibrary.trustDecision === "conditional"
                    ? "info"
                    : "danger"
                }
              />
            </div>

            <div className="mt-4 text-sm text-white/75">
              <div className="mb-2 text-white/50">Authority Source</div>
              <div>{selectedLibrary.authoritySource ?? "No authoritative source assigned."}</div>
            </div>
          </div>

          <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
            <div className="text-xs uppercase tracking-wide text-white/50">Recommended
Action</div>
            <p className="mt-3 text-sm
text-white/80">{selectedLibrary.recommendedAction}</p>

            <div className="mt-4">
              <div className="mb-2 text-xs uppercase tracking-wide text-white/50">
                Suggested Maintenance Mode
              </div>
              <div className="flex items-center justify-between rounded-xl border border-white/10
bg-black/20 px-3 py-2">
                <span className="text-sm">{selectedLibrary.suggestedMaintenanceMode ??
"—"}</span>
                <button
                  type="button"
```

```
                  className="rounded-lg border border-white/10 bg-white/5 px-3 py-1.5 text-xs
hover:bg-white/10"
                  onClick={() =>
                    selectedLibrary.suggestedMaintenanceMode &&
                    assignMaintenanceMode(
                      selectedLibrary.id,
                      selectedLibrary.suggestedMaintenanceMode
                    )
                  }
                >
                  Assign
                </button>
              </div>
            </div>
          </div>
        </div>

        <div className="space-y-5 xl:col-span-8">
          <div className="grid gap-4 md:grid-cols-4">
            <MiniStat label="Coverage" value={selectedLibrary.coveragePercent} />
            <MiniStat label="Freshness" value={selectedLibrary.freshnessPercent} />
            <MiniStat label="Integrity" value={selectedLibrary.integrityPercent} />
            <MiniStat label="Dependency Fitness"
value={selectedLibrary.dependencyFitnessPercent} />
          </div>

          <div className="grid gap-4 md:grid-cols-2">
            <DetailListCard title="Missing Artifacts" items={selectedLibrary.missingArtifacts} />
            <DetailListCard title="Stale Artifacts" items={selectedLibrary.staleArtifacts} />
            <DetailListCard title="Broken References" items={selectedLibrary.brokenReferences}
/>
            <DetailListCard title="Unresolved Conflicts"
items={selectedLibrary.unresolvedConflicts} />
            <DetailListCard title="Upstream Dependencies"
items={selectedLibrary.upstreamDependencies} />
            <DetailListCard title="Downstream Dependents"
items={selectedLibrary.downstreamDependents} />
          </div>
        </div>
      </div>
    )}
  </GlassPanel>

  {/* rest of lower panels still inline for now */}
```

```
    </div>
  );
}

function DetailListCard({
  title,
  items,
}: {
  title: string;
  items: string[];
}) {
  return (
    <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
      <div className="text-sm font-medium">{title}</div>
      <div className="mt-3">
        {items.length === 0 ? (
          <div className="text-sm text-white/45">None</div>
        ) : (
          <ul className="space-y-2 text-sm text-white/70">
            {items.map((item) => (
              <li key={item}>• {item}</li>
            ))}
          </ul>
        )}
      </div>
    </div>
  );
}
```

One cleanup note

In that updated page example, `MiniStat` is still numeric-only, so it will show raw values like `88` instead of `88%`. For the final extracted detail panel, I'd switch those to dedicated metric tiles or use `formatPercent(...)` in the detail section.

What batch 2 removed from the page

The page now hands off:

- health overview
- filter bar

- table rendering

That is the largest chunk of UI complexity, so the file is already much more stable.

extraction batch 3

extraction batch 3.

This batch removes the largest remaining inspection and analysis sections from the page:

- `selected-library-detail.tsx`
- `leverage-fixes-panel.tsx`
- `blockers-risks-panel.tsx`

After this batch, the page becomes much closer to a true orchestration shell.

App/src/components/library-control/selected-library-detail.tsx

```
import { Shield } from "lucide-react";
import type { LibraryDetail } from "../../data/library-control";
import { GlassPanel } from "../glass-panel";
import { StatusChip } from "../status-chip";
import { formatPercent, getAuthorityTone, MiniStat } from "./shared";

type SelectedLibraryDetailProps = {
 library: LibraryDetail | null;
 onAssignMaintenanceMode: (libraryId: string, modeId: string) => void;
};

export function SelectedLibraryDetail({
 library,
 onAssignMaintenanceMode,
}: SelectedLibraryDetailProps) {
 return (
   <GlassPanel className="rounded-3xl p-5">
     <div className="mb-4 flex items-center gap-2">
       <Shield className="h-5 w-5" />
       <h2 className="text-lg font-semibold">Selected Library Detail</h2>
     </div>

     {!library ? (
```

```
      <div className="rounded-2xl border border-dashed border-white/10 bg-white/5 p-8
text-sm text-white/50">
        Select a library to inspect governance state, gaps, dependencies, and actions.
      </div>
    ) : (
      <div className="grid gap-5 xl:grid-cols-12">
        <div className="space-y-5 xl:col-span-4">
          <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
            <div className="text-xs uppercase tracking-wide text-white/50">Identity</div>
            <div className="mt-2 text-xl font-semibold">{library.name}</div>
            <div className="mt-1 text-sm text-white/50">{library.shortName}</div>

            <div className="mt-4 space-y-2 text-sm text-white/75">
              <div>
                <span className="text-white/50">Owner:</span> {library.owner ?? "—"}
              </div>
              <div>
                <span className="text-white/50">Role:</span> {library.estateRole ?? "—"}
              </div>
              <div>
                <span className="text-white/50">Route:</span> {library.route ?? "—"}
              </div>
            </div>
          </div>

          <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
            <div className="text-xs uppercase tracking-wide text-white/50">
              Governance State
            </div>

            <div className="mt-3 flex flex-wrap gap-2">
              <StatusChip
                label={library.authorityStatus}
                tone={getAuthorityTone(library.authorityStatus)}
              />
              <StatusChip
                label={library.trustDecision}
                tone={
                  library.trustDecision === "trusted"
                    ? "success"
                    : library.trustDecision === "conditional"
                    ? "info"
                    : "danger"
                }
```

```
            />
          </div>

          <div className="mt-4 text-sm text-white/75">
            <div className="mb-2 text-white/50">Authority Source</div>
            <div>{library.authoritySource ?? "No authoritative source assigned."}</div>
          </div>
        </div>

        <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
          <div className="text-xs uppercase tracking-wide text-white/50">
            Recommended Action
          </div>

          <p className="mt-3 text-sm text-white/80">{library.recommendedAction}</p>

          <div className="mt-4">
            <div className="mb-2 text-xs uppercase tracking-wide text-white/50">
              Suggested Maintenance Mode
            </div>

            <div className="flex items-center justify-between rounded-xl border border-white/10
bg-black/20 px-3 py-2">
              <span className="text-sm">{library.suggestedMaintenanceMode ?? "—"}</span>

              <button
                type="button"
                className="rounded-lg border border-white/10 bg-white/5 px-3 py-1.5 text-xs
hover:bg-white/10"
                onClick={() =>
                  library.suggestedMaintenanceMode &&
                  onAssignMaintenanceMode(library.id, library.suggestedMaintenanceMode)
                }
              >
                Assign
              </button>
            </div>
          </div>
        </div>
      </div>

      <div className="space-y-5 xl:col-span-8">
        <div className="grid gap-4 md:grid-cols-4">
          <MetricTile label="Coverage" value={formatPercent(library.coveragePercent)} />
```

```
            <MetricTile label="Freshness" value={formatPercent(library.freshnessPercent)} />
            <MetricTile label="Integrity" value={formatPercent(library.integrityPercent)} />
            <MetricTile
              label="Dependency Fitness"
              value={formatPercent(library.dependencyFitnessPercent)}
            />
          </div>

          <div className="grid gap-4 md:grid-cols-2">
            <DetailListCard title="Missing Artifacts" items={library.missingArtifacts} />
            <DetailListCard title="Stale Artifacts" items={library.staleArtifacts} />
            <DetailListCard title="Broken References" items={library.brokenReferences} />
            <DetailListCard title="Unresolved Conflicts" items={library.unresolvedConflicts} />
            <DetailListCard title="Upstream Dependencies" items={library.upstreamDependencies}
/>
            <DetailListCard title="Downstream Dependents" items={library.downstreamDependents}
/>
          </div>
        </div>
      </div>
    )}
  </GlassPanel>
 );
}

function MetricTile({
 label,
 value,
}: {
 label: string;
 value: string;
}) {
 return (
   <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
     <div className="text-xs uppercase tracking-wide text-white/50">{label}</div>
     <div className="mt-2 text-2xl font-semibold">{value}</div>
   </div>
 );
}

function DetailListCard({
 title,
 items,
}: {
```

```
 title: string;
 items: string[];
}) {
 return (
   <div className="rounded-2xl border border-white/10 bg-white/5 p-4">
     <div className="text-sm font-medium">{title}</div>
     <div className="mt-3">
       {items.length === 0 ? (
         <div className="text-sm text-white/45">None</div>
       ) : (
         <ul className="space-y-2 text-sm text-white/70">
           {items.map((item) => (
             <li key={item}>• {item}</li>
           ))}
         </ul>
       )}
     </div>
   </div>
 );
}
```

App/src/components/library-control/leverage-fixes-panel.tsx

```
import type { LeverageFix } from "../../data/library-control";
import { GlassPanel } from "../glass-panel";
import { StatusChip } from "../status-chip";

type LeverageFixesPanelProps = {
 fixes: LeverageFix[];
};

export function LeverageFixesPanel({
 fixes,
}: LeverageFixesPanelProps) {
 return (
   <GlassPanel className="rounded-3xl p-5">
     <h2 className="mb-4 text-lg font-semibold">Highest Leverage Fixes</h2>

     <div className="space-y-3">
       {fixes.map((fix) => (
         <div
```

```
          key={fix.id}
          className="rounded-2xl border border-white/10 bg-white/5 p-4"
        >
          <div className="flex items-start justify-between gap-4">
            <div>
              <div className="font-medium">{fix.title}</div>
              <div className="mt-1 text-sm text-white/60">{fix.summary}</div>
            </div>

            <div className="rounded-full border border-white/10 bg-black/20 px-3 py-1 text-xs
text-white/60">
              {fix.payoffScore}
            </div>
          </div>

          <div className="mt-4 flex flex-wrap gap-2">
            <StatusChip label={fix.impactRadius} tone="info" />
            <StatusChip label={fix.effort} tone="warning" />
          </div>

          <div className="mt-4 text-sm text-white/75">
            {fix.recommendedAction}
          </div>
        </div>
      ))}
    </div>
  </GlassPanel>
 );
}
```

App/src/components/library-control/blockers-risks-panel.tsx

```
import type { ActiveBlocker, SystemRisk } from "../../data/library-control";
import { GlassPanel } from "../glass-panel";
import { StatusChip } from "../status-chip";
import { getRiskTone } from "./shared";

type BlockersRisksPanelProps = {
 blockers: ActiveBlocker[];
 risks: SystemRisk[];
};
```

```
export function BlockersRisksPanel({
 blockers,
 risks,
}: BlockersRisksPanelProps) {
 return (
   <GlassPanel className="rounded-3xl p-5">
     <h2 className="mb-4 text-lg font-semibold">Blockers and Risks</h2>

     <div className="grid gap-4 md:grid-cols-2">
       <div className="space-y-3">
         <div className="text-sm font-medium text-white/80">Active Blockers</div>

         {blockers.map((blocker) => (
           <div
             key={blocker.id}
             className="rounded-2xl border border-white/10 bg-white/5 p-4"
           >
             <div className="flex items-center justify-between gap-3">
               <div className="font-medium">{blocker.title}</div>
               <StatusChip
                 label={blocker.severity}
                 tone={getRiskTone(blocker.severity)}
               />
             </div>

             <ul className="mt-3 space-y-2 text-sm text-white/70">
               {blocker.items.map((item) => (
                 <li key={item}>• {item}</li>
               ))}
             </ul>
           </div>
         ))}
       </div>

       <div className="space-y-3">
         <div className="text-sm font-medium text-white/80">System Risks</div>

         {risks.map((risk) => (
           <div
             key={risk.id}
             className="rounded-2xl border border-white/10 bg-white/5 p-4"
           >
             <div className="flex items-center justify-between gap-3">
```

```
            <div className="font-medium">{risk.title}</div>
            <StatusChip
              label={risk.severity}
              tone={getRiskTone(risk.severity)}
            />
          </div>

          <p className="mt-3 text-sm text-white/70">{risk.summary}</p>
        </div>
      ))}
    </div>
  </div>
 </GlassPanel>
);
}
```

Updated `App/src/pages/library-control.tsx` after batch 3

```
import { useMemo } from "react";

import { useLibraryControl } from "../hooks/use-library-control";

import { GlassPanel } from "../components/glass-panel";
import { StatusChip } from "../components/status-chip";

import { LibraryControlHeader } from "../components/library-control/library-control-header";
import { EstateStatusGrid } from "../components/library-control/estate-status-grid";
import { ControlActionsStrip } from "../components/library-control/control-actions-strip";
import { LibraryHealthOverview } from "../components/library-control/library-health-overview";
import { LibraryEstateFilters } from "../components/library-control/library-estate-filters";
import { LibraryEstateTable } from "../components/library-control/library-estate-table";
import { SelectedLibraryDetail } from "../components/library-control/selected-library-detail";
import { LeverageFixesPanel } from "../components/library-control/leverage-fixes-panel";
import { BlockersRisksPanel } from "../components/library-control/blockers-risks-panel";
import {
 MiniStat,
 getRiskTone,
} from "../components/library-control/shared";

export default function LibraryControlCenterPage() {
 const {
```

```
  header,
  metrics,
  actions,
  healthBands,
  filteredLibraries,
  selectedLibrary,
  leverageFixes,
  blockers,
  risks,
  governanceRules,
  maintenanceModes,
  dependencyLinks,
  recentActivity,

  filters,
  setQuery,
  setAuthorityFilter,
  setControlStateFilter,
  setRiskFilter,
  setSortBy,

  toggleStaleOnly,
  toggleBlockedOnly,
  toggleMissingAuthorityOnly,
  toggleDriftOnly,

  clearFilters,
  selectLibrary,
  runControlAction,
  assignMaintenanceMode,

  hasActiveFilters,
  visibleCount,
  totalCount,
  getLibraryControlScore,
} = useLibraryControl();

const dependencySummary = useMemo(() => {
  const governedLinks = dependencyLinks.filter((l) => l.relation === "governs").length;
  const blockedLinks = dependencyLinks.filter((l) => l.relation === "blocks").length;
  const supportLinks = dependencyLinks.filter((l) => l.relation === "supports").length;

  return {
    total: dependencyLinks.length,
```

```
    governedLinks,
    blockedLinks,
    supportLinks,
  };
}, [dependencyLinks]);

return (
  <div className="space-y-6 p-6">
    <LibraryControlHeader header={header} />

    <EstateStatusGrid
      metrics={metrics}
      visibleCount={visibleCount}
      totalCount={totalCount}
    />

    <ControlActionsStrip
      actions={actions}
      onRunAction={runControlAction}
    />

    <LibraryHealthOverview healthBands={healthBands} />

    <GlassPanel className="rounded-3xl p-5">
      <LibraryEstateFilters
        filters={filters}
        hasActiveFilters={hasActiveFilters}
        onQueryChange={setQuery}
        onAuthorityFilterChange={setAuthorityFilter}
        onControlStateFilterChange={setControlStateFilter}
        onRiskFilterChange={setRiskFilter}
        onSortChange={setSortBy}
        onToggleStaleOnly={toggleStaleOnly}
        onToggleBlockedOnly={toggleBlockedOnly}
        onToggleMissingAuthorityOnly={toggleMissingAuthorityOnly}
        onToggleDriftOnly={toggleDriftOnly}
        onClearFilters={clearFilters}
      />

      <LibraryEstateTable
        libraries={filteredLibraries}
        onSelectLibrary={selectLibrary}
        getLibraryControlScore={getLibraryControlScore}
      />
```

```
</GlassPanel>

<SelectedLibraryDetail
  library={selectedLibrary}
  onAssignMaintenanceMode={assignMaintenanceMode}
/>

<div className="grid gap-6 xl:grid-cols-12">
  <div className="space-y-6 xl:col-span-7">
    <LeverageFixesPanel fixes={leverageFixes} />
    <BlockersRisksPanel blockers={blockers} risks={risks} />

    {/* governance rules still inline for now */}
    <GlassPanel className="rounded-3xl p-5">
      <h2 className="mb-4 text-lg font-semibold">Library Governance Rules</h2>

      <div className="space-y-3">
        {governanceRules.map((rule) => (
          <div
            key={rule.id}
            className="rounded-2xl border border-white/10 bg-white/5 p-4"
          >
            <div className="flex items-center justify-between gap-3">
              <div className="font-medium">{rule.title}</div>
              <StatusChip label={rule.severity} tone="info" />
            </div>

            <p className="mt-2 text-sm text-white/70">{rule.description}</p>
          </div>
        ))}
      </div>
    </GlassPanel>
  </div>

  <div className="space-y-6 xl:col-span-5">
    {/* maintenance modes still inline for now */}
    <GlassPanel className="rounded-3xl p-5">
      <h2 className="mb-4 text-lg font-semibold">Maintenance Modes</h2>

      <div className="space-y-3">
        {maintenanceModes.map((mode) => (
          <div
            key={mode.id}
            className="rounded-2xl border border-white/10 bg-white/5 p-4"
```

```
          >
            <div className="flex items-center justify-between gap-3">
              <div>
                <div className="font-medium">
                  {mode.id} — {mode.name}
                </div>
                <div className="mt-1 text-sm text-white/60">{mode.purpose}</div>
              </div>

              <StatusChip
                label={mode.launchable ? "launchable" : "locked"}
                tone={mode.launchable ? "success" : "warning"}
              />
            </div>

            <div className="mt-3 text-sm text-white/75">
              {mode.triggerCondition}
            </div>
          </div>
        ))}
      </div>
    </GlassPanel>

    {/* dependency panel still inline for now */}
    <GlassPanel className="rounded-3xl p-5">
      <h2 className="mb-4 text-lg font-semibold">Dependency and Authority Map</h2>

      <div className="grid gap-3 sm:grid-cols-2">
        <MiniStat label="Total Links" value={dependencySummary.total} />
        <MiniStat label="Governance Links" value={dependencySummary.governedLinks} />
        <MiniStat label="Blocked Links" value={dependencySummary.blockedLinks} />
        <MiniStat label="Support Links" value={dependencySummary.supportLinks} />
      </div>

      <div className="mt-4 space-y-2">
        {dependencyLinks.slice(0, 8).map((link, index) => (
          <div
            key={`${link.fromLibraryId}-${link.toLibraryId}-${index}`}
            className="flex items-center justify-between rounded-xl border border-white/10
bg-white/5 px-3 py-2 text-sm"
          >
            <div className="text-white/75">
              {link.fromLibraryId} → {link.toLibraryId}
            </div>
```

```
          <div className="flex items-center gap-2">
            <StatusChip label={link.relation} tone="info" />
            {link.severity && (
              <StatusChip
                label={link.severity}
                tone={getRiskTone(link.severity)}
              />
            )}
          </div>
        </div>
      ))}
    </div>
  </GlassPanel>

  {/* recent activity still inline for now */}
  <GlassPanel className="rounded-3xl p-5">
    <h2 className="mb-4 text-lg font-semibold">Recent Library Activity</h2>

    <div className="space-y-3">
      {recentActivity.map((event) => (
        <div
          key={event.id}
          className="rounded-2xl border border-white/10 bg-white/5 p-4"
        >
          <div className="flex items-center justify-between gap-3">
            <div className="font-medium">{event.title}</div>
            <div className="text-xs text-white/45">{event.timestamp}</div>
          </div>

          <p className="mt-2 text-sm text-white/70">{event.summary}</p>

          <div className="mt-3 flex items-center gap-2">
            <StatusChip label={event.type} tone="info" />
            {event.libraryId && (
              <StatusChip label={event.libraryId} tone="default" />
            )}
          </div>
        </div>
      ))}
    </div>
  </GlassPanel>
 </div>
</div>
```

```
  </div>
 );
}
```

One cleanup note

In `selected-library-detail.tsx`, I imported `MiniStat` initially while using the custom `MetricTile`. That import is unnecessary. The cleaner version is to remove `MiniStat` from the import line.

Use this import instead:

```
import { formatPercent, getAuthorityTone } from "./shared";
```

What batch 3 removed from the page

The page now hands off:

- selected library inspection
- leverage fix ranking
- blockers/risk analysis

That is the core control/inspection UI, so the page file is now much closer to a final orchestrator.

extraction batch 4

Here is **extraction batch 4**.

This finishes the remaining inline sections:

- `governance-rules-panel.tsx`
- `maintenance-modes-panel.tsx`
- `dependency-map-panel.tsx`
- `recent-library-activity-panel.tsx`

After this batch, `library-control.tsx` becomes the clean orchestration page it was supposed to be.

App/src/components/library-control/governance-rules-panel.tsx

```
import type { GovernanceRule } from "../../data/library-control";
import { GlassPanel } from "../glass-panel";
import { StatusChip } from "../status-chip";

type GovernanceRulesPanelProps = {
 rules: GovernanceRule[];
};

export function GovernanceRulesPanel({
 rules,
}: GovernanceRulesPanelProps) {
 return (
   <GlassPanel className="rounded-3xl p-5">
     <h2 className="mb-4 text-lg font-semibold">Library Governance Rules</h2>

     <div className="space-y-3">
       {rules.map((rule) => (
         <div
           key={rule.id}
           className="rounded-2xl border border-white/10 bg-white/5 p-4"
         >
```

```
          <div className="flex items-center justify-between gap-3">
            <div className="font-medium">{rule.title}</div>
            <StatusChip label={rule.severity} tone="info" />
          </div>

          <p className="mt-2 text-sm text-white/70">{rule.description}</p>
        </div>
      ))}
    </div>
  </GlassPanel>
 );
}
```

App/src/components/library-control/maintenance-modes-panel.tsx

```
import type { MaintenanceModeCard } from "../../data/library-control";
import { GlassPanel } from "../glass-panel";
import { StatusChip } from "../status-chip";

type MaintenanceModesPanelProps = {
 modes: MaintenanceModeCard[];
};

export function MaintenanceModesPanel({
 modes,
}: MaintenanceModesPanelProps) {
 return (
   <GlassPanel className="rounded-3xl p-5">
     <h2 className="mb-4 text-lg font-semibold">Maintenance Modes</h2>

     <div className="space-y-3">
       {modes.map((mode) => (
         <div
           key={mode.id}
           className="rounded-2xl border border-white/10 bg-white/5 p-4"
         >
           <div className="flex items-center justify-between gap-3">
             <div>
               <div className="font-medium">
                 {mode.id} — {mode.name}
               </div>
```

```
            <div className="mt-1 text-sm text-white/60">{mode.purpose}</div>
          </div>

          <StatusChip
            label={mode.launchable ? "launchable" : "locked"}
            tone={mode.launchable ? "success" : "warning"}
          />
        </div>

        <div className="mt-3 text-sm text-white/75">
          {mode.triggerCondition}
        </div>
      </div>
    ))}
  </div>
 </GlassPanel>
);
}
```

App/src/components/library-control/dependency-map-panel.tsx

```
import type { DependencyLink } from "../../data/library-control";
import { GlassPanel } from "../glass-panel";
import { StatusChip } from "../status-chip";
import { MiniStat, getRiskTone } from "./shared";

type DependencyMapPanelProps = {
 links: DependencyLink[];
};

export function DependencyMapPanel({
 links,
}: DependencyMapPanelProps) {
 const governedLinks = links.filter((l) => l.relation === "governs").length;
 const blockedLinks = links.filter((l) => l.relation === "blocks").length;
 const supportLinks = links.filter((l) => l.relation === "supports").length;

 return (
  <GlassPanel className="rounded-3xl p-5">
   <h2 className="mb-4 text-lg font-semibold">Dependency and Authority Map</h2>
```

```
    <div className="grid gap-3 sm:grid-cols-2">
      <MiniStat label="Total Links" value={links.length} />
      <MiniStat label="Governance Links" value={governedLinks} />
      <MiniStat label="Blocked Links" value={blockedLinks} />
      <MiniStat label="Support Links" value={supportLinks} />
    </div>

    <div className="mt-4 space-y-2">
      {links.slice(0, 8).map((link, index) => (
        <div
          key={`${link.fromLibraryId}-${link.toLibraryId}-${index}`}
          className="flex items-center justify-between rounded-xl border border-white/10
bg-white/5 px-3 py-2 text-sm"
        >
          <div className="text-white/75">
            {link.fromLibraryId} → {link.toLibraryId}
          </div>

          <div className="flex items-center gap-2">
            <StatusChip label={link.relation} tone="info" />
            {link.severity && (
              <StatusChip
                label={link.severity}
                tone={getRiskTone(link.severity)}
              />
            )}
          </div>
        </div>
      ))}
    </div>
  </GlassPanel>
 );
}
```

App/src/components/library-control/recent-library-activity-panel.tsx

```
import type { LibraryActivityEvent } from "../../data/library-control";
import { GlassPanel } from "../glass-panel";
import { StatusChip } from "../status-chip";

type RecentLibraryActivityPanelProps = {
```

```
 events: LibraryActivityEvent[];
};

export function RecentLibraryActivityPanel({
 events,
}: RecentLibraryActivityPanelProps) {
 return (
   <GlassPanel className="rounded-3xl p-5">
     <h2 className="mb-4 text-lg font-semibold">Recent Library Activity</h2>

     <div className="space-y-3">
       {events.map((event) => (
         <div
           key={event.id}
           className="rounded-2xl border border-white/10 bg-white/5 p-4"
         >
           <div className="flex items-center justify-between gap-3">
             <div className="font-medium">{event.title}</div>
             <div className="text-xs text-white/45">{event.timestamp}</div>
           </div>

           <p className="mt-2 text-sm text-white/70">{event.summary}</p>

           <div className="mt-3 flex items-center gap-2">
             <StatusChip label={event.type} tone="info" />
             {event.libraryId && (
               <StatusChip label={event.libraryId} tone="default" />
             )}
           </div>
         </div>
       ))}
     </div>
   </GlassPanel>
 );
}
```

Final `App/src/pages/library-control.tsx`

```
import { useLibraryControl } from "../hooks/use-library-control";

import { GlassPanel } from "../components/glass-panel";
import { LibraryControlHeader } from "../components/library-control/library-control-header";
import { EstateStatusGrid } from "../components/library-control/estate-status-grid";
```

```
import { ControlActionsStrip } from "../components/library-control/control-actions-strip";
import { LibraryHealthOverview } from "../components/library-control/library-health-overview";
import { LibraryEstateFilters } from "../components/library-control/library-estate-filters";
import { LibraryEstateTable } from "../components/library-control/library-estate-table";
import { SelectedLibraryDetail } from "../components/library-control/selected-library-detail";
import { LeverageFixesPanel } from "../components/library-control/leverage-fixes-panel";
import { BlockersRisksPanel } from "../components/library-control/blockers-risks-panel";
import { GovernanceRulesPanel } from "../components/library-control/governance-rules-panel";
import { MaintenanceModesPanel } from
"../components/library-control/maintenance-modes-panel";
import { DependencyMapPanel } from "../components/library-control/dependency-map-panel";
import { RecentLibraryActivityPanel } from
"../components/library-control/recent-library-activity-panel";

export default function LibraryControlCenterPage() {
 const control = useLibraryControl();

 return (
   <div className="space-y-6 p-6">
     <LibraryControlHeader header={control.header} />

     <EstateStatusGrid
       metrics={control.metrics}
       visibleCount={control.visibleCount}
       totalCount={control.totalCount}
     />

     <ControlActionsStrip
       actions={control.actions}
       onRunAction={control.runControlAction}
     />

     <LibraryHealthOverview healthBands={control.healthBands} />

     <GlassPanel className="rounded-3xl p-5">
       <LibraryEstateFilters
         filters={control.filters}
         hasActiveFilters={control.hasActiveFilters}
         onQueryChange={control.setQuery}
         onAuthorityFilterChange={control.setAuthorityFilter}
         onControlStateFilterChange={control.setControlStateFilter}
         onRiskFilterChange={control.setRiskFilter}
         onSortChange={control.setSortBy}
         onToggleStaleOnly={control.toggleStaleOnly}
```

```
        onToggleBlockedOnly={control.toggleBlockedOnly}
        onToggleMissingAuthorityOnly={control.toggleMissingAuthorityOnly}
        onToggleDriftOnly={control.toggleDriftOnly}
        onClearFilters={control.clearFilters}
      />

      <LibraryEstateTable
        libraries={control.filteredLibraries}
        onSelectLibrary={control.selectLibrary}
        getLibraryControlScore={control.getLibraryControlScore}
      />
    </GlassPanel>

    <SelectedLibraryDetail
      library={control.selectedLibrary}
      onAssignMaintenanceMode={control.assignMaintenanceMode}
    />

    <div className="grid gap-6 xl:grid-cols-12">
      <div className="space-y-6 xl:col-span-7">
        <LeverageFixesPanel fixes={control.leverageFixes} />
        <BlockersRisksPanel
          blockers={control.blockers}
          risks={control.risks}
        />
        <GovernanceRulesPanel rules={control.governanceRules} />
      </div>

      <div className="space-y-6 xl:col-span-5">
        <MaintenanceModesPanel modes={control.maintenanceModes} />
        <DependencyMapPanel links={control.dependencyLinks} />
        <RecentLibraryActivityPanel events={control.recentActivity} />
      </div>
    </div>
  </div>
 );
}
```

What the page is now

At this point, the page file is finally doing the right job:

- call the hook
- lay out sections
- pass props

Everything else is pushed into section components.

Current extracted set

You now have:

- `library-control-header.tsx`
- `estate-status-grid.tsx`
- `control-actions-strip.tsx`
- `library-health-overview.tsx`
- `library-estate-filters.tsx`
- `library-estate-table.tsx`
- `selected-library-detail.tsx`
- `leverage-fixes-panel.tsx`
- `blockers-risks-panel.tsx`
- `governance-rules-panel.tsx`
- `maintenance-modes-panel.tsx`
- `dependency-map-panel.tsx`
- `recent-library-activity-panel.tsx`
- `shared.tsx`

cleanup pass

Here's the **cleanup pass** to finish the repurpose cleanly.

At this point the architecture is right. The remaining work is mostly about:

- removing leftovers from the old upgrade page
- tightening shared helpers
- standardizing UI behavior
- making sure the new page is consistent and low-drift

Cleanup pass goals

1. Remove old upgrade-era leftovers

Delete anything still tied to the old upgrade matrix model:

- old nav label and route references
- `useUpgradeProgress`
- `upgrade-matrix` data imports
- phase-based helpers
- phase timeline imports/components
- phase toggle state
- artifact completion UI that only existed for upgrade tracking

2. Normalize the new component family

Make all `library-control/*` components follow the same rules for:

- headings

- spacing
- chip tones
- card padding
- empty states
- list styling

3. Reduce duplicate helpers

You already moved most helper logic into `shared.tsx`. Now finish consolidating any repeated:

- tone mappers
- percent formatting
- section heading patterns
- small stat card patterns

Final file check

Your final folder should look like this:

```
App/src/pages/library-control.tsx

App/src/components/library-control/
 shared.tsx
 library-control-header.tsx
 estate-status-grid.tsx
 control-actions-strip.tsx
 library-health-overview.tsx
 library-estate-filters.tsx
 library-estate-table.tsx
 selected-library-detail.tsx
 leverage-fixes-panel.tsx
```

blockers-risks-panel.tsx
governance-rules-panel.tsx
maintenance-modes-panel.tsx
dependency-map-panel.tsx
recent-library-activity-panel.tsx

App/src/hooks/use-library-control.ts
App/src/data/library-control.ts

Cleanup checklist by file

A. `App/src/pages/library-control.tsx`

Keep only:

- `useLibraryControl`
- section component imports
- page layout markup

Remove:

- inline helper functions
- inline list cards
- inline metric tiles
- inline tone mapping
- any `useMemo` no longer needed
- any icon imports not used directly in this page

Final page should look roughly like:

```
export default function LibraryControlCenterPage() {
 const control = useLibraryControl();
```

```
return (
  <div className="space-y-6 p-6">
    <LibraryControlHeader header={control.header} />

    <EstateStatusGrid
      metrics={control.metrics}
      visibleCount={control.visibleCount}
      totalCount={control.totalCount}
    />

    <ControlActionsStrip
      actions={control.actions}
      onRunAction={control.runControlAction}
    />

    <LibraryHealthOverview healthBands={control.healthBands} />

    <GlassPanel className="rounded-3xl p-5">
      <LibraryEstateFilters
        filters={control.filters}
        hasActiveFilters={control.hasActiveFilters}
        onQueryChange={control.setQuery}
        onAuthorityFilterChange={control.setAuthorityFilter}
        onControlStateFilterChange={control.setControlStateFilter}
        onRiskFilterChange={control.setRiskFilter}
        onSortChange={control.setSortBy}
        onToggleStaleOnly={control.toggleStaleOnly}
        onToggleBlockedOnly={control.toggleBlockedOnly}
        onToggleMissingAuthorityOnly={control.toggleMissingAuthorityOnly}
        onToggleDriftOnly={control.toggleDriftOnly}
        onClearFilters={control.clearFilters}
      />

      <LibraryEstateTable
        libraries={control.filteredLibraries}
        onSelectLibrary={control.selectLibrary}
        getLibraryControlScore={control.getLibraryControlScore}
      />
    </GlassPanel>

    <SelectedLibraryDetail
      library={control.selectedLibrary}
      onAssignMaintenanceMode={control.assignMaintenanceMode}
```

```
    />

    <div className="grid gap-6 xl:grid-cols-12">
      <div className="space-y-6 xl:col-span-7">
        <LeverageFixesPanel fixes={control.leverageFixes} />
        <BlockersRisksPanel blockers={control.blockers} risks={control.risks} />
        <GovernanceRulesPanel rules={control.governanceRules} />
      </div>

      <div className="space-y-6 xl:col-span-5">
        <MaintenanceModesPanel modes={control.maintenanceModes} />
        <DependencyMapPanel links={control.dependencyLinks} />
        <RecentLibraryActivityPanel events={control.recentActivity} />
      </div>
    </div>
  </div>
 );
}
```

B. `shared.tsx`

Keep here:

- `BadgeTone`
- `getRiskTone`
- `getControlStateTone`
- `getAuthorityTone`
- `formatPercent`
- `MiniStat`
- `SectionTitle`

Optional additions

Add a reusable empty state block so multiple components stay consistent:

```
import { ReactNode } from "react";

export function EmptyState({
 message,
 className = "",
}: {
 message: string;
 className?: string;
}) {
 return (
   <div
     className={`rounded-2xl border border-dashed border-white/10 bg-white/5 p-8 text-sm
text-white/50 ${className}`}
   >
     {message}
   </div>
 );
}
```

Then use it in:

- `selected-library-detail.tsx`
- `library-estate-table.tsx` empty state

Optional addition

Add a shared severity tone mapper for governance rules if you want stricter visual consistency:

```
export function getRuleSeverityTone(severity: string): BadgeTone {
 switch (severity) {
   case "strict":
     return "danger";
   case "warning":
     return "warning";
   case "info":
   default:
     return "info";
 }
}
```

C. `selected-library-detail.tsx`

Cleanup target

Remove unused imports.

You noted this already:

- remove `MiniStat` import if not used

Recommended final import

```
import { formatPercent, getAuthorityTone } from "./shared";
```

Optional cleanup

Move `MetricTile` to shared only if you plan to reuse it elsewhere.
If it is only used here, leave it local.

D. `library-estate-table.tsx`

Good as-is, but add one refinement:

Highlight the selected row once you decide to pass `selectedLibraryId`.

Updated prop

```
type LibraryEstateTableProps = {
 libraries: LibraryControlRecord[];
 selectedLibraryId?: string | null;
 onSelectLibrary: (libraryId: string) => void;
 getLibraryControlScore: (library: LibraryControlRecord) => number;
};
```

Row class improvement

```
className={`cursor-pointer rounded-2xl border transition ${
 selectedLibraryId === library.id
   ? "border-white/25 bg-white/10"
   : "border-white/10 bg-white/5 hover:bg-white/10"
}`}
```

That makes table selection much clearer.

E. `library-health-overview.tsx`

Cleanup improvement

Right now the band ID pill is plain text. You may want a tone system here too so the health bands communicate severity better.

Suggested mapping:

- authoritative → success
- stable → info
- incomplete → warning
- stale → warning
- fragmented → danger
- untrusted → danger

If you want that, add a helper in this component or in `shared.tsx`.

F. `governance-rules-panel.tsx`

Cleanup improvement

Right now all rules use `tone="info"`.

That flattens the importance of:

- `strict`
- `warning`

- `info`

Use the rule severity tone mapper instead:

```
<StatusChip
 label={rule.severity}
 tone={getRuleSeverityTone(rule.severity)}
/>
```

G. `recent-library-activity-panel.tsx`

Cleanup improvement

Format timestamps before display.

Right now raw ISO timestamps are not ideal.

Add a helper:

```
function formatEventTime(timestamp: string) {
 const date = new Date(timestamp);
 return date.toLocaleString();
}
```

Then:

```
<div className="text-xs text-white/45">{formatEventTime(event.timestamp)}</div>
```

That will read much better in the UI.

H. `maintenance-modes-panel.tsx`

Optional cleanup

Add a second line or chip row for `suggestedForStates` if you want the modes to feel more operational.

Example:

```
<div className="mt-3 flex flex-wrap gap-2">
```

```
 {mode.suggestedForStates.map((state) => (
   <StatusChip key={state} label={state} tone="info" />
 ))}
</div>
```

This makes the panel more useful without needing deeper drill-in behavior yet.

I. `dependency-map-panel.tsx`

Cleanup improvement

The summary math is inside the component, which is fine.
But if you want consistency and testability, move it into a tiny helper:

```
function summarizeLinks(links: DependencyLink[]) {
 return {
   total: links.length,
   governed: links.filter((l) => l.relation === "governs").length,
   blocked: links.filter((l) => l.relation === "blocks").length,
   support: links.filter((l) => l.relation === "supports").length,
 };
}
```

This keeps the render cleaner.

Hook cleanup pass

`use-library-control.ts`

Keep

- filters
- selected library state
- derived subsets

- filter setters
- quick presets
- action handlers

Review for cleanup

1. Make sure there are no imports from old upgrade hooks.
2. Make sure no dead presets remain unused by the page.
3. Decide whether `healthBands` should stay static or become filter-aware.

Best choice right now

Keep `healthBands` static.
That gives the page a stable estate-level view even when the table is filtered.

Small improvement

Expose `selectedLibraryRecord` in addition to `selectedLibrary` if you later want table selection syncing or route opening behavior.

Data pack cleanup pass

`library-control.ts`

Review these areas:

- replace placeholder library names with real names
- replace placeholder routes
- tighten detail text

- tighten dependency links
- verify control states align with the helper formulas
- verify header metrics still match actual library data

Important cleanup

Because the current library data was built with placeholders, the strongest improvement now is replacing:

- `Library 01`
- `Library 02`
- etc.

with the real 16 library names from your actual system.

That is the biggest quality jump left in the data layer.

Route and nav cleanup

Final route decision

Choose one and standardize everywhere:

- `/library-control`
  or
- `/libraries/control`

I'd recommend:

/libraries/control

because it fits a system that may later also have:

- `/libraries`
- `/libraries/:id`
- `/libraries/upgrade`
- `/libraries/audit`

Nav label

Use exactly:

Library Control

Page title

Use exactly:

Library Control Center

Remove old route/page references

Search your repo for:

- `Library Upgrade Matrix`
- `upgrade matrix`
- `useUpgradeProgress`
- `upgrade-matrix`
- `PhaseTimeline`
- `phase progress`

- `crossPhaseRules`
- `fastestLiftOrder` if still imported from old sources
- `openPhases`
- `togglePhase`

Anything still powering this page should be removed or moved to the old planning page.

Visual consistency pass

Apply these rules across all extracted components:

Card shells

Use one standard shell:

rounded-2xl border border-white/10 bg-white/5

Panel shells

Use:

rounded-3xl p-5

or

rounded-3xl p-6

but not both randomly unless there is a real hierarchy reason.

Section titles

Use `SectionTitle` wherever possible.

Right now some extracted components still do:

<h2 className="mb-4 text-lg font-semibold">...</h2>

That is okay, but a more unified system is better.

You can either:

- keep `SectionTitle` only for multi-line headers
- or use it everywhere for consistency

I'd lean toward using it more broadly.

Type cleanup

Do a final pass for:

- `any`
- cast-heavy select handlers
- duplicate prop types
- unnecessary `string` where union types exist

Example improvement

In filter selects, the casts are acceptable, but if you want them tighter you can extract option arrays and typed parsers instead of direct casting from `e.target.value`.

Not required for now, but worth noting.

Best final polish items

These are the most valuable finishing touches:

1. Selected row highlight in table

This is probably the best UX cleanup still missing.

2. Formatted timestamps

Makes the activity panel feel finished.

3. Severity-aware governance rule chips

Improves scan value immediately.

4. Replace placeholder library names

This is the biggest content cleanup left.

5. Route/nav rename completion

Ensures no old “upgrade matrix” identity leaks remain.

Final cleanup target state

When done, this page should have:

- no upgrade-phase language
- no phase UI
- no old upgrade data imports
- no inline helper clutter in the page file
- no unused imports in extracted components
- one clear page identity: **Library Control Center**
- one clean nav identity: **Library Control**

Recommended order to finish

1. remove old upgrade leftovers
2. clean page imports
3. clean extracted component imports
4. standardize chip tones
5. format timestamps
6. add selected row highlight
7. swap placeholder library names/routes for real ones
8. update nav/route labels

repo search cleanup map

Repo search cleanup map

Use this as the final sweep to remove old identity and prevent drift.

1. Search terms to remove from the old page identity

Search the repo for each of these:

Library Upgrade Matrix
upgrade matrix
useUpgradeProgress
upgrade-matrix
PhaseTimeline
openPhases
togglePhase
getOverallProgress
getPhaseProgress
crossPhaseRules
fastestLiftOrder
artifact completion
phase progress
phase detail
phase target outcomes

2. Replace targets

Old page identity

Replace:

Library Upgrade Matrix

with:

Library Control Center

Old nav label

Replace old nav labels like:

Upgrade Matrix

Library Upgrade

with:

Library Control

Old route

If the old page route is still something like:

/upgrade-matrix
/library-upgrade

replace with:

/libraries/control

---

3. Files most likely to still contain old references

Check these first:

App/src/pages/
App/src/components/navigation/
App/src/router/
App/src/routes/
App/src/data/
App/src/hooks/
App/src/layout/

Especially:

- nav config
- sidebar config
- route registration
- breadcrumb config
- page title/meta config

---

4. Remove old imports from the page

In the new `library-control.tsx`, confirm none of these remain:

useUpgradeProgress
../data/upgrade-matrix
PhaseTimeline
crossPhaseRules
fastestLiftOrder

If any still exist, they should be fully removed from the control page.

---

5. Search for old phase-oriented UI concepts

These are the most dangerous drift points because they can survive even after the rename:

Phase 1
Phase 2
Phase 3
phase-based
phase completion
phase breakdown
phase checklist
phase expansion

If those are still in the control page flow, remove or relocate them to a separate planning page.

---

6. Search for lingering upgrade-specific data fields

Look for fields like:

phase
priority
tier
done
score
artifactsComplete
phaseStatus

Not all of these are wrong everywhere, but if they are still driving the **Library Control Center** table, they should be replaced by:

authorityStatus
coveragePercent
freshnessPercent
integrityPercent
dependencyPressure
controlState
risk
recommendedAction

---

7. Nav and routing cleanup checklist

Verify these are all aligned:

- sidebar label = `Library Control`
- route path = `/libraries/control`
- page title = `Library Control Center`
- breadcrumbs = `Libraries / Control`
- browser title / metadata = `Library Control Center`

---

8. New symbol map to standardize on

After cleanup, the control page should consistently use these symbols:

Page

LibraryControlCenterPage

Hook

useLibraryControl

Data

libraryControlCenterState

Components

LibraryControlHeader
EstateStatusGrid
ControlActionsStrip
LibraryHealthOverview
LibraryEstateFilters
LibraryEstateTable
SelectedLibraryDetail
LeverageFixesPanel
BlockersRisksPanel
GovernanceRulesPanel
MaintenanceModesPanel
DependencyMapPanel
RecentLibraryActivityPanel

If older names still coexist for the same responsibility, remove them.

---

9. Final dead-code sweep

After the rename/search pass, run a dead-code check for:

- unused imports
- unused helper functions
- unused local components
- old data files no longer referenced
- old hook files no longer referenced

Most likely candidates:

- old upgrade page file
- old upgrade hook
- old phase timeline component

- old upgrade matrix data file

---

10. Best final manual test

After cleanup, verify this flow:

1. open `Library Control`
2. page title says `Library Control Center`
3. no mention of upgrade phases appears
4. filters work
5. table row selection works
6. selected detail updates
7. blockers/risks render
8. maintenance modes render
9. recent activity renders
10. no console errors from removed old imports

11. Clean search-and-replace execution order

Do it in this order:

1. nav label + route
2. page title strings
3. old imports
4. old hook/data references

5. phase-based symbols
6. dead-code sweep
7. manual UI check

final integration checklist

Here's the **final integration checklist** for moving the repurposed page into the app cleanly.

1. Register the new page

Create or confirm the page file

Use:

App/src/pages/library-control.tsx

Make sure it exports:

```
export default function LibraryControlCenterPage() { ... }
```

Confirm supporting files exist

You should now have:

```
App/src/data/library-control.ts
App/src/hooks/use-library-control.ts

App/src/components/library-control/
 shared.tsx
 library-control-header.tsx
 estate-status-grid.tsx
 control-actions-strip.tsx
 library-health-overview.tsx
 library-estate-filters.tsx
 library-estate-table.tsx
 selected-library-detail.tsx
 leverage-fixes-panel.tsx
 blockers-risks-panel.tsx
 governance-rules-panel.tsx
 maintenance-modes-panel.tsx
 dependency-map-panel.tsx
 recent-library-activity-panel.tsx
```

---

2. Wire the route

Recommended route

Use:

/libraries/control

Router update

Wherever your app registers page routes, add:

```
<Route path="/libraries/control" component={LibraryControlCenterPage} />
```

Or if your router style is object-based:

```
{
 path: "/libraries/control",
 component: LibraryControlCenterPage,
}
```

Remove or retire old route

If the old upgrade page had its own route, choose one:

Option A — replace it directly

Point the old route to the new page temporarily.

Option B — retire it

Remove the old route and let the new control route become canonical.

Option C — keep both

Only do this if you are intentionally preserving:

- `Library Control Center`
- `Library Upgrade Matrix`

as two separate pages.

If you keep both, make sure they do not share the same nav label.

---

3. Update nav / sidebar

Final nav label

Use exactly:

Library Control

Nav entry shape

Example:

```
{
 label: "Library Control",
 href: "/libraries/control",
 icon: Library,
}
```

Remove old labels

Retire labels such as:

- Upgrade Matrix
- Library Upgrade
- Library Upgrade Matrix

unless you are intentionally keeping a separate planning page.

---

4. Update page identity everywhere

Standardize these values:

Page title

Library Control Center

Nav label

Library Control

Breadcrumbs

Libraries / Control

Metadata / document title

Library Control Center

Do not leave mixed identity like:

- route says control
- page says upgrade matrix
- nav says library upgrade

That creates drift immediately.

---

5. Replace old page import targets

In any route or nav config, replace old imports like:

import UpgradeMatrixPage from "...";

with:

import LibraryControlCenterPage from "../pages/library-control";

Also remove any old references to:

- old page component name

- old route path
- old page title constants

---

6. Swap data source to the new control model

Old source to remove from this page

The new page should no longer read from:

- old upgrade matrix datasets
- upgrade progress hooks
- phase progress helpers

New source

The control page should use only:

import { useLibraryControl } from "../hooks/use-library-control";

and the hook should read from:

import { libraryControlCenterState } from "../data/library-control";

This is the main behavior/data pivot that finishes the repurpose.

---

7. Replace placeholder library data with real library records

This is the biggest integration task still left.

Replace in `library-control.ts`

Swap out:

- `Library 01`
- `Library 02`
- ...
- `Library 16`

with your real library names.

Also replace:

- fake routes
- generic purpose text
- placeholder dependency links
- placeholder ownership fields

Minimum real fields to fill first

For each library, fill:

- `id`
- `name`
- `shortName`
- `route`
- `authorityStatus`

- `coveragePercent`
- `freshnessPercent`
- `integrityPercent`
- `dependencyPressure`
- `risk`
- `controlState`
- `recommendedAction`

Then refine:

- `authoritySource`
- `missingArtifacts`
- `staleArtifacts`
- `upstreamDependencies`
- `downstreamDependents`
- `blockerImpact`

---

8. Verify component import paths

Before running the page, confirm these imports match your actual app structure:

- `GlassPanel`

- `MetricCard`
- `StatusChip`

If those live in different paths in your app, correct them now.

This is one of the most likely integration break points.

---

9. Add selected row highlighting

This is worth integrating before calling the page done.

Update table props

Pass:

selectedLibraryId={control.selectedLibraryId}

to `LibraryEstateTable`.

Update table component

Use the selected row styling so the inspection flow is obvious.

That turns the table + detail panel into a much cleaner operator workflow.

---

10. Format recent activity timestamps

Before shipping, update `recent-library-activity-panel.tsx` so it does not display raw ISO timestamps.

Use a small formatter inside that component, for example:

```
function formatEventTime(timestamp: string) {
 return new Date(timestamp).toLocaleString();
}
```

Then render the formatted value.

---

11. Standardize chip severity mapping

Do one final pass so chip tones are meaningful across the page.

Recommended mapping

Risk

- low → success
- moderate → info
- high → warning
- critical → danger

Control state

- governed → success
- review-required → info
- degraded → warning
- blocked → warning
- unsafe → danger
- recovery → info

Rule severity

- info → info
- warning → warning

- strict → danger

This helps the page scan much faster.

---

12. Remove old page leftovers from the repo

Once the new route works, remove or isolate the old upgrade page artifacts.

Candidates to retire

- old upgrade page component
- old upgrade hook
- old upgrade matrix data file
- old phase timeline component
- old phase detail helpers

If you want to preserve them, move them under a clearly separate planning namespace such as:

App/src/pages/library-upgrade.tsx
App/src/data/library-upgrade.ts

Do not leave them appearing to be the same page family.

---

13. Validate the full page flow

Run this final test sequence:

Navigation

- nav shows `Library Control`
- clicking it opens `/libraries/control`

Identity

- page title says `Library Control Center`
- no upgrade-phase wording appears

Top sections

- hero renders
- metrics render
- control actions render
- health overview renders

Main work surface

- search works
- filters work
- sorting works
- quick filter pills work
- table rows render
- clicking a row updates the selected detail panel

Lower panels

- leverage fixes render
- blockers/risks render
- governance rules render
- maintenance modes render
- dependency panel renders
- recent activity renders

Stability

- no missing imports
- no undefined data errors
- no console errors
- no stale references to old upgrade symbols

---

14. Decide what happens to the old upgrade page

You need a clear final decision here.

Option A — replace it entirely

Best if this page fully supersedes the old one.

Option B — keep a separate planning page

Use a clearly different identity such as:

- `Library Upgrade Planner`
- `Library Upgrade Matrix`

If you do this, keep it out of the primary nav unless it is still actively used.

My recommendation is:

- primary nav → `Library Control`
- old upgrade page → separate admin/planning route only if you still need it

---

15. Final target state

When integration is complete, the app should have:

- `Library Control` in nav
- `/libraries/control` route
- `Library Control Center` page title
- control-oriented data source
- no phase/upgrade framing on the main page
- real library data in `library-control.ts`
- clean extracted component structure
- old upgrade page either retired or clearly separated

real data worksheet

Here's the **real data worksheet** for replacing the 16 placeholder library records in `library-control.ts`.

This is designed so you can fill in your actual library estate without drifting field by field.

1. What this worksheet is for

Use this to replace each placeholder record like:

- `LIB-01`
- `Library 01`
- `/library-01`

with the real library data for the **Library Control Center**.

The goal is to fill the **minimum control fields first**, then the **detail fields**, then the **dependency/control context**.

---

2. Fill order

Use this order for each library:

Pass 1 — identity

- id
- name
- shortName
- route
- owner

Pass 2 — control state

- authorityStatus
- coveragePercent
- freshnessPercent
- integrityPercent
- dependencyPressure
- risk
- controlState
- trustDecision
- recommendedAction

Pass 3 — detail panel

- purpose
- estateRole
- authoritySource
- missingArtifacts
- staleArtifacts
- brokenReferences
- unresolvedConflicts
- unsupportedDependents
- upstreamDependencies
- downstreamDependents

- blockerImpact
- suggestedMaintenanceMode

---

3. Allowed values

`authorityStatus`

Use one of:

"authoritative" | "partial" | "missing" | "disputed"

`controlState`

Use one of:

"governed" | "review-required" | "degraded" | "blocked" | "unsafe" | "recovery"

`risk`

Use one of:

"low" | "moderate" | "high" | "critical"

`trustDecision`

Use one of:

"trusted" | "conditional" | "untrusted"

---

4. Master worksheet template

Use this as the fill-in template for each real library.

type RealLibraryWorksheet = {

```
  // identity
  id: string;
  name: string;
  shortName: string;
  route: string;
  owner: string;

  // control summary
  authorityStatus: "authoritative" | "partial" | "missing" | "disputed";
  coveragePercent: number;
  freshnessPercent: number;
  integrityPercent: number;
  dependencyPressure: number;
  risk: "low" | "moderate" | "high" | "critical";
  controlState: "governed" | "review-required" | "degraded" | "blocked" | "unsafe" | "recovery";
  trustDecision: "trusted" | "conditional" | "untrusted";
  recommendedAction: string;

  // detail panel
  purpose: string;
  estateRole: string;
  authoritySource?: string;
  missingArtifacts: string[];
  staleArtifacts: string[];
  brokenReferences: string[];
  unresolvedConflicts: string[];
  unsupportedDependents: string[];
  upstreamDependencies: string[];
  downstreamDependents: string[];
  blockerImpact: string[];
  suggestedMaintenanceMode?: string;
};
```

---

5. Quick scoring guide

Use this to assign values more consistently.

`coveragePercent`

How complete is the library’s governed content?

- `90–100` = very complete
- `75–89` = mostly complete
- `60–74` = usable but incomplete
- `40–59` = weak coverage
- `<40` = severely incomplete

`freshnessPercent`

How current is the library's governing material?

- `90–100` = very current
- `75–89` = current enough
- `60–74` = aging
- `40–59` = stale
- `<40` = severely stale

`integrityPercent`

How structurally sound is the library?

Think:

- broken links
- bad references
- conflicting records
- missing registry data
- internal contradictions

`dependencyPressure`

How many other libraries or systems are exposed if this library is weak?

- `0–25` = isolated
- `26–50` = moderate influence
- `51–75` = important dependency surface
- `76–100` = major bottleneck / high blast radius

---

6. State assignment guide

Use `governed` when

- authority is authoritative
- no major blockers
- coverage/freshness/integrity are strong
- trust is high

Use `review-required` when

- library is usable
- but there are meaningful gaps, age, or minor authority concerns

Use `degraded` when

- the library still functions

- but weakness is visible and risky

Use `blocked` when

- an active dependency or registry issue is preventing normal control use

Use `unsafe` when

- authority is missing/disputed
- trust should not propagate
- downstream use should be constrained

Use `recovery` when

- restoration work is underway
- but the library is not yet stable

---

7. Real record template for `LIBRARIES`

Use this format to replace each placeholder `LibraryControlRecord`.

```
{
 id: "LIB-01",
 name: "REAL_LIBRARY_NAME",
 shortName: "RLN",
 route: "/libraries/real-library-name",
 owner: "REAL_OWNER",

 authorityStatus: "authoritative",
 coveragePercent: 0,
 freshnessPercent: 0,
```

```
 integrityPercent: 0,
 dependencyPressure: 0,

 lastAuditAt: "2026-03-11T00:00:00Z",
 risk: "moderate",
 controlState: "review-required",
 trustDecision: "conditional",

 recommendedAction: "REPLACE_WITH_REAL_NEXT_ACTION",
 dependencyBlockers: [],

 driftAlert: false,
 stale: false,
 missingAuthority: false,
 blocked: false,
}
```

---

8. Real record template for `LIBRARY_DETAILS`

Use this format to replace each placeholder `LibraryDetail`.

```
{
 id: "LIB-01",
 name: "REAL_LIBRARY_NAME",
 shortName: "RLN",
 route: "/libraries/real-library-name",
 owner: "REAL_OWNER",

 purpose: "WHAT_THIS_LIBRARY_GOVERNS",
 estateRole: "WHAT_ROLE_IT_PLAYS_IN_THE_ESTATE",

 authoritySource: "CANONICAL_SOURCE_NAME_OR_PATH",
 authorityStatus: "authoritative",
 trustDecision: "trusted",

 coveragePercent: 0,
 freshnessPercent: 0,
 integrityPercent: 0,
 dependencyFitnessPercent: 0,
```

```
 missingArtifacts: [],
 staleArtifacts: [],
 brokenReferences: [],
 unresolvedConflicts: [],
 unsupportedDependents: [],

 upstreamDependencies: [],
 downstreamDependents: [],
 blockerImpact: [],

 recommendedAction: "REAL_NEXT_ACTION",
 urgency: "moderate",
 suggestedMaintenanceMode: "MM-01",
}
```

---

9. 16-library fill sheet

Use this exact worksheet block and replace each row.

```
export const REAL_LIBRARY_FILL_SHEET = [
 { id: "LIB-01", name: "", shortName: "", route: "", owner: "" },
 { id: "LIB-02", name: "", shortName: "", route: "", owner: "" },
 { id: "LIB-03", name: "", shortName: "", route: "", owner: "" },
 { id: "LIB-04", name: "", shortName: "", route: "", owner: "" },
 { id: "LIB-05", name: "", shortName: "", route: "", owner: "" },
 { id: "LIB-06", name: "", shortName: "", route: "", owner: "" },
 { id: "LIB-07", name: "", shortName: "", route: "", owner: "" },
 { id: "LIB-08", name: "", shortName: "", route: "", owner: "" },
 { id: "LIB-09", name: "", shortName: "", route: "", owner: "" },
 { id: "LIB-10", name: "", shortName: "", route: "", owner: "" },
 { id: "LIB-11", name: "", shortName: "", route: "", owner: "" },
 { id: "LIB-12", name: "", shortName: "", route: "", owner: "" },
 { id: "LIB-13", name: "", shortName: "", route: "", owner: "" },
 { id: "LIB-14", name: "", shortName: "", route: "", owner: "" },
 { id: "LIB-15", name: "", shortName: "", route: "", owner: "" },
 { id: "LIB-16", name: "", shortName: "", route: "", owner: "" },
];
```

---

10. Compact operator worksheet

This version is faster to fill manually before converting to TS.

Library 01

- Name:
- Short name:
- Route:
- Owner:
- Purpose:
- Estate role:
- Authority status:
- Authority source:
- Coverage %:
- Freshness %:
- Integrity %:
- Dependency pressure %:
- Risk:
- Control state:
- Trust decision:
- Recommended action:
- Missing artifacts:
- Stale artifacts:

- Broken references:
- Unresolved conflicts:
- Upstream dependencies:
- Downstream dependents:
- Suggested maintenance mode:

Repeat that same block for all 16 libraries.

---

11. Dependency worksheet

Use this to replace placeholder dependency links.

```
type RealDependencyWorksheet = {
 fromLibraryId: string;
 toLibraryId: string;
 relation: "depends-on" | "governs" | "blocks" | "supports";
 severity?: "low" | "moderate" | "high" | "critical";
};
```

Fill it like:

```
export const REAL_DEPENDENCY_LINKS = [
 {
   fromLibraryId: "LIB-01",
   toLibraryId: "LIB-04",
   relation: "governs",
   severity: "high",
 },
];
```

---

12. Health band assignment worksheet

After real data is filled, reassign band membership.

`authoritative`

Libraries that are strongly governed and trusted

`stable`

Libraries that are sound but not ideal

`incomplete`

Libraries with meaningful coverage or structure gaps

`stale`

Libraries whose freshness is weak

`fragmented`

Libraries suffering from dependency/integrity fragmentation

`untrusted`

Libraries with missing/disputed authority or unsafe control use

You should reassign these based on real values, not placeholder distribution.

---

13. Recommended fill strategy

Best practical order:

Step 1

Fill only:

- id

- name
- shortName
- route
- owner

Step 2

Fill:

- authorityStatus
- coveragePercent
- freshnessPercent
- integrityPercent
- dependencyPressure
- risk
- controlState
- trustDecision
- recommendedAction

Step 3

Fill detail panel fields

Step 4

Fill dependency links

Step 5

Recompute:

- header stats
- estate status metrics
- health bands
- leverage fixes
- blockers
- risks

That prevents you from trying to perfect everything at once.

---

14. Best minimal real-data pass

If you want the page to become useful fast, the minimum real pass is:

For all 16 libraries, replace:

- name
- shortName
- route
- authorityStatus
- coveragePercent
- freshnessPercent
- integrityPercent
- dependencyPressure
- risk

- controlState
- recommendedAction

That alone will make the page feel substantially real.

TS scaffold

```typescript
// App/src/data/library-control.ts
// Real-data scaffold for Library Control Center
// Replace placeholder values with your actual library estate data.

export type AuthorityStatus = "authoritative" | "partial" | "missing" | "disputed";
export type ControlState =
  | "governed"
  | "review-required"
  | "degraded"
  | "blocked"
  | "unsafe"
  | "recovery";
export type RiskLevel = "low" | "moderate" | "high" | "critical";
export type TrustDecision = "trusted" | "conditional" | "untrusted";
export type HealthBandId =
  | "authoritative"
  | "stable"
  | "incomplete"
  | "stale"
  | "fragmented"
  | "untrusted";
export type ActivityType =
  | "audit-run"
  | "drift-detected"
  | "authority-restored"
  | "blocker-cleared"
  | "registry-rebuilt"
  | "downgraded"
  | "restored";
export type DependencyRelation = "depends-on" | "governs" | "blocks" | "supports";
export type ControlActionType =
  | "health-check"
  | "coverage-audit"
  | "drift-detection"
  | "rebuild-registries"
  | "review-authority-gaps"
  | "dependency-refresh"
  | "open-unsafe"
  | "resolve-blockers";

export type EstateHeaderStats = {
  controlScore: number;
  governedLibraries: number;
  activeRisks: number;
```

```
  blockedDependencies: number;
};

export type EstateStatusMetrics = {
  totalLibraries: number;
  governed: number;
  reviewRequired: number;
  stale: number;
  unsafe: number;
  blocked: number;
  missingAuthority: number;
  driftAlerts: number;
};

export type ControlAction = {
  id: string;
  label: string;
  icon: string;
  actionType: ControlActionType;
  enabled: boolean;
};

export type LibraryHealthBand = {
  id: HealthBandId;
  label: string;
  count: number;
  libraryIds: string[];
  recommendedAction: string;
};

export type LibraryControlRecord = {
  id: string;
  name: string;
  shortName: string;
  route?: string;
  owner?: string;

  authorityStatus: AuthorityStatus;
  coveragePercent: number;
  freshnessPercent: number;
  integrityPercent: number;
  dependencyPressure: number;

  lastAuditAt?: string;
```

```
  risk: RiskLevel;
  controlState: ControlState;
  trustDecision: TrustDecision;

  recommendedAction: string;
  dependencyBlockers: string[];

  driftAlert: boolean;
  stale: boolean;
  missingAuthority: boolean;
  blocked: boolean;
};

export type LibraryDetail = {
  id: string;
  name: string;
  shortName: string;
  route?: string;
  owner?: string;

  purpose: string;
  estateRole?: string;

  authoritySource?: string;
  authorityStatus: AuthorityStatus;
  trustDecision: TrustDecision;

  coveragePercent: number;
  freshnessPercent: number;
  integrityPercent: number;
  dependencyFitnessPercent: number;

  missingArtifacts: string[];
  staleArtifacts: string[];
  brokenReferences: string[];
  unresolvedConflicts: string[];
  unsupportedDependents: string[];

  upstreamDependencies: string[];
  downstreamDependents: string[];
  blockerImpact: string[];

  recommendedAction: string;
  urgency: RiskLevel;
```

```
  suggestedMaintenanceMode?: string;
};

export type LeverageFix = {
  id: string;
  libraryId: string;
  title: string;
  summary: string;
  impactRadius: "local" | "cross-library" | "estate-wide";
  effort: "small" | "medium" | "large";
  payoffScore: number;
  recommendedAction: string;
};

export type ActiveBlocker = {
  id: string;
  title: string;
  severity: RiskLevel;
  items: string[];
  affectedLibraries: string[];
};

export type SystemRisk = {
  id: string;
  title: string;
  severity: RiskLevel;
  summary: string;
  affectedLibraries: string[];
};

export type GovernanceRule = {
  id: string;
  title: string;
  description: string;
  severity: "info" | "warning" | "strict";
};

export type MaintenanceModeCard = {
  id: string;
  name: string;
  purpose: string;
  triggerCondition: string;
  suggestedForStates: ControlState[];
  launchable: boolean;
```

```
};

export type DependencyLink = {
  fromLibraryId: string;
  toLibraryId: string;
  relation: DependencyRelation;
  severity?: RiskLevel;
};

export type LibraryActivityEvent = {
  id: string;
  timestamp: string;
  type: ActivityType;
  libraryId?: string;
  title: string;
  summary: string;
};

export type LibraryControlFilters = {
  query: string;
  authorityStatus?: AuthorityStatus[];
  controlState?: ControlState[];
  risk?: RiskLevel[];
  staleOnly?: boolean;
  blockedOnly?: boolean;
  missingAuthorityOnly?: boolean;
  driftOnly?: boolean;
  sortBy?:
    | "risk"
    | "dependency-pressure"
    | "freshness"
    | "coverage"
    | "last-audit"
    | "control-score";
};

export type LibraryControlCenterState = {
  header: EstateHeaderStats;
  metrics: EstateStatusMetrics;
  actions: ControlAction[];
  healthBands: LibraryHealthBand[];
  libraries: LibraryControlRecord[];
  libraryDetails: Record<string, LibraryDetail>;
  leverageFixes: LeverageFix[];
```

```
  blockers: ActiveBlocker[];
  risks: SystemRisk[];
  governanceRules: GovernanceRule[];
  maintenanceModes: MaintenanceModeCard[];
  dependencyLinks: DependencyLink[];
  recentActivity: LibraryActivityEvent[];
  defaultFilters: LibraryControlFilters;
};

const AUTHORITY_SCORE: Record<AuthorityStatus, number> = {
  authoritative: 100,
  partial: 60,
  disputed: 30,
  missing: 0,
};

export function computeLibraryControlScore(input: {
  authorityStatus: AuthorityStatus;
  coveragePercent: number;
  freshnessPercent: number;
  integrityPercent: number;
  dependencyPressure: number;
}) {
  const authorityScore = AUTHORITY_SCORE[input.authorityStatus];
  return Math.round(
    input.coveragePercent * 0.3 +
      input.freshnessPercent * 0.2 +
      input.integrityPercent * 0.25 +
      (100 - input.dependencyPressure) * 0.1 +
      authorityScore * 0.15
  );
}

export function getEstateHeaderStats(libraries: LibraryControlRecord[]): EstateHeaderStats {
  const governedLibraries = libraries.filter((l) => l.controlState === "governed").length;
  const activeRisks = libraries.filter((l) => l.risk === "high" || l.risk === "critical").length;
  const blockedDependencies = libraries.filter((l) => l.blocked).length;

  const controlScore = libraries.length
    ? Math.round(
        libraries.reduce((sum, lib) => {
          return (
            sum +
            computeLibraryControlScore({
```

```
            authorityStatus: lib.authorityStatus,
            coveragePercent: lib.coveragePercent,
            freshnessPercent: lib.freshnessPercent,
            integrityPercent: lib.integrityPercent,
            dependencyPressure: lib.dependencyPressure,
          })
        );
      }, 0) / libraries.length
    )
  : 0;

  return {
    controlScore,
    governedLibraries,
    activeRisks,
    blockedDependencies,
  };
}

export function getEstateStatusMetrics(libraries: LibraryControlRecord[]): EstateStatusMetrics {
  return {
    totalLibraries: libraries.length,
    governed: libraries.filter((l) => l.controlState === "governed").length,
    reviewRequired: libraries.filter((l) => l.controlState === "review-required").length,
    stale: libraries.filter((l) => l.stale).length,
    unsafe: libraries.filter((l) => l.controlState === "unsafe").length,
    blocked: libraries.filter((l) => l.blocked).length,
    missingAuthority: libraries.filter((l) => l.missingAuthority).length,
    driftAlerts: libraries.filter((l) => l.driftAlert).length,
  };
}

// Fill these 16 with your real libraries.
export const LIBRARIES: LibraryControlRecord[] = [
  {
    id: "LIB-01",
    name: "",
    shortName: "",
    route: "",
    owner: "",
    authorityStatus: "authoritative",
    coveragePercent: 0,
    freshnessPercent: 0,
    integrityPercent: 0,
```

```
  dependencyPressure: 0,
  lastAuditAt: "2026-03-11T00:00:00Z",
  risk: "low",
  controlState: "governed",
  trustDecision: "trusted",
  recommendedAction: "",
  dependencyBlockers: [],
  driftAlert: false,
  stale: false,
  missingAuthority: false,
  blocked: false,
},
{
  id: "LIB-02",
  name: "",
  shortName: "",
  route: "",
  owner: "",
  authorityStatus: "partial",
  coveragePercent: 0,
  freshnessPercent: 0,
  integrityPercent: 0,
  dependencyPressure: 0,
  lastAuditAt: "2026-03-11T00:00:00Z",
  risk: "moderate",
  controlState: "review-required",
  trustDecision: "conditional",
  recommendedAction: "",
  dependencyBlockers: [],
  driftAlert: false,
  stale: false,
  missingAuthority: false,
  blocked: false,
},
{
  id: "LIB-03",
  name: "",
  shortName: "",
  route: "",
  owner: "",
  authorityStatus: "partial",
  coveragePercent: 0,
  freshnessPercent: 0,
  integrityPercent: 0,
```

```
  dependencyPressure: 0,
  lastAuditAt: "2026-03-11T00:00:00Z",
  risk: "moderate",
  controlState: "review-required",
  trustDecision: "conditional",
  recommendedAction: "",
  dependencyBlockers: [],
  driftAlert: false,
  stale: false,
  missingAuthority: false,
  blocked: false,
},
{
  id: "LIB-04",
  name: "",
  shortName: "",
  route: "",
  owner: "",
  authorityStatus: "authoritative",
  coveragePercent: 0,
  freshnessPercent: 0,
  integrityPercent: 0,
  dependencyPressure: 0,
  lastAuditAt: "2026-03-11T00:00:00Z",
  risk: "low",
  controlState: "governed",
  trustDecision: "trusted",
  recommendedAction: "",
  dependencyBlockers: [],
  driftAlert: false,
  stale: false,
  missingAuthority: false,
  blocked: false,
},
{
  id: "LIB-05",
  name: "",
  shortName: "",
  route: "",
  owner: "",
  authorityStatus: "partial",
  coveragePercent: 0,
  freshnessPercent: 0,
  integrityPercent: 0,
```

```
  dependencyPressure: 0,
  lastAuditAt: "2026-03-11T00:00:00Z",
  risk: "high",
  controlState: "degraded",
  trustDecision: "conditional",
  recommendedAction: "",
  dependencyBlockers: [],
  driftAlert: false,
  stale: false,
  missingAuthority: false,
  blocked: false,
},
{
  id: "LIB-06",
  name: "",
  shortName: "",
  route: "",
  owner: "",
  authorityStatus: "missing",
  coveragePercent: 0,
  freshnessPercent: 0,
  integrityPercent: 0,
  dependencyPressure: 0,
  lastAuditAt: "2026-03-11T00:00:00Z",
  risk: "critical",
  controlState: "unsafe",
  trustDecision: "untrusted",
  recommendedAction: "",
  dependencyBlockers: [],
  driftAlert: false,
  stale: false,
  missingAuthority: true,
  blocked: false,
},
{
  id: "LIB-07",
  name: "",
  shortName: "",
  route: "",
  owner: "",
  authorityStatus: "authoritative",
  coveragePercent: 0,
  freshnessPercent: 0,
  integrityPercent: 0,
```

```
  dependencyPressure: 0,
  lastAuditAt: "2026-03-11T00:00:00Z",
  risk: "low",
  controlState: "governed",
  trustDecision: "trusted",
  recommendedAction: "",
  dependencyBlockers: [],
  driftAlert: false,
  stale: false,
  missingAuthority: false,
  blocked: false,
},
{
  id: "LIB-08",
  name: "",
  shortName: "",
  route: "",
  owner: "",
  authorityStatus: "disputed",
  coveragePercent: 0,
  freshnessPercent: 0,
  integrityPercent: 0,
  dependencyPressure: 0,
  lastAuditAt: "2026-03-11T00:00:00Z",
  risk: "high",
  controlState: "unsafe",
  trustDecision: "untrusted",
  recommendedAction: "",
  dependencyBlockers: [],
  driftAlert: false,
  stale: false,
  missingAuthority: false,
  blocked: false,
},
{
  id: "LIB-09",
  name: "",
  shortName: "",
  route: "",
  owner: "",
  authorityStatus: "partial",
  coveragePercent: 0,
  freshnessPercent: 0,
  integrityPercent: 0,
```

```
    dependencyPressure: 0,
    lastAuditAt: "2026-03-11T00:00:00Z",
    risk: "moderate",
    controlState: "review-required",
    trustDecision: "conditional",
    recommendedAction: "",
    dependencyBlockers: [],
    driftAlert: false,
    stale: false,
    missingAuthority: false,
    blocked: false,
  },
  {
    id: "LIB-10",
    name: "",
    shortName: "",
    route: "",
    owner: "",
    authorityStatus: "authoritative",
    coveragePercent: 0,
    freshnessPercent: 0,
    integrityPercent: 0,
    dependencyPressure: 0,
    lastAuditAt: "2026-03-11T00:00:00Z",
    risk: "low",
    controlState: "governed",
    trustDecision: "trusted",
    recommendedAction: "",
    dependencyBlockers: [],
    driftAlert: false,
    stale: false,
    missingAuthority: false,
    blocked: false,
  },
  {
    id: "LIB-11",
    name: "",
    shortName: "",
    route: "",
    owner: "",
    authorityStatus: "partial",
    coveragePercent: 0,
    freshnessPercent: 0,
    integrityPercent: 0,
```

```
  dependencyPressure: 0,
  lastAuditAt: "2026-03-11T00:00:00Z",
  risk: "high",
  controlState: "degraded",
  trustDecision: "conditional",
  recommendedAction: "",
  dependencyBlockers: [],
  driftAlert: false,
  stale: false,
  missingAuthority: false,
  blocked: false,
},
{
  id: "LIB-12",
  name: "",
  shortName: "",
  route: "",
  owner: "",
  authorityStatus: "authoritative",
  coveragePercent: 0,
  freshnessPercent: 0,
  integrityPercent: 0,
  dependencyPressure: 0,
  lastAuditAt: "2026-03-11T00:00:00Z",
  risk: "low",
  controlState: "governed",
  trustDecision: "trusted",
  recommendedAction: "",
  dependencyBlockers: [],
  driftAlert: false,
  stale: false,
  missingAuthority: false,
  blocked: false,
},
{
  id: "LIB-13",
  name: "",
  shortName: "",
  route: "",
  owner: "",
  authorityStatus: "partial",
  coveragePercent: 0,
  freshnessPercent: 0,
  integrityPercent: 0,
```

```
  dependencyPressure: 0,
  lastAuditAt: "2026-03-11T00:00:00Z",
  risk: "high",
  controlState: "blocked",
  trustDecision: "conditional",
  recommendedAction: "",
  dependencyBlockers: [],
  driftAlert: false,
  stale: false,
  missingAuthority: false,
  blocked: true,
},
{
  id: "LIB-14",
  name: "",
  shortName: "",
  route: "",
  owner: "",
  authorityStatus: "authoritative",
  coveragePercent: 0,
  freshnessPercent: 0,
  integrityPercent: 0,
  dependencyPressure: 0,
  lastAuditAt: "2026-03-11T00:00:00Z",
  risk: "low",
  controlState: "governed",
  trustDecision: "trusted",
  recommendedAction: "",
  dependencyBlockers: [],
  driftAlert: false,
  stale: false,
  missingAuthority: false,
  blocked: false,
},
{
  id: "LIB-15",
  name: "",
  shortName: "",
  route: "",
  owner: "",
  authorityStatus: "disputed",
  coveragePercent: 0,
  freshnessPercent: 0,
  integrityPercent: 0,
```

```
    dependencyPressure: 0,
    lastAuditAt: "2026-03-11T00:00:00Z",
    risk: "high",
    controlState: "unsafe",
    trustDecision: "untrusted",
    recommendedAction: "",
    dependencyBlockers: [],
    driftAlert: false,
    stale: false,
    missingAuthority: false,
    blocked: false,
  },
  {
    id: "LIB-16",
    name: "",
    shortName: "",
    route: "",
    owner: "",
    authorityStatus: "partial",
    coveragePercent: 0,
    freshnessPercent: 0,
    integrityPercent: 0,
    dependencyPressure: 0,
    lastAuditAt: "2026-03-11T00:00:00Z",
    risk: "moderate",
    controlState: "review-required",
    trustDecision: "conditional",
    recommendedAction: "",
    dependencyBlockers: [],
    driftAlert: false,
    stale: false,
    missingAuthority: false,
    blocked: false,
  },
];

export const LIBRARY_DETAILS: Record<string, LibraryDetail> = {
  "LIB-01": {
    id: "LIB-01",
    name: "",
    shortName: "",
    route: "",
    owner: "",
    purpose: "",
```

```
  estateRole: "",
  authoritySource: "",
  authorityStatus: "authoritative",
  trustDecision: "trusted",
  coveragePercent: 0,
  freshnessPercent: 0,
  integrityPercent: 0,
  dependencyFitnessPercent: 0,
  missingArtifacts: [],
  staleArtifacts: [],
  brokenReferences: [],
  unresolvedConflicts: [],
  unsupportedDependents: [],
  upstreamDependencies: [],
  downstreamDependents: [],
  blockerImpact: [],
  recommendedAction: "",
  urgency: "low",
  suggestedMaintenanceMode: "MM-01",
},
"LIB-02": {
  id: "LIB-02",
  name: "",
  shortName: "",
  route: "",
  owner: "",
  purpose: "",
  estateRole: "",
  authoritySource: "",
  authorityStatus: "partial",
  trustDecision: "conditional",
  coveragePercent: 0,
  freshnessPercent: 0,
  integrityPercent: 0,
  dependencyFitnessPercent: 0,
  missingArtifacts: [],
  staleArtifacts: [],
  brokenReferences: [],
  unresolvedConflicts: [],
  unsupportedDependents: [],
  upstreamDependencies: [],
  downstreamDependents: [],
  blockerImpact: [],
  recommendedAction: "",
```

```
  urgency: "moderate",
  suggestedMaintenanceMode: "MM-03",
},
"LIB-03": {
  id: "LIB-03",
  name: "",
  shortName: "",
  route: "",
  owner: "",
  purpose: "",
  estateRole: "",
  authoritySource: "",
  authorityStatus: "partial",
  trustDecision: "conditional",
  coveragePercent: 0,
  freshnessPercent: 0,
  integrityPercent: 0,
  dependencyFitnessPercent: 0,
  missingArtifacts: [],
  staleArtifacts: [],
  brokenReferences: [],
  unresolvedConflicts: [],
  unsupportedDependents: [],
  upstreamDependencies: [],
  downstreamDependents: [],
  blockerImpact: [],
  recommendedAction: "",
  urgency: "moderate",
  suggestedMaintenanceMode: "MM-03",
},
"LIB-04": {
  id: "LIB-04",
  name: "",
  shortName: "",
  route: "",
  owner: "",
  purpose: "",
  estateRole: "",
  authoritySource: "",
  authorityStatus: "authoritative",
  trustDecision: "trusted",
  coveragePercent: 0,
  freshnessPercent: 0,
  integrityPercent: 0,
```

```
  dependencyFitnessPercent: 0,
  missingArtifacts: [],
  staleArtifacts: [],
  brokenReferences: [],
  unresolvedConflicts: [],
  unsupportedDependents: [],
  upstreamDependencies: [],
  downstreamDependents: [],
  blockerImpact: [],
  recommendedAction: "",
  urgency: "low",
  suggestedMaintenanceMode: "MM-01",
},
"LIB-05": {
  id: "LIB-05",
  name: "",
  shortName: "",
  route: "",
  owner: "",
  purpose: "",
  estateRole: "",
  authoritySource: "",
  authorityStatus: "partial",
  trustDecision: "conditional",
  coveragePercent: 0,
  freshnessPercent: 0,
  integrityPercent: 0,
  dependencyFitnessPercent: 0,
  missingArtifacts: [],
  staleArtifacts: [],
  brokenReferences: [],
  unresolvedConflicts: [],
  unsupportedDependents: [],
  upstreamDependencies: [],
  downstreamDependents: [],
  blockerImpact: [],
  recommendedAction: "",
  urgency: "high",
  suggestedMaintenanceMode: "MM-04",
},
"LIB-06": {
  id: "LIB-06",
  name: "",
  shortName: "",
```

```
    route: "",
    owner: "",
    purpose: "",
    estateRole: "",
    authoritySource: "",
    authorityStatus: "missing",
    trustDecision: "untrusted",
    coveragePercent: 0,
    freshnessPercent: 0,
    integrityPercent: 0,
    dependencyFitnessPercent: 0,
    missingArtifacts: [],
    staleArtifacts: [],
    brokenReferences: [],
    unresolvedConflicts: [],
    unsupportedDependents: [],
    upstreamDependencies: [],
    downstreamDependents: [],
    blockerImpact: [],
    recommendedAction: "",
    urgency: "critical",
    suggestedMaintenanceMode: "MM-21",
  },
  "LIB-07": {
    id: "LIB-07",
    name: "",
    shortName: "",
    route: "",
    owner: "",
    purpose: "",
    estateRole: "",
    authoritySource: "",
    authorityStatus: "authoritative",
    trustDecision: "trusted",
    coveragePercent: 0,
    freshnessPercent: 0,
    integrityPercent: 0,
    dependencyFitnessPercent: 0,
    missingArtifacts: [],
    staleArtifacts: [],
    brokenReferences: [],
    unresolvedConflicts: [],
    unsupportedDependents: [],
    upstreamDependencies: [],
```

```
  downstreamDependents: [],
  blockerImpact: [],
  recommendedAction: "",
  urgency: "low",
  suggestedMaintenanceMode: "MM-01",
},
"LIB-08": {
  id: "LIB-08",
  name: "",
  shortName: "",
  route: "",
  owner: "",
  purpose: "",
  estateRole: "",
  authoritySource: "",
  authorityStatus: "disputed",
  trustDecision: "untrusted",
  coveragePercent: 0,
  freshnessPercent: 0,
  integrityPercent: 0,
  dependencyFitnessPercent: 0,
  missingArtifacts: [],
  staleArtifacts: [],
  brokenReferences: [],
  unresolvedConflicts: [],
  unsupportedDependents: [],
  upstreamDependencies: [],
  downstreamDependents: [],
  blockerImpact: [],
  recommendedAction: "",
  urgency: "high",
  suggestedMaintenanceMode: "MM-20",
},
"LIB-09": {
  id: "LIB-09",
  name: "",
  shortName: "",
  route: "",
  owner: "",
  purpose: "",
  estateRole: "",
  authoritySource: "",
  authorityStatus: "partial",
  trustDecision: "conditional",
```

```
  coveragePercent: 0,
  freshnessPercent: 0,
  integrityPercent: 0,
  dependencyFitnessPercent: 0,
  missingArtifacts: [],
  staleArtifacts: [],
  brokenReferences: [],
  unresolvedConflicts: [],
  unsupportedDependents: [],
  upstreamDependencies: [],
  downstreamDependents: [],
  blockerImpact: [],
  recommendedAction: "",
  urgency: "moderate",
  suggestedMaintenanceMode: "MM-03",
},
"LIB-10": {
  id: "LIB-10",
  name: "",
  shortName: "",
  route: "",
  owner: "",
  purpose: "",
  estateRole: "",
  authoritySource: "",
  authorityStatus: "authoritative",
  trustDecision: "trusted",
  coveragePercent: 0,
  freshnessPercent: 0,
  integrityPercent: 0,
  dependencyFitnessPercent: 0,
  missingArtifacts: [],
  staleArtifacts: [],
  brokenReferences: [],
  unresolvedConflicts: [],
  unsupportedDependents: [],
  upstreamDependencies: [],
  downstreamDependents: [],
  blockerImpact: [],
  recommendedAction: "",
  urgency: "low",
  suggestedMaintenanceMode: "MM-01",
},
"LIB-11": {
```

```
  id: "LIB-11",
  name: "",
  shortName: "",
  route: "",
  owner: "",
  purpose: "",
  estateRole: "",
  authoritySource: "",
  authorityStatus: "partial",
  trustDecision: "conditional",
  coveragePercent: 0,
  freshnessPercent: 0,
  integrityPercent: 0,
  dependencyFitnessPercent: 0,
  missingArtifacts: [],
  staleArtifacts: [],
  brokenReferences: [],
  unresolvedConflicts: [],
  unsupportedDependents: [],
  upstreamDependencies: [],
  downstreamDependents: [],
  blockerImpact: [],
  recommendedAction: "",
  urgency: "high",
  suggestedMaintenanceMode: "MM-03",
},
"LIB-12": {
  id: "LIB-12",
  name: "",
  shortName: "",
  route: "",
  owner: "",
  purpose: "",
  estateRole: "",
  authoritySource: "",
  authorityStatus: "authoritative",
  trustDecision: "trusted",
  coveragePercent: 0,
  freshnessPercent: 0,
  integrityPercent: 0,
  dependencyFitnessPercent: 0,
  missingArtifacts: [],
  staleArtifacts: [],
  brokenReferences: [],
```

```
    unresolvedConflicts: [],
    unsupportedDependents: [],
    upstreamDependencies: [],
    downstreamDependents: [],
    blockerImpact: [],
    recommendedAction: "",
    urgency: "low",
    suggestedMaintenanceMode: "MM-01",
  },
  "LIB-13": {
    id: "LIB-13",
    name: "",
    shortName: "",
    route: "",
    owner: "",
    purpose: "",
    estateRole: "",
    authoritySource: "",
    authorityStatus: "partial",
    trustDecision: "conditional",
    coveragePercent: 0,
    freshnessPercent: 0,
    integrityPercent: 0,
    dependencyFitnessPercent: 0,
    missingArtifacts: [],
    staleArtifacts: [],
    brokenReferences: [],
    unresolvedConflicts: [],
    unsupportedDependents: [],
    upstreamDependencies: [],
    downstreamDependents: [],
    blockerImpact: [],
    recommendedAction: "",
    urgency: "high",
    suggestedMaintenanceMode: "MM-20",
  },
  "LIB-14": {
    id: "LIB-14",
    name: "",
    shortName: "",
    route: "",
    owner: "",
    purpose: "",
    estateRole: "",
```

```
    authoritySource: "",
    authorityStatus: "authoritative",
    trustDecision: "trusted",
    coveragePercent: 0,
    freshnessPercent: 0,
    integrityPercent: 0,
    dependencyFitnessPercent: 0,
    missingArtifacts: [],
    staleArtifacts: [],
    brokenReferences: [],
    unresolvedConflicts: [],
    unsupportedDependents: [],
    upstreamDependencies: [],
    downstreamDependents: [],
    blockerImpact: [],
    recommendedAction: "",
    urgency: "low",
    suggestedMaintenanceMode: "MM-01",
  },
  "LIB-15": {
    id: "LIB-15",
    name: "",
    shortName: "",
    route: "",
    owner: "",
    purpose: "",
    estateRole: "",
    authoritySource: "",
    authorityStatus: "disputed",
    trustDecision: "untrusted",
    coveragePercent: 0,
    freshnessPercent: 0,
    integrityPercent: 0,
    dependencyFitnessPercent: 0,
    missingArtifacts: [],
    staleArtifacts: [],
    brokenReferences: [],
    unresolvedConflicts: [],
    unsupportedDependents: [],
    upstreamDependencies: [],
    downstreamDependents: [],
    blockerImpact: [],
    recommendedAction: "",
    urgency: "high",
```

```
    suggestedMaintenanceMode: "MM-20",
  },
  "LIB-16": {
    id: "LIB-16",
    name: "",
    shortName: "",
    route: "",
    owner: "",
    purpose: "",
    estateRole: "",
    authoritySource: "",
    authorityStatus: "partial",
    trustDecision: "conditional",
    coveragePercent: 0,
    freshnessPercent: 0,
    integrityPercent: 0,
    dependencyFitnessPercent: 0,
    missingArtifacts: [],
    staleArtifacts: [],
    brokenReferences: [],
    unresolvedConflicts: [],
    unsupportedDependents: [],
    upstreamDependencies: [],
    downstreamDependents: [],
    blockerImpact: [],
    recommendedAction: "",
    urgency: "moderate",
    suggestedMaintenanceMode: "MM-03",
  },
};

export const CONTROL_ACTIONS: ControlAction[] = [
  { id: "act-health", label: "Run Health Check", icon: "HeartPulse", actionType: "health-check",
enabled: true },
  { id: "act-coverage", label: "Run Coverage Audit", icon: "ScanSearch", actionType:
"coverage-audit", enabled: true },
  { id: "act-drift", label: "Run Drift Detection", icon: "GitCompareArrows", actionType:
"drift-detection", enabled: true },
  { id: "act-registry", label: "Rebuild Registries", icon: "DatabaseZap", actionType:
"rebuild-registries", enabled: true },
  { id: "act-authority", label: "Review Authority Gaps", icon: "ShieldAlert", actionType:
"review-authority-gaps", enabled: true },
  { id: "act-refresh", label: "Refresh Dependencies", icon: "RefreshCcw", actionType:
"dependency-refresh", enabled: true },
```

```
  { id: "act-unsafe", label: "Open Unsafe Libraries", icon: "TriangleAlert", actionType:
"open-unsafe", enabled: true },
  { id: "act-blockers", label: "Resolve Blockers", icon: "Hammer", actionType: "resolve-blockers",
enabled: true },
];

// Reassign these after your real data is filled.
export const HEALTH_BANDS: LibraryHealthBand[] = [
  { id: "authoritative", label: "Authoritative", count: 0, libraryIds: [], recommendedAction: "" },
  { id: "stable", label: "Stable", count: 0, libraryIds: [], recommendedAction: "" },
  { id: "incomplete", label: "Incomplete", count: 0, libraryIds: [], recommendedAction: "" },
  { id: "stale", label: "Stale", count: 0, libraryIds: [], recommendedAction: "" },
  { id: "fragmented", label: "Fragmented", count: 0, libraryIds: [], recommendedAction: "" },
  { id: "untrusted", label: "Untrusted", count: 0, libraryIds: [], recommendedAction: "" },
];

// Replace these with real prioritized items after library data is filled.
export const LEVERAGE_FIXES: LeverageFix[] = [];

// Replace these with real blockers.
export const ACTIVE_BLOCKERS: ActiveBlocker[] = [];

// Replace these with real risk summaries.
export const SYSTEM_RISKS: SystemRisk[] = [];

export const GOVERNANCE_RULES: GovernanceRule[] = [
  {
    id: "rule-01",
    title: "One authoritative source per governed subject",
    description: "Every governed library area must resolve to one canonical authority source.",
    severity: "strict",
  },
  {
    id: "rule-02",
    title: "Stale authority cannot silently govern runtime",
    description: "Freshness decay forces review or remediation before trust continues.",
    severity: "strict",
  },
  {
    id: "rule-03",
    title: "No template or derivative control without source backing",
    description: "Derived governance artifacts require valid authority support.",
    severity: "strict",
  },
```

```
  {
    id: "rule-04",
    title: "Registry integrity required before trust restoration",
    description: "Libraries cannot return to governed state with broken registry links.",
    severity: "strict",
  },
  {
    id: "rule-05",
    title: "Unresolved conflicts force review-required or unsafe state",
    description: "Authority disputes must never remain invisible to operators.",
    severity: "warning",
  },
  {
    id: "rule-06",
    title: "Dependency-critical libraries require fresh audit evidence",
    description: "High-pressure libraries need tighter audit cadence than low-pressure libraries.",
    severity: "warning",
  },
];

export const MAINTENANCE_MODES: MaintenanceModeCard[] = [
  {
    id: "MM-01",
    name: "Health Check",
    purpose: "Run a broad estate health pass across libraries.",
    triggerCondition: "Use when you need an overall health baseline.",
    suggestedForStates: ["governed", "review-required", "degraded"],
    launchable: true,
  },
  {
    id: "MM-03",
    name: "Coverage Audit",
    purpose: "Validate completeness and identify coverage gaps.",
    triggerCondition: "Use when coverage percent is low or incomplete.",
    suggestedForStates: ["review-required", "degraded", "recovery"],
    launchable: true,
  },
  {
    id: "MM-04",
    name: "Drift Detection",
    purpose: "Detect divergence between intended governance and current library state.",
    triggerCondition: "Use when freshness declines or drift alerts appear.",
    suggestedForStates: ["review-required", "degraded"],
    launchable: true,
```

},
{
id: "MM-12",
name: "Rebuild Indexes/Registries",
purpose: "Regenerate indexes and registries supporting control surfaces.",
triggerCondition: "Use when registry pathing or reference integrity is weak.",
suggestedForStates: ["degraded", "blocked", "recovery"],
launchable: true,
},
{
id: "MM-13",
name: "Registry Integrity Cleanup",
purpose: "Repair registry-level integrity issues and broken authority linkages.",
triggerCondition: "Use when authority is missing or registry references are broken.",
suggestedForStates: ["review-required", "unsafe", "recovery"],
launchable: true,
},
{
id: "MM-17",
name: "Dependency Refresh",
purpose: "Refresh dependency assumptions and supporting references.",
triggerCondition: "Use when dependency pressure or cross-library fragility is high.",
suggestedForStates: ["degraded", "blocked"],
launchable: true,
},
{
id: "MM-18",
name: "Workspace Compatibility Check",
purpose: "Validate that library state remains compatible across connected workspaces.",
triggerCondition: "Use after major updates or suspected compatibility drift.",
suggestedForStates: ["review-required", "degraded"],
launchable: true,
},
{
id: "MM-20",
name: "Failure Triage Mode",
purpose: "Contain, inspect, and prioritize active failures.",
triggerCondition: "Use when blockers or authority disputes are actively affecting governance.",
suggestedForStates: ["blocked", "unsafe"],
launchable: true,
},
{
id: "MM-21",

```
    name: "Emergency Recovery Mode",
    purpose: "Recover control authority after severe integrity or trust failure.",
    triggerCondition: "Use for critical authority or registry failures.",
    suggestedForStates: ["unsafe", "recovery"],
    launchable: true,
  },
];

// Replace with real dependency links.
export const DEPENDENCY_LINKS: DependencyLink[] = [];

// Replace with real recent events.
export const RECENT_ACTIVITY: LibraryActivityEvent[] = [];

export const DEFAULT_FILTERS: LibraryControlFilters = {
  query: "",
  authorityStatus: undefined,
  controlState: undefined,
  risk: undefined,
  staleOnly: false,
  blockedOnly: false,
  missingAuthorityOnly: false,
  driftOnly: false,
  sortBy: "risk",
};

export const libraryControlCenterState: LibraryControlCenterState = {
  header: getEstateHeaderStats(LIBRARIES),
  metrics: getEstateStatusMetrics(LIBRARIES),
  actions: CONTROL_ACTIONS,
  healthBands: HEALTH_BANDS,
  libraries: LIBRARIES,
  libraryDetails: LIBRARY_DETAILS,
  leverageFixes: LEVERAGE_FIXES,
  blockers: ACTIVE_BLOCKERS,
  risks: SYSTEM_RISKS,
  governanceRules: GOVERNANCE_RULES,
  maintenanceModes: MAINTENANCE_MODES,
  dependencyLinks: DEPENDENCY_LINKS,
  recentActivity: RECENT_ACTIVITY,
  defaultFilters: DEFAULT_FILTERS,
};
```

Tab 16